# Jet Ranger Cockpit User Guide

Setting up, running, wiring, flashing and troubleshooting the Jet Ranger panel

Regenerated 31 October 2026

# Contents

1 Overview....5
2 Windows and simulator....6
2.1 Starting up....6
2.2 Running the MSFS bridge....7
2.2.1 Aircraft with estimated dials....8
2.3 Running the FSUIPC bridge (Prepar3D and FSX)....9
2.4 Installing the DodoSim event file (FSUIPC)....9
2.4.1 Event naming....10
2.4.2 Event groups at a glance....10
3 Colour codes and cabling....11
3.1 Reading the diagrams....11
3.2 Colours in the programs....11
3.3 Cabling colour codes....12
3.3.1 Round power connectors....12
3.3.2 PCB connectors....12
3.4 OLED connectors....12
4 Lowrance AirMap GPS....13
4.1 Setting it up....13
4.2 What the GPS is told....13
4.3 Testing without the simulator....14
5 Program windows....15
5.1 Bridges....16
5.2 Health Monitor....18
5.3 SwingTester....21
5.4 StepperVSITester....23
5.5 Emulators....26
5.6 GPS interface....28
5.7 GPS bench tester....29
6 Network....30
6.1 Packet format....31
6.2 What a board does....32
6.3 Unreachable hosts and the fast Ethernet timeout....33
6.4 When the panel loses power....33
6.5 Backlight....34
7 Arduino servo controller....35
7.1 Uploading the firmware....35
7.2 Servos....35
7.3 Other pins....36
7.4 What happens at power-up....36
7.5 Normal running....36
8 Stepper controller (13 Stepper board)....38

8.1 Behaviour....38
8.2 Testing a gauge on demand....39
8.3 Known issues....39
9 OLED dual stepper controller....41
9.1 OLED displays....42
9.2 Behaviour....42
9.3 Known issues....42
10 Radio controller (COM and NAV)....43
10.1 Boards....43
10.2 Inputs and outputs....43
10.3 Data from the PC (UDP 13136)....44
10.4 Commands to the PC (UDP 27001)....45
10.5 Auto Pilot and Audio panels....46
10.6 Auto Pilot and Audio panel LEDs....47
10.7 Huey Warning Panel....47
10.8 Auto Pilot display....48
10.9 When the USB HID board is not connected....48
10.10 Backlight....48
10.11 Display behaviour....48
10.12 Known issues....49
11 Joystick (HID) interface....50
11.1 How a switch becomes a joystick button....50
11.2 Auto Pilot, Audio and ADF panel buttons....51
11.3 Switch mapping in use....51
11.4 Setting FSUIPC events from the joystick buttons....52
11.5 Known issues....53
12 ADF and Upper controller....54
12.1 Using the panel....54
12.2 Inputs and outputs....55
12.3 Panel switches and joystick buttons....56
12.4 Upper panel switches and keypad....56
12.5 Landing gear lamps....56
12.6 Power status....56
12.7 Messages to the PC (UDP 27003)....57
12.8 Control packets....57
12.9 Panel blanking and backlight....57
12.10 Checking the panel for the first time....57
13 COM ports and flashing....59
13.1 COM port to Arduino mapping....59
13.2 Flashing a board....60
14 PCBs and wiring....61
14.1 Backlight MOSFET (revisions of 18 October 2026)....62
14.1.1 Backlight resistor values....62

14.2 Connections on the servo carrier board....63
14.3 Wiring checklist....63
15 Calibration....64
15.1 Calibrating a servo....64
15.2 Calibrating a stepper gauge....64
16 Troubleshooting....65
16.1 Known issues in the code....66
16.2 Open questions....66
17 Reference: UDP tokens....67
17.1 Gauge tokens....67
17.2 Lamp tokens (servo board)....67
17.3 Aircraft information to the GPS interface (UDP 27002)....67
17.4 Status and control tokens....68
18 PCB schematics and layouts....69
18.1 Jet Ranger Radio Auto Pilot....70
18.2 Jet Ranger COM and NAV....72
18.3 Jet Ranger Front Panel Servo....74
18.4 Jet Ranger 13 Stepper....76
18.5 Jet Ranger OLED Dual Stepper....78
18.6 Jet Ranger X27 Small Stepper....80
18.7 Jet Ranger Upper ADF Controller....82
18.8 Jet Ranger Audio Panel....84
18.9 Jet Ranger Auto Pilot Panel....86
18.10 Jet Ranger ADF Panel....88
18.11 Annunciator....90
18.12 Jet Ranger Altimeter....91
18.13 Large Guage (Simmeters Large Guage Replacement)....93
18.14 Simkits Large Dual Gauge....95
18.15 Simkits Large Guage....97
18.16 Simkits Small Dual Gauge....99
18.17 Simkits Small Guage....101

# 1 Overview

The Jet Ranger cockpit drives real gauge needles, warning lights, radio displays and switches from the simulator. A Windows PC exchanges UDP packets over Ethernet with Arduino Mega 2560 panel boards, and a small USB board presents the panel switches to Windows as joysticks.

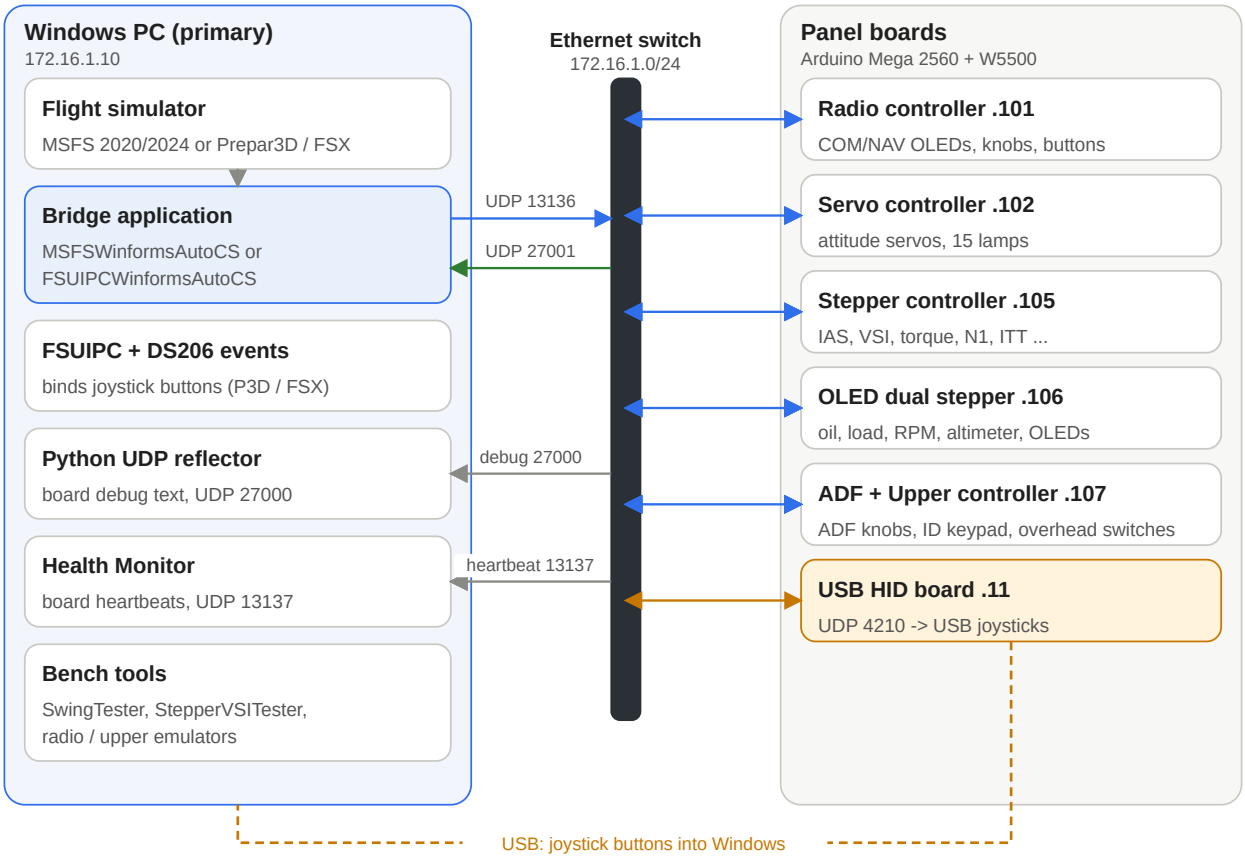



*Figure 1 — The whole system*

Data flows from the PC to the boards as short text packets. On the primary PC a Python listener shows the boards' debug messages and a separate Windows app, the Health Monitor, tracks their keep-alives. Cockpit switches reach the simulator separately, as joystick buttons (and, in Prepar3D and FSX, FSUIPC events).

| Part | Hardware and software | Address | Section |
|---|---|---|---|
| Primary PC | Windows, MSFS 2020 or 2024, or Prepar3D / FSX with DodoSim 206 V2, one bridge program, debug listener, Health Monitor | 172.16.1.10 | Windows and simulator |
| Radio controller | Mega 2560 with the Radio Auto Pilot PCB and the COM and NAV panel | 172.16.1.101 | Radio controller |
| Servo board | Mega 2560 and Ethernet module on the Front Panel Servo PCB | 172.16.1.102 | Servo controller |
| ADF and Upper controller | Mega 2560 on the combined Upper ADF Controller PCB: the ADF knobs, aircraft ID keypad, gear, flaps, speed brake, brakes, anti-ice, hyd boost and caution dim switches, landing gear lamps, and the overhead switch matrix and keypad of the old Upper controller (172.16.1.103, retired) | 172.16.1.107 | ADF and Upper controller, Joystick (HID) interface |
| Stepper board | Mega 2560 on the Jet Ranger 13 Stepper PCB | 172.16.1.105 | Stepper controller |
| Dual stepper board | Mega 2560 on the OLED Dual Stepper PCB | 172.16.1.106 | Dual stepper controller |
| USB HID board | Leonardo-class board with native USB | 172.16.1.11 | Joystick (HID) interface |

# 2 Windows and simulator

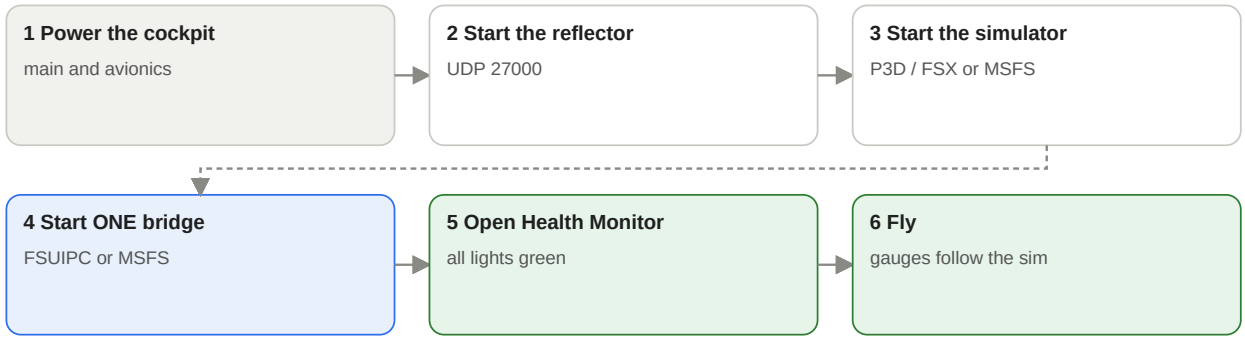
The Windows PC runs the simulator and a **bridge program** that turns simulator variables into UDP text packets for the Arduino boards. Switch inputs travel the other way as joystick buttons.

The bridge sends the Servo, Stepper and Dual Stepper boards each their own packet in **real units** (degrees, knots, feet per minute, psi, percent, degrees C), and the boards convert them with their own calibration. It sends about 10 packets per second to each board; the radio gets a packet whenever a frequency changes and at least every 5 seconds. The Dual Stepper's temperature OLED shows the outside air temperature from the simulator.

| Component | Role | Notes |
|---|---|---|
| Simulator | Flies the DodoSim 206 or another Bell 206 Jet Ranger | MSFS 2020 or 2024, Prepar3D or FSX |
| FSUIPCWinformsAutoCS | Bridge for Prepar3D and FSX, reads FSUIPC offsets | Needs FSUIPC |
| MSFSWinformsAutoCS | Bridge for MSFS, reads SimConnect | Has estimates for the FlyInside B206 and CowaSim Jet Ranger |
| FSUIPC and DODSIM206V2.EVT | Exposes 234 DodoSim custom events so switches can be bound | Prepar3D and FSX only |
| Python debug listener | Shows debug messages from the boards | Listens on UDP 27000 |
| Health Monitor | Tracks each board's keep-alive and flags a board that goes quiet | Listens on UDP 13137 |
| LowranceAirMapGps | Feeds a Lowrance AirMap 2000C GPS from the simulator over a serial port | Listens on UDP 27002; see Lowrance AirMap GPS |

Run **only one bridge at a time**: both listen for the radio board's commands on UDP 27001. Both bridges also send the aircraft position, altitude, airspeed, heading and simulator date and time to the GPS interface on UDP 27002 (on the same PC).

## 2.1 Starting up



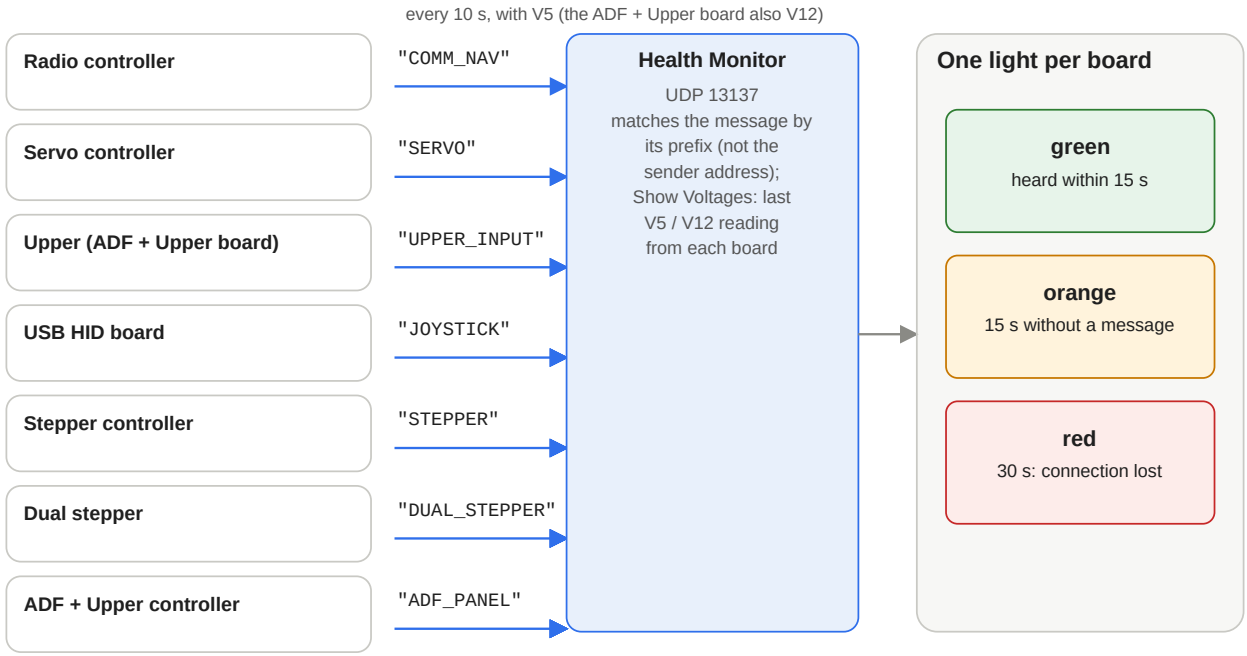

Order matters little: every board waits for data, and the bridge reconnects to the simulator on its own every second.

If a gauge stays at zero for 30 s without data, the board's watchdog parked it on purpose.

*Figure 2 — Start-up order*

1. Power the cockpit (main and avionics).
2. Start the debug listener (UDP 27000).
3. Start the simulator.
4. Start **one** bridge.
5. Open the Health Monitor: every light should turn green within about 10 seconds.
6. Fly. The gauges follow the simulator.

The order matters little: every board waits for data and the bridge reconnects to the simulator on its own every second. A gauge that stays at zero for 30 seconds without data was parked on purpose by the board's watchdog.

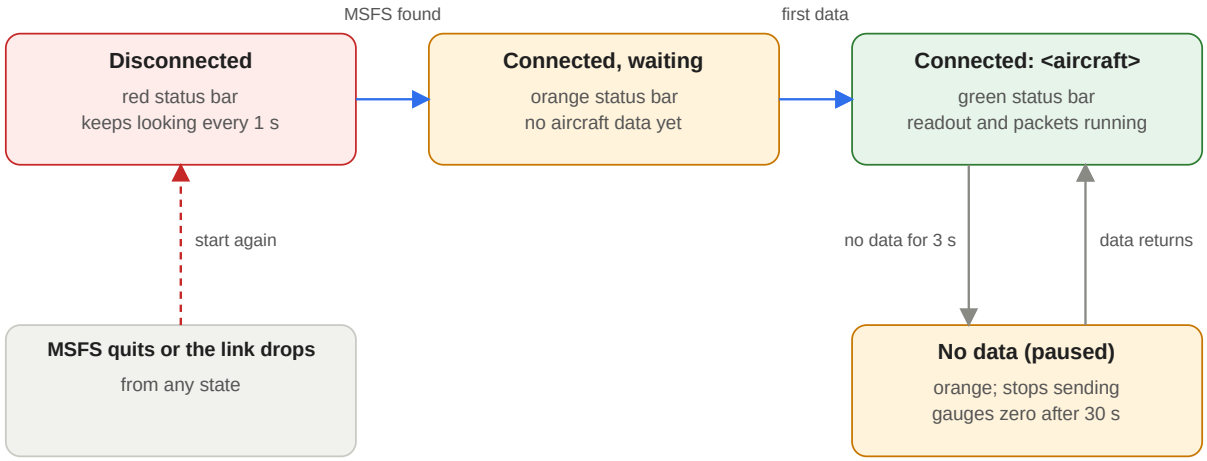



The monitor only listens; it never sends anything. It does not care which bridge is running.

*Figure 3 — Health Monitor*

## 2.2 Running the MSFS bridge

1. Build the project `MSFSWinformsAutoCS` (in the repository's `C# Code` folder) and run `MSFSWinformsAutoCS.exe` from its `bin\Debug` folder. Build output is not stored in git, so a fresh copy of the repository has to be built first.
2. Close the FSUIPC bridge if it is running, then start MSFS and load a Jet Ranger. The status bar changes from *Disconnected* (red) to *Connected to MSFS* with the aircraft name (green) once data arrives.
3. The window lists the values being sent and the warning lamps. The Avionics Master box follows the simulator and can also be used to switch it.
4. If the status turns orange and says no data, the simulator is paused or loading and the bridge has stopped sending; the boards return their gauges to zero after 30 seconds and carry on when data returns.
5. Five lamps are driven: engine out, rotor low, low fuel, generator fail (engine speed below 10 percent) and transmission oil pressure (below 40 psi, using the estimated pressure on the FlyInside and CowaSim). The other lamps need aircraft-specific variables. Some MSFS Jet Rangers do not report every value (for example transmission pressure and temperature), so those gauges may stay at zero. With the FlyInside B206 the estimated transmission pressure (35 psi) is below the 40 psi threshold, so that lamp stays on.
6. The status bar shows the aircraft name the simulator reports and follows it when you load a different one.
7. The electrical load follows the size of the total load whichever way the current flows, so with the engine stopped it still shows a load.




A new aircraft is picked up while connected: the name in the status bar changes and the estimate models restart.

SimConnect errors are caught on every path, so a closed simulator never stops the bridge. FSUIPCWinformsAutoCS behaves the same way with FSUIPC.

*Figure 4 — Bridge status*

## 2.2.1 Aircraft with estimated dials

| Value | Standard SimVars | FlyInside B206 | CowaSim Jet Ranger |
|---|---|---|---|
| **Rotor, engine speed** | **ROTOR RPM PCT,** GENERAL ENG PCT MAX RPM | **L:Aircraft.Rotor.RPM.Percent,** L:...Turbine.N2.Smooth | **standard** |
| **Gas producer (N1)** | **TURB ENG CORRECTED N1** | **0.92 x turbine N2** | **standard** |
| **Torque (TQ)** | **ENG TORQUE PERCENT** | **collective lever curve,** up to 120 % | **0.42 x SimVar,** smoothed 2.5 s |
| **Turbine out temp (ITT)** | **TURB ENG ITT** | **6 x N2 + 1.05 x** (torque - 21) | **EGT - 200 C** |
| **Fuel load (psi)** | **ENG FUEL PRESSURE** | **10 x SimVar** | **standard** |
| **Electrical load (%)** | **\|total amps\| / 300 A** | **\|total amps\| / 300 A** | **\|total amps\| / 139 A** |
| **Transmission pressure** | **SimVar (usually 0)** | **35 psi x rotor fraction** | **48 psi x rotor fraction** |
| **Transmission temp** | **SimVar (usually 0)** | **slow warm-up from air temp** | **0.92 x engine oil temp** |
| **Recognised by** | **otherwise** | **turbine L:var, or title** contains FlyInside | **title contains 206B3** |

Green = the standard SimVar is used unchanged. Estimates come from a few dial readings and may be a few percent off.

*Figure 5 — Which values come from where*

**FlyInside B206.** Its own gauges show values that are not in the standard simulator variables: rotor speed, engine speed, gas producer, torque, turbine out temperature, fuel pressure and transmission oil. The bridge reads the aircraft's own rotor and engine speed variables and estimates the rest from cockpit dial readings: torque follows the collective lever, turbine out temperature rises with torque, and the transmission oil pressure and temperature are modelled (the temperature warms up slowly from the outside air temperature and starts again from cold each time the bridge restarts). In this aircraft holding full collective for more than about 2 seconds over-torques the engine and breaks it. Its cockpit load dial stays at about 50 percent whatever the load.




Both curves were fitted to dial readings from screenshots; check them against the cockpit at other power settings.

FlyInside above lever 73.6 climbs at about 1.74 points per percent; the engine over-torques if full collective is held for 2 s.

*Figure 6 — Torque estimates*

**CowaSim Jet Ranger** (shown as `206B3` with its registration). Most gauges match the standard variables. Its torque, turbine out temperature and electrical load dials do not, and it has no transmission oil values: the bridge smooths the torque, takes the turbine out temperature from the exhaust gas temperature, rescales the load and estimates the transmission oil from rotor speed and engine oil temperature.

These estimates were fitted to a few dial readings and may be a few percent off at other power settings, so compare the panel gauges with the cockpit dials. Other aircraft use the standard variables.

## 2.3 Running the FSUIPC bridge (Prepar3D and FSX)

Start FSUIPC with the simulator, then run `FSUIPCWinformsAutoCS.exe`. The status bar shows when it is connected to Prepar3D. The window lists the values being sent and the 21 annunciator bits. The bridge sends about 10 packets per second to each board and listens on UDP 27001 for the radio board's commands (frequency swaps, standby tuning, battery, alternator, avionics master).

## 2.4 Installing the DodoSim event file (FSUIPC)

Copy `DODSIM206V2.EVT` into the `{P3D or FSX}/Modules` folder, next to `FSUIPC.ini`. Add this to `FSUIPC.ini` if FSUIPC has not added it itself:

```
[EventFiles]
0=DodoSim206
```

Restart the simulator. The `DS206.*` events now appear in FSUIPC's button and key assignment lists. Bind each physical switch to its event. After an EVT update FSUIPC re-indexes events alphabetically, so check and re-map bindings.

### 2.4.1 Event naming

| Suffix | Parameter | Behaviour | Example |
|---|---|---|---|
| `_SET` | Integer (usually 1 = on, 0 = off) | Sets a state directly; best for latching toggle switches | `DS206.BATTERY_SWITCH_SET` |
| `_TOGGLE` | None | Flips the state each press | `DS206.FUEL_VALVE_SWITCH_TOGGLE` |
| `_TRIGGER` | None (some take 1 to 10 as a multiplier) | Momentary; tick Control to repeat while held | `DS206.STARTER_BUTTON_TRIGGER` |
| none | None | One-shot action | `DS206.CLOCK_MODE_BUTTON` |

For physical latching switches prefer the `_SET` events: send 1 on switch-on and 0 on switch-off so the simulator never drifts out of step with the panel.

### 2.4.2 Event groups at a glance

| Group | Event numbers | Examples |
|---|---|---|
| Circuit breakers | 36 to 95, 107 to 110 | Hoist, SAS, fuel pumps, governor, ignition, starter |
| Lights and master switches | 96 to 101, 111 to 122, 209 to 212, 224 to 227 | Battery, generator, avionics, pitot heat, landing lights |
| Engine and flight controls | 1 to 2, 207 to 208, 213 to 223, 228 to 232 | Rotor brake, hydraulics, force trim, starter, governor beep |
| Instruments | 3 to 33 | Annunciator test, AI cage, radar altimeter, HSI and OBS knobs |
| Radios and audio | 129 to 192 | COM and NAV swap and tuning, ADF, DME, audio panel |
| Transponder | 193 to 206 | Mode knob, digit buttons, IDT, VFR |
| SAS modes | 123 to 128 | Master, HDG, NAV, SPD, ALT, VRT |
| Other | 102 to 106, 233 to 234 | Difficulty rotary, heat and vent, external power |

# 3 Colour codes and cabling

This chapter explains the colours used in the diagrams and programs, and the colour codes used for cables and connectors.

## 3.1 Reading the diagrams

Each diagram box is shaded to say what kind of item it is. The same shades are used in every diagram.

| Colour | What it means in the diagrams |
|---|---|
| White | An ordinary item: a board, program, step or detail |
| Blue | The subject of the figure or the main step: the bridge, the Health Monitor, the code that decides |
| Green | Running or healthy, power on, or an output: connected, the gauge or backlight being driven |
| Amber | Waiting, a warning or a special path: no data, the default position, the USB HID path |
| Red | Lost, off, failed or a reset |
| Gray | Supporting or inactive: notes, items that stay off |
| Purple | Tells boards or groups apart where the figure says so (for example the Dual Stepper in the Swing ALL sequence, the Upper controller in the PCB map) |
| Teal | The same, for the ADF controller in the PCB map |
| Dark | Column headers and the Ethernet switch |

Arrows follow the same idea: blue is data going to a board over UDP, green is commands or events coming back, amber is the USB HID path, gray is an ordinary link, and a dashed line is an optional or recovery path.

## 3.2 Colours in the programs

| Where | Colour | Meaning |
|---|---|---|
| Health Monitor light | Green | A heartbeat was heard within the last 15 seconds |
| Health Monitor light | Orange | 15 seconds without a heartbeat (connection warning) |
| Health Monitor light | Red | 30 seconds without a heartbeat (connection lost) |
| Bridge status bar | Red | Disconnected from the simulator |
| Bridge status bar | Orange | Connected but no data (the simulator is paused or loading) |
| Bridge status bar | Green | Connected, with the aircraft name |
| Board status LEDs (pins 14 green, 15 red) | Green and red | Alternate every 200 ms while the sketch runs; red flashes fast while Ethernet starts |
| SwingTester | Gold | The gauge row being swung, or a warning lamp that is on |
| GPS bench tester | Pink | A box whose value could not be read (it is not used) |
| Upper controller emulator | Green | A switch that is on |
| GPS interface | Green | The position comes from aircraft data over UDP |
| GPS interface | Orange | No data for 30 seconds: the Brisbane Airport default is being sent |

# 3.3 Cabling colour codes

## 3.3.1 Round power connectors

| Colour | Supply |
|---|---|
| Red | 12 V |
| Gray | 5 V |

## 3.3.2 PCB connectors

| Colour | Use | Example |
|---|---|---|
| Uncoloured | Digital input | |
| Blue | Analog | |
| Red | 12 V | Magnetic coils |
| Green | 12 V backlighting | |
| Yellow | OLED | |
| Orange | LEDs | |

The two 12 V colours are different on purpose: red is 12 V for coils, green is 12 V for backlighting.

# 3.4 OLED connectors

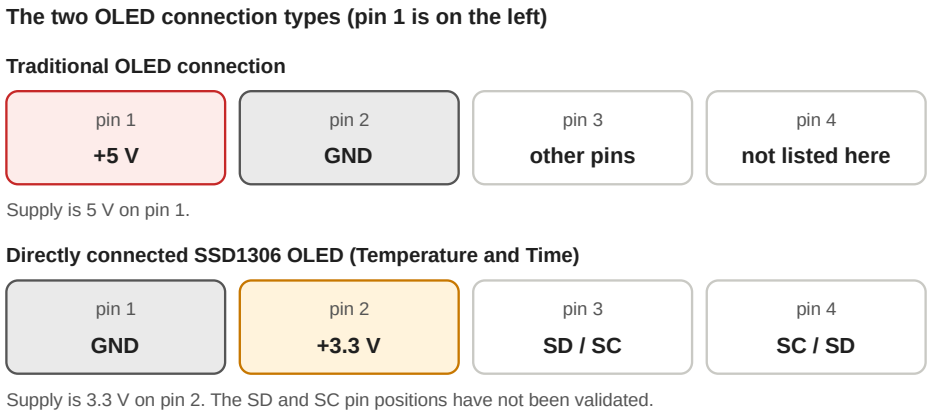



*Figure 7 — The two OLED connection types*

The OLED connectors are **not the same** on every display:

- **Traditional OLED connection:** +5 V on pin 1 and GND on pin 2.
- **Directly connected SSD1306 OLEDs** (the Temperature and Time displays): GND on pin 1 and +3.3 V on pin 2.

The SD and SC (data and clock) pin positions of the directly connected displays have not been validated. Do not plug one type of cable into the other type of connector: pins 1 and 2 are reversed and the supply voltages differ.

# 4 Lowrance AirMap GPS

`LowranceAirMapGps` makes the PC the GPS for a **Lowrance AirMap 2000C**. It receives the aircraft position from the bridge and sends it to the GPS over a serial cable as NMEA sentences twice a second.

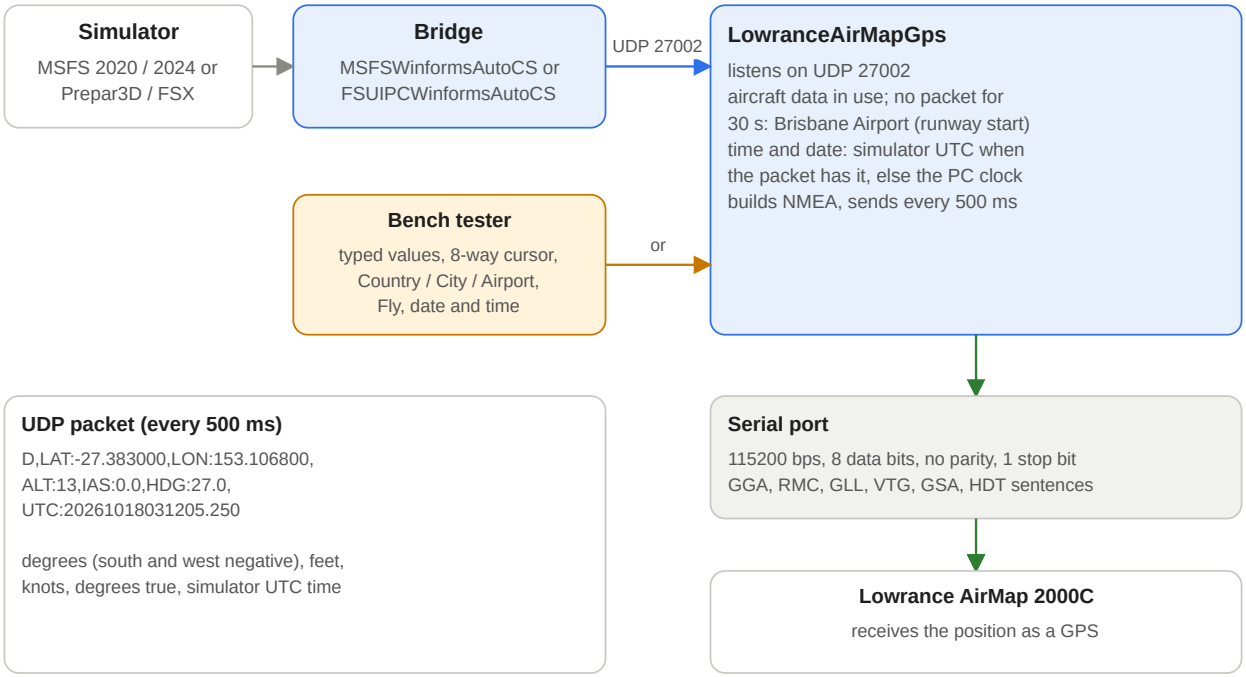



*Figure 8 — From the simulator to the GPS*

## 4.1 Setting it up

1. Connect the GPS to the PC with its serial cable (a USB serial adapter works).
2. Run `LowranceAirMapGps.exe`. The first time it asks which serial port the GPS is on; it remembers the answer (*Change serial port...* changes it later). The port runs at **115200 bps, 8 data bits, no parity, 1 stop bit**.
3. Start the simulator and one bridge on the same PC. The bridge sends the aircraft information to UDP port 27002 without any setting; if the GPS interface runs on a different PC, change `GpsHost` in the bridge's `Form1.cs`.
4. The window shows the serial port state, the number of packets received, where the position comes from, the values being sent and the time source. The sentences being sent are listed at the bottom.

## 4.2 What the GPS is told

- **Position, altitude, airspeed and heading** from the simulator, sent as GGA, RMC, GLL, VTG, GSA and HDT sentences. NMEA has no airspeed sentence a GPS accepts, so the airspeed is sent as the GPS speed.
- **Date and time** are the simulator's UTC, ticking in real time between packets. If the packets have no time, or nothing has arrived for 30 seconds, the PC clock is used. The window says which.
- **No data for 30 seconds** (the simulator is paused, closed or not running): the GPS is told it is at Brisbane Airport, at the start of the runway, speed 0. It returns to the live position as soon as packets arrive again.

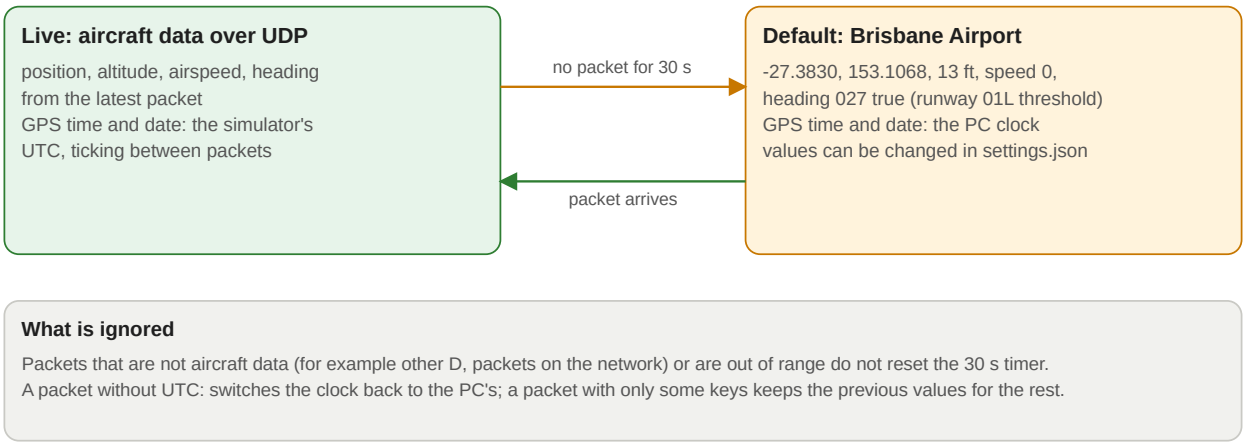



*Figure 9 — Live data and the Brisbane default*

The default position and the UDP port can be changed in `%AppData%\LowranceAirMapGps\settings.json`. The Brisbane coordinates came from SkyVector and were not checked against the AIP.

## 4.3 Testing without the simulator

`LowranceAirMapGpsBenchTester` sends the same packets from a window:

- Choose a **Country, City and Airport** to put the aircraft on the ground at that airport.
- Type the **latitude, longitude, altitude, speed and heading**, and press Enter after each.
- Use the **8-way cursor** (the buttons or the arrow keys) to move the position in steps (default 100 m); hold to repeat.
- Tick **Fly** to move along the heading at the current speed; **Time x** speeds it up (1 to 100).
- Set the **date and time (UTC)** the GPS should show, or press *PC clock*.

Use one sender at a time (a bridge or the tester): a second one just overwrites the position. The GPS interface has not yet been tested with the real AirMap 2000C, so check that the GPS shows the position and that it accepts the 115200 bps rate.

# 5 Program windows

This chapter shows every window of the Windows programs in `Jet Ranger/ C# Code/`. The screenshots were taken from the current builds. The two bridges were captured with no simulator running, so they show their disconnected state; with a simulator connected the left box fills with the values being sent and the list shows the lamps lighting.

| Program | Used for | Section |
|---|---|---|
| FSUIPCWinformsAutoCS | Bridge for Prepar3D and FSX | Bridges |
| MSFSWinformsAutoCS | Bridge for MSFS 2020 and 2024 | Bridges |
| JetRangerHealthMonitor | Shows which boards are alive and the last voltage reading from each | Health Monitor |
| SwingTester | Swings every gauge, lamps, OLEDs, tests the Auto Pilot, Audio and Huey panel LEDs, resets boards, sets brightness | SwingTester |
| StepperVSITester | Sends real and raw values to the stepper boards and the servo | StepperVSITester |
| JetRangerRadioControllerEmulator, JetRangerUpperControllerEmulator | Stand in for the radio and upper boards | Emulators |
| LowranceAirMapGps | Feeds the Lowrance AirMap 2000C | GPS interface |
| LowranceAirMapGpsBenchTester | Sends aircraft data to the GPS interface for testing | GPS bench tester |

> **Careful:** the SwingTester, StepperVSITester and the two emulators send packets to the real boards (the addresses are in the boxes) as soon as you use their controls. Close them, or point them at the right addresses, before you press anything while the cockpit is powered.

## 5.1 Bridges

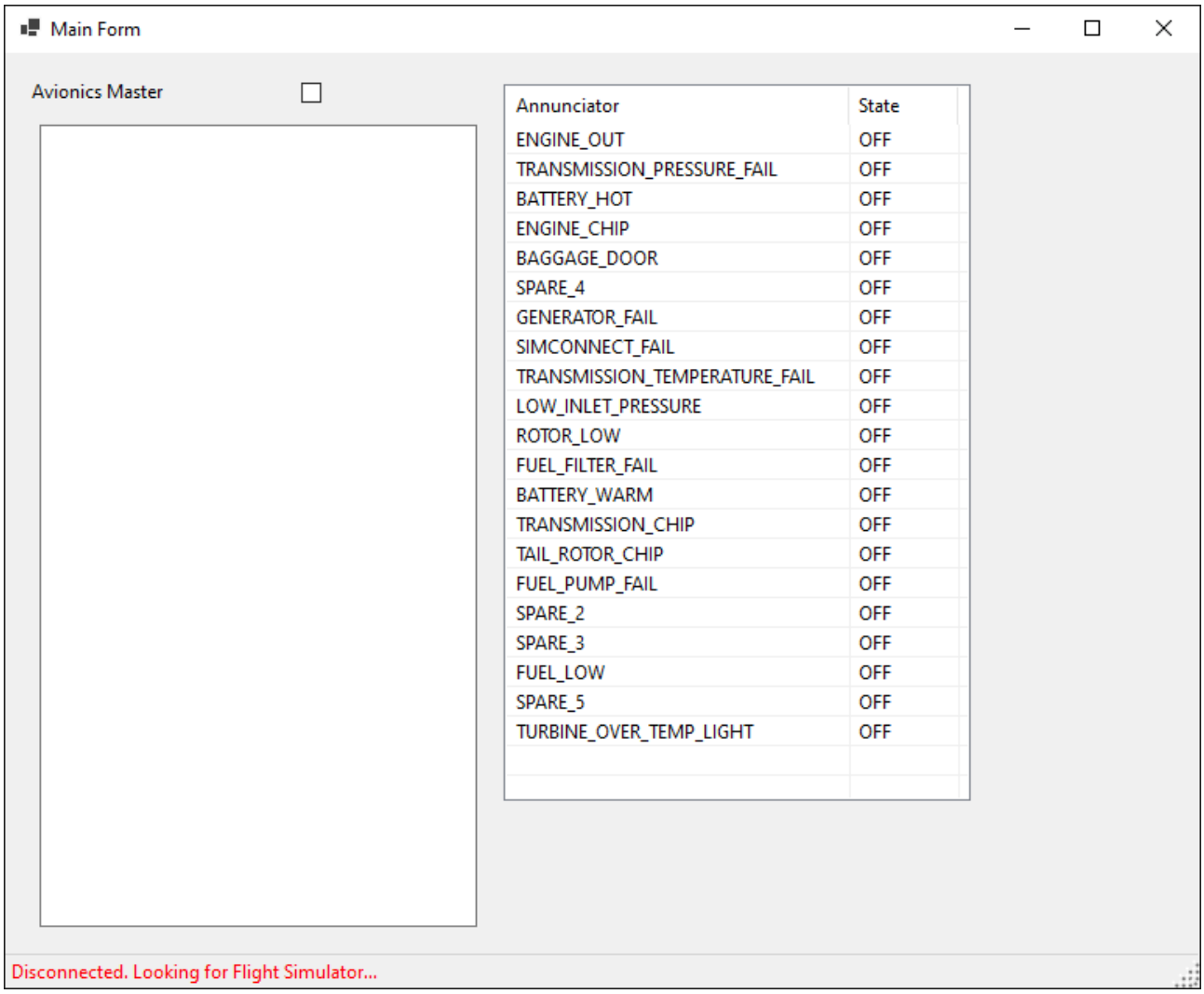



*Figure 10 — FSUIPCWinformsAutoCS (Prepar3D and FSX)*

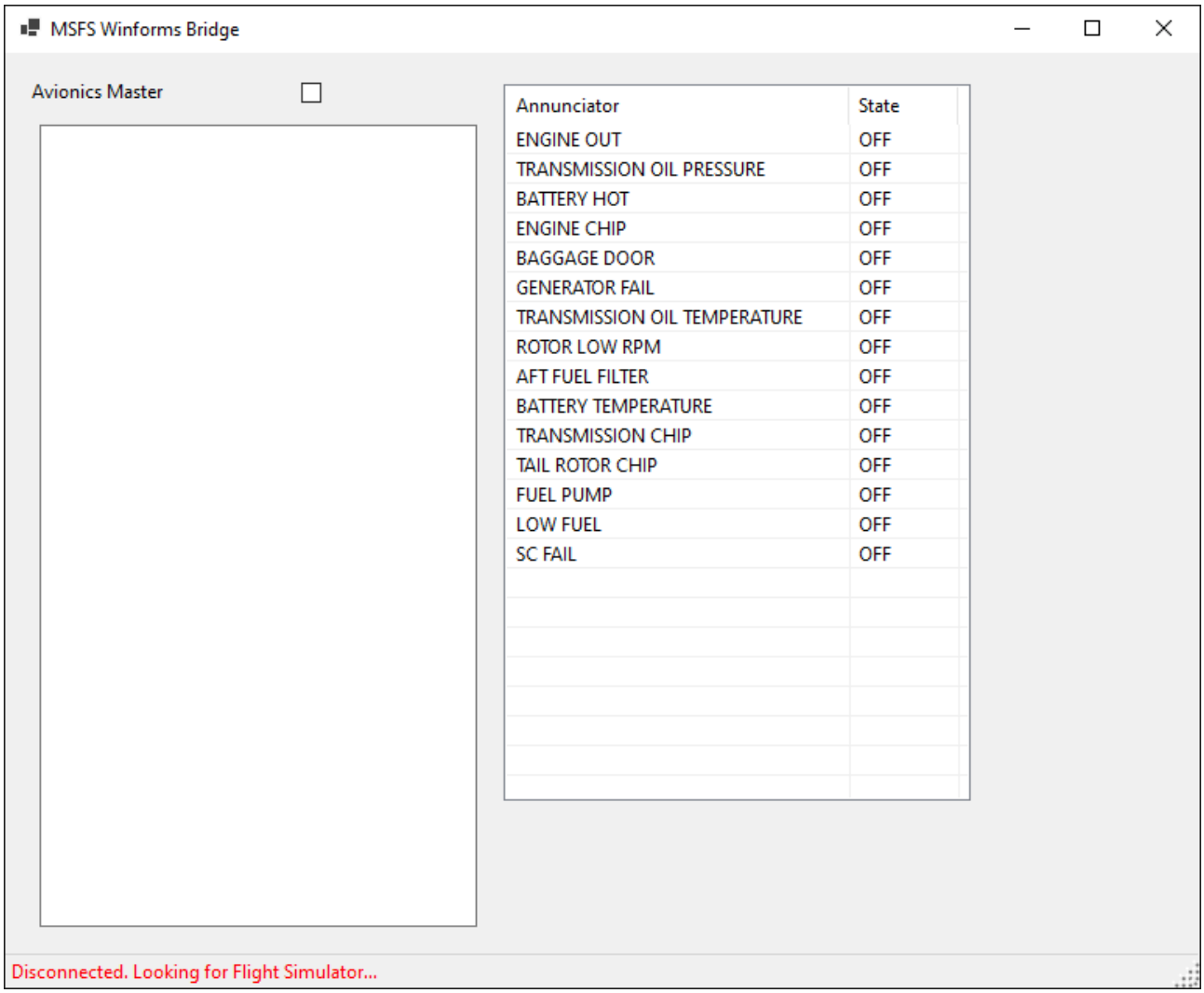



*Figure 11 — MSFSWinformsAutoCS (MSFS 2020 and 2024)*

Both bridges have one window with the same layout.

- **Avionics Master** check box: follows the simulator's avionics master switch and can also be used to switch it.
- **Readout box** (left): the values being sent to the boards, one per line (airspeed, altitude, bus voltages, engine and transmission values, radio frequencies and, in the MSFS bridge, the GPS position and heading). It is empty until the simulator is connected.
- **Annunciator list** (right): every warning lamp with its state, ON or OFF. The FSUIPC bridge lists the 21 lamp bits of the DodoSim aircraft; the MSFS bridge lists the 15 lamps of the panel.
- **Status bar** (bottom): *Disconnected. Looking for Flight Simulator...* in red until the simulator is found, then the connection and (MSFS) the aircraft name in green, or orange when the simulator is paused or loading and no data arrives.

Run only one bridge at a time.

## 5.2 Health Monitor

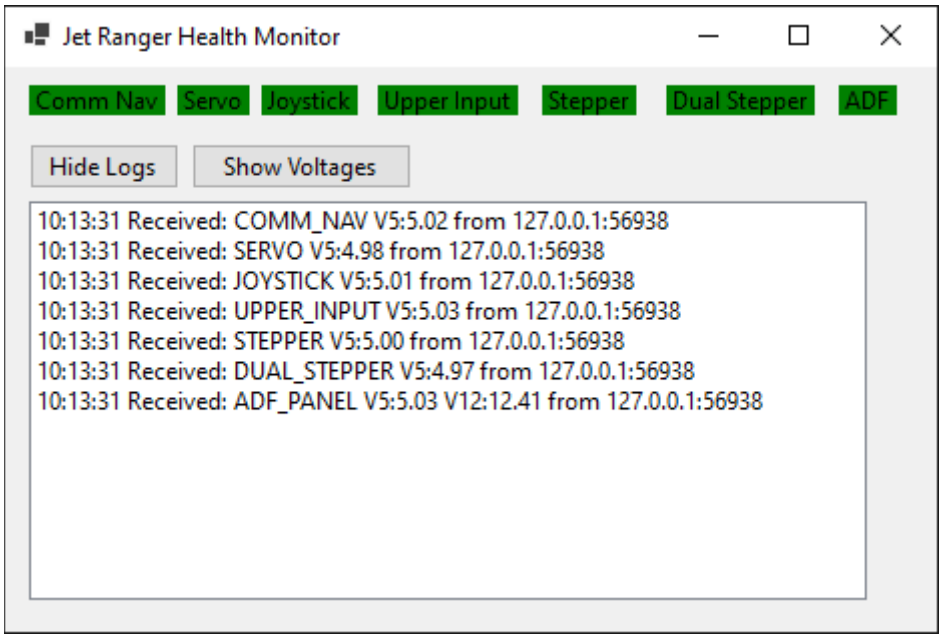



*Figure 12 — JetRangerHealthMonitor: all seven boards heard*

One coloured label per board: Comm Nav, Servo, Joystick, Upper Input, Stepper, Dual Stepper and ADF (the combined ADF and Upper board lights both the Upper Input and ADF labels). Each turns green when that board's 10 second heartbeat arrives on UDP 13137. The **Hide Logs** button hides the list of received heartbeats (the sender address and time of each one) and shrinks the window; it then reads **Show Logs**.

The **Show Voltages** button opens a table with one row per board:

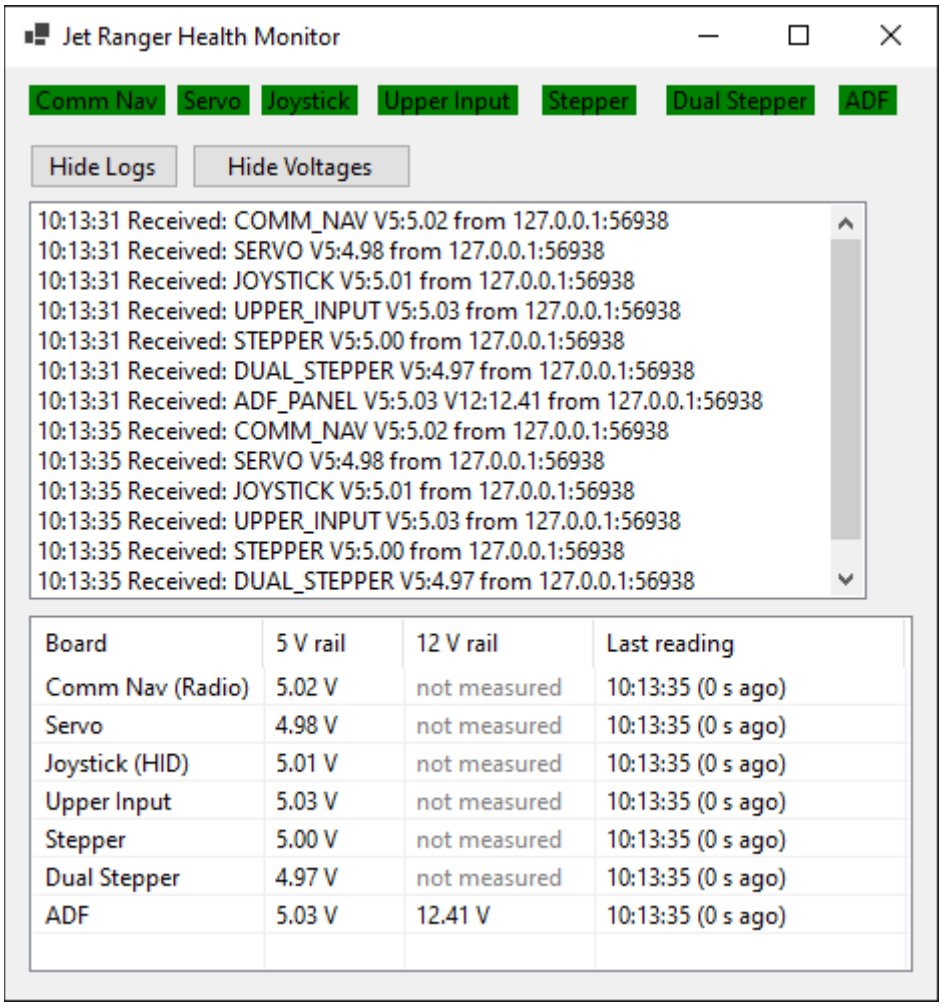



*Figure 13 — Voltage table*

Each board adds its 5 V rail to its heartbeat, and the ADF also adds its 12 V rail, so the table shows the last 5 V reading from every board, the **12 V** reading for the ADF only (the other rows say "not measured") and when the reading arrived. A reading outside its limits (5 V outside 4.75 to 5.25, 12 V outside 10.8 to 15.0) is orange, and a reading that is more than 35 seconds old, or that has not arrived yet, is grey. A board that has not been reflashed since the voltages were added shows "no reading" but still lights green. *Hide Voltages* closes the table.

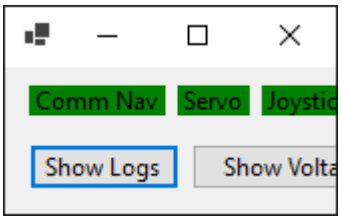



*Figure 14 — Logs hidden*

A board that goes quiet turns orange after 15 seconds and red after 30 seconds.

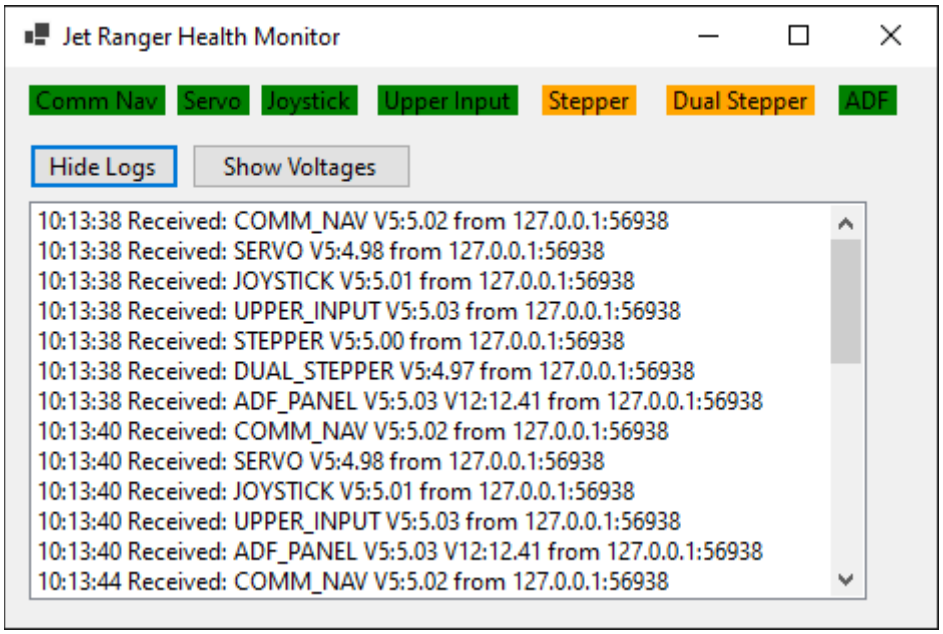



*Figure 15 — Stepper and Dual Stepper quiet for 15 seconds: orange*

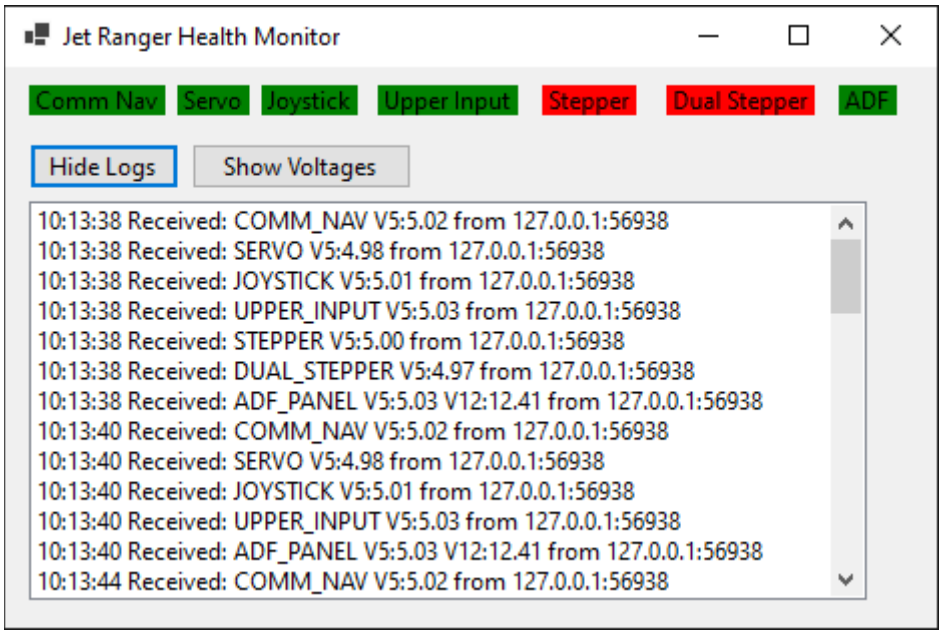



*Figure 16 — Stepper and Dual Stepper quiet for 30 seconds: red*

## 5.3 SwingTester

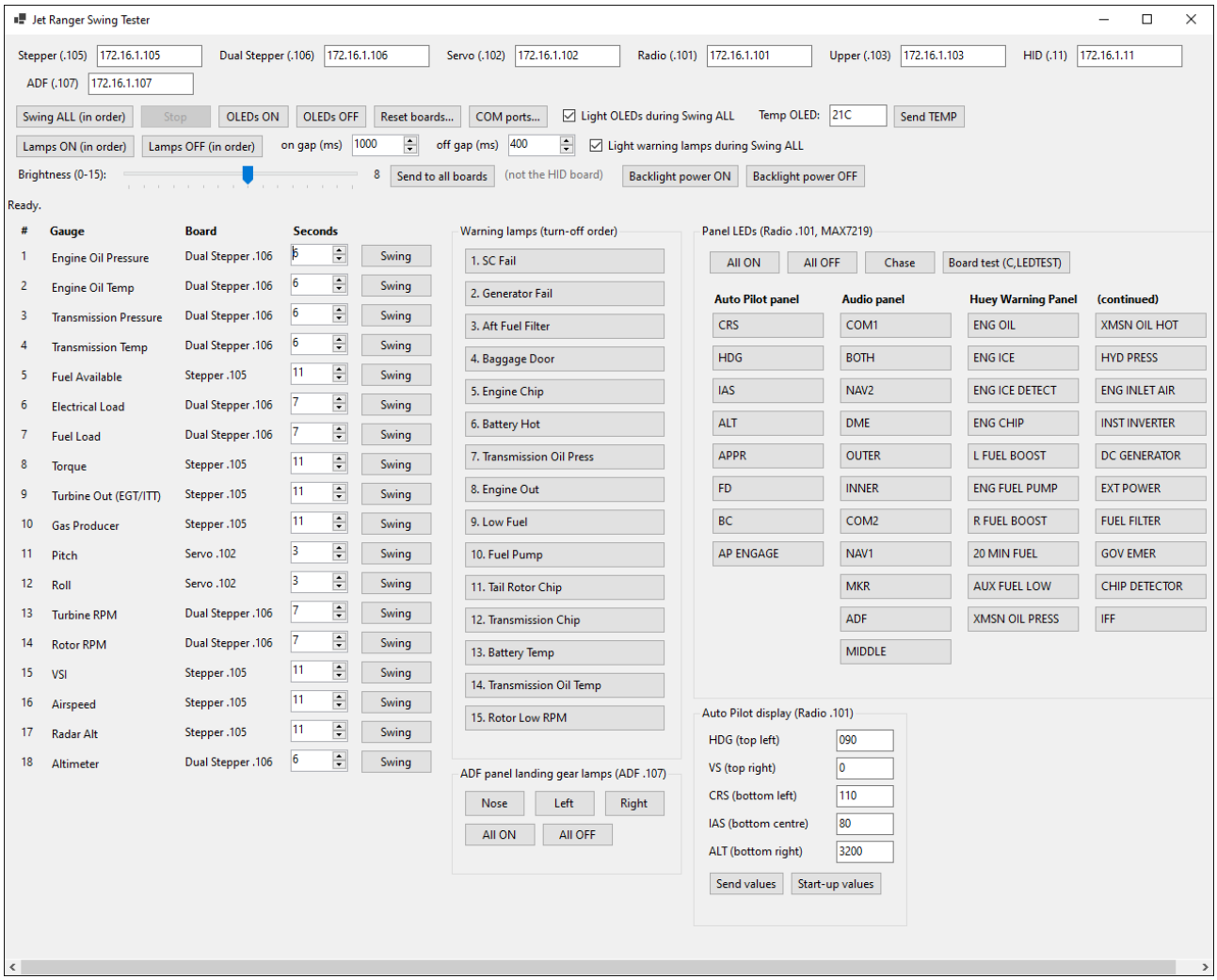



*Figure 17 — SwingTester main window*

- **Top row:** the IP addresses of the boards (Stepper, Dual Stepper, Servo, Radio, Upper, ADF and HID; the Upper address .103 is the retired board, the ADF and Upper board is .107).
- **Swing ALL (in order)** runs the 18 gauge swings in the fixed order, **Stop** cancels the remaining waits, **OLEDs ON / OFF** fill or restore the Dual Stepper's four OLEDs, **Reset boards...** opens the reset dialog, and **Light OLEDs during Swing ALL** controls the OLEDs. **Temp OLED** with **Send TEMP** sends text to the Dual Stepper's temperature display.
- **Lamps ON (in order)** and **Lamps OFF (in order)** with the **on gap** and **off gap** times, and **Light warning lamps during Swing ALL**.
- **Brightness (0-15)** slider with **Send to all boards** sends the backlight brightness to every board except the USB HID board (the Radio, Servo, Stepper, Dual Stepper and ADF and Upper, and the retired Upper address); **Backlight power ON / OFF** send `BPWR` to those boards to test the main bus rule.
- **Status line:** what was last sent and to which boards.
- **Panel LEDs group** (far right): one toggle for each of the 39 Auto Pilot, Audio and Huey Warning Panel LEDs (gold when on; the Huey LEDs are in two columns, *Huey Warning Panel* and *(continued)*), which lights that LED on its own through the Radio controller (`D,NAVCOM1:1,LED_xxx:1/0` or `LED_H_xxx`). **All ON** and **All OFF** do all 39, **Chase** lights them one at a time for the *on gap* time (click again to stop; a full chase of 39 takes about 39 seconds at the default), and **Board test (C,LEDTEST)** lights all 39 with the Radio's own test. The Radio on COM20 was flashed with the Huey build on 31 October 2026; the LEDs have not been checked on the panel yet.

- **Auto Pilot display group** (far right, below the panel LEDs): five boxes for HDG, VS, CRS, IAS and ALT (start-up values 090, 0, 110, 80 and 3200). **Send values** sends them to the Radio for the Auto Pilot panel's OLED and **Start-up values** puts the start-up values back.
- **ADF panel landing gear lamps** (under the warning lamps): **Nose**, **Left** and **Right** toggles (gold when on) light the three lamps on the ADF panel through the ADF controller (172.16.1.107, set in the `ADF (.107)` box at the top), with **All ON** and **All OFF**.
- **COM ports...** opens a window that lists the PC's COM ports with the board type, the USB serial number and whether another program has the port open, and can restart a USB device (see COM ports window below).
- **Gauge rows** (left): the 18 gauges with the board each is on, the wait in seconds and a **Swing** button for each. The row being swung is highlighted gold.
- **Warning lamps** (right): one button for each of the 15 lamps, in turn-off order. A lamp that is on is gold.

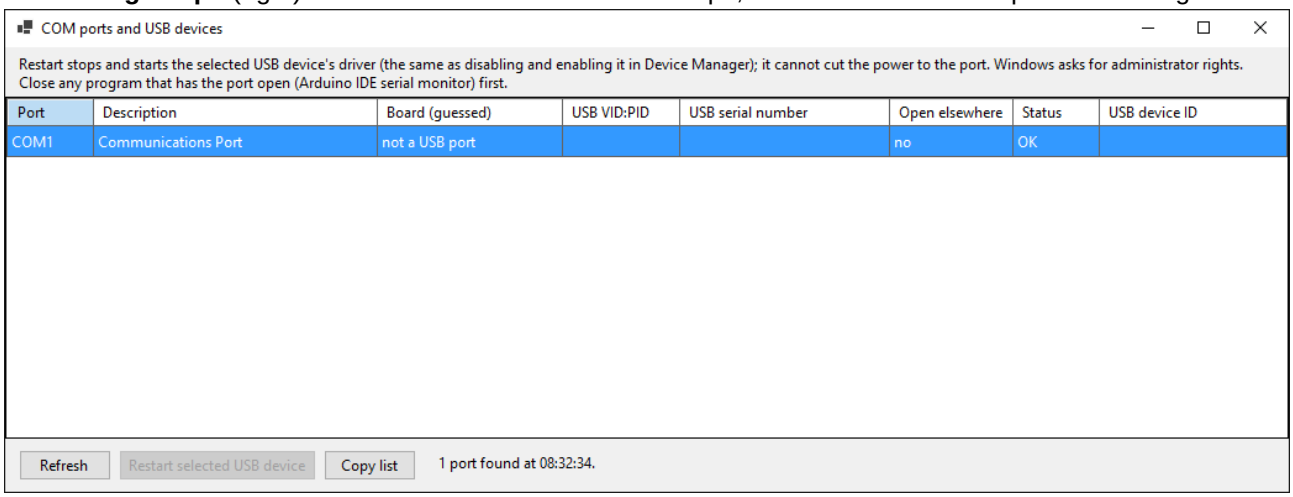



*Figure 18 — The COM ports window*

**COM ports...** lists every COM port Windows knows: the port, Windows' description, the board type (guessed from the USB vendor and product ids), the USB serial number (genuine Arduino boards have one; most clones do not), whether another program has the port open (for example the Arduino IDE serial monitor, which blocks flashing), the status and the USB device id. It refreshes by itself when a device is plugged in or removed. **Copy list** puts the table on the clipboard. **Restart selected USB device** stops and starts the selected board's USB device with `pnputil` (the same as disabling and enabling it in Device Manager, so the port reappears; it cannot cut the power): Windows asks for administrator rights, and the port must not be open in another program.

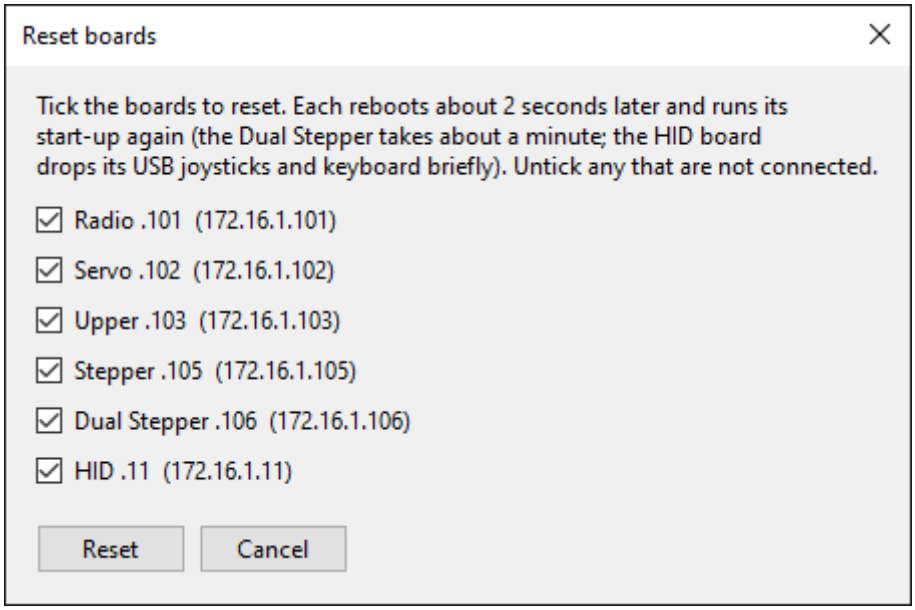



*Figure 19 — The Reset boards dialog*

The **Reset boards...** dialog has a tick box for each board (all ticked; untick any that are not connected). **Reset** sends `C,RESET:1` to each ticked board and **Cancel** closes the dialog without sending anything.

## 5.4 StepperVSITester

The window scrolls; both halves are shown.

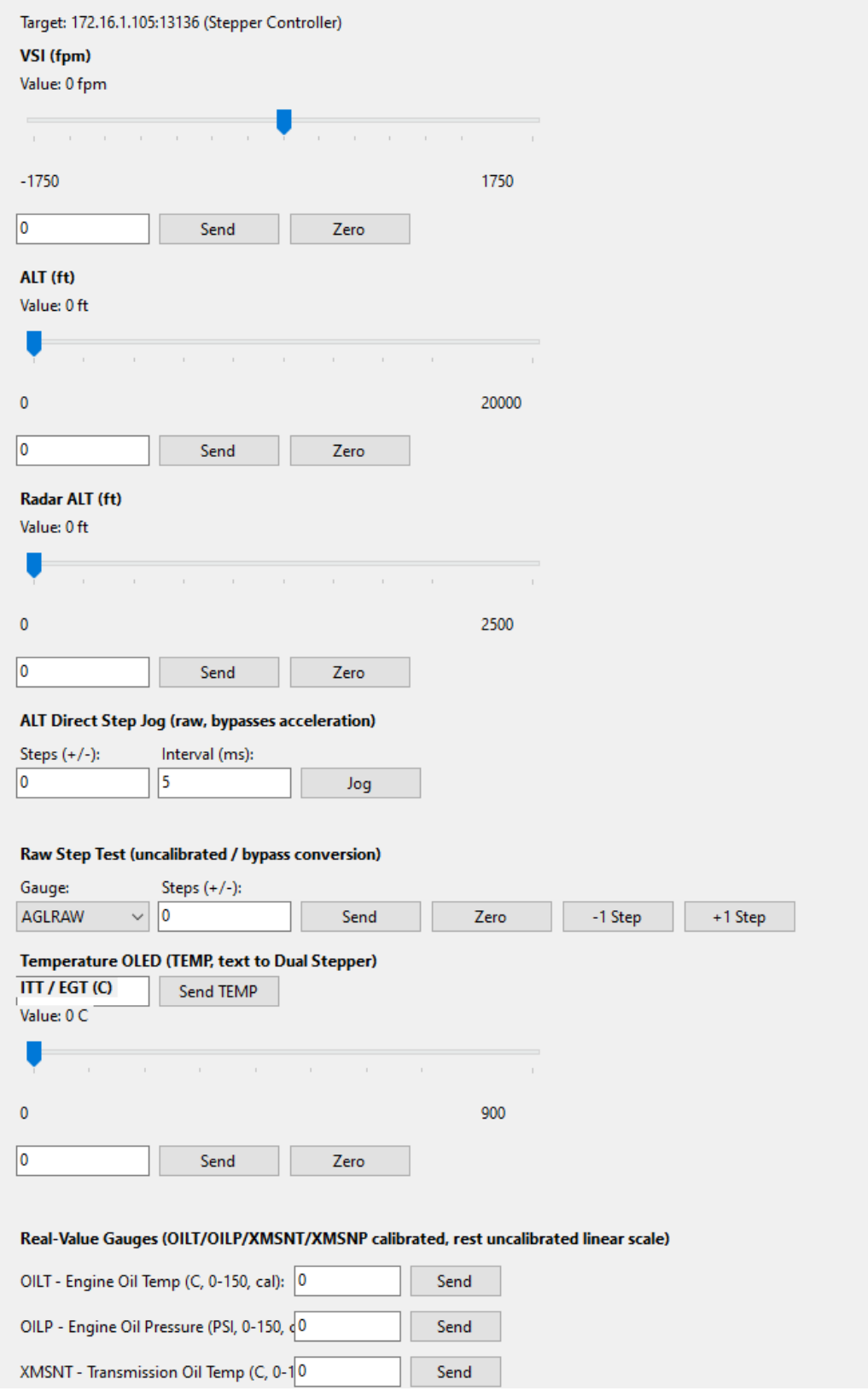



*Figure 20 — StepperVSITester, top half*

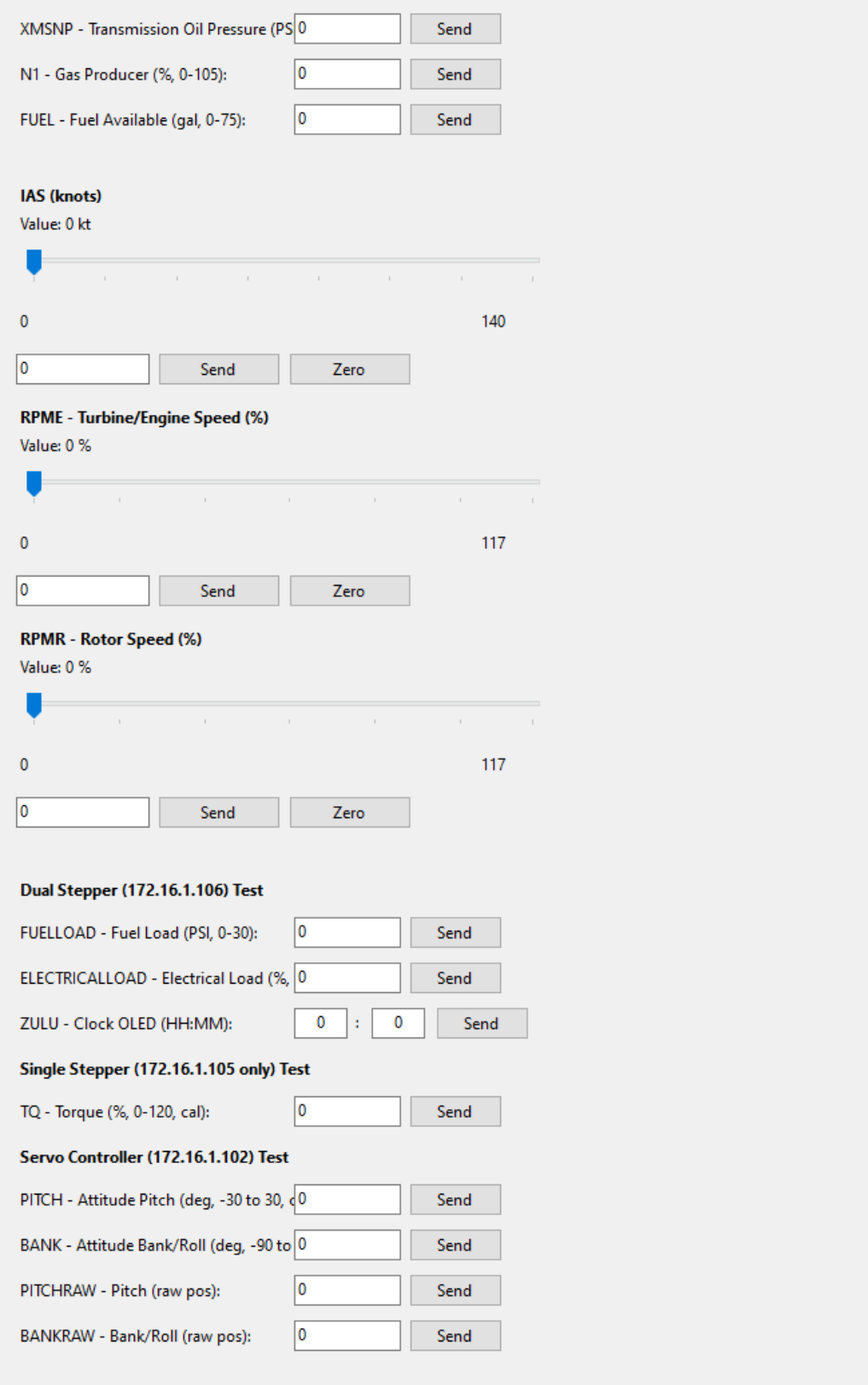



*Figure 21 — StepperVSITester, bottom half*

The target is shown at the top (the Stepper board, 172.16.1.105 port 13136). Each gauge has a slider, a text box, **Send** and **Zero**:

- **VSI (fpm)**, **ALT (ft)**, **Radar ALT (ft)**: real values for the vertical speed, altimeter and radar altimeter.
- **ALT Direct Step Jog** sends a signed step count at a fixed interval, bypassing the acceleration ramp.

- **Raw Step Test** sends raw steps to the chosen gauge code (with **-1 Step** and **+1 Step** nudge buttons).
- **Temperature OLED** sends text to the Dual Stepper's temperature display, and the **ITT / EGT** slider below it sends turbine out temperature.
- **Real-Value Gauges** rows for engine oil temperature and pressure, transmission oil temperature and pressure, gas producer and fuel.
- **IAS**, **RPME** and **RPMR** sliders for airspeed, engine speed and rotor speed.
- **Dual Stepper test** rows (fuel load, electrical load, the clock), **Single Stepper test** (torque) and **Servo Controller test** (pitch and bank, in degrees or as raw servo positions).

## 5.5 Emulators

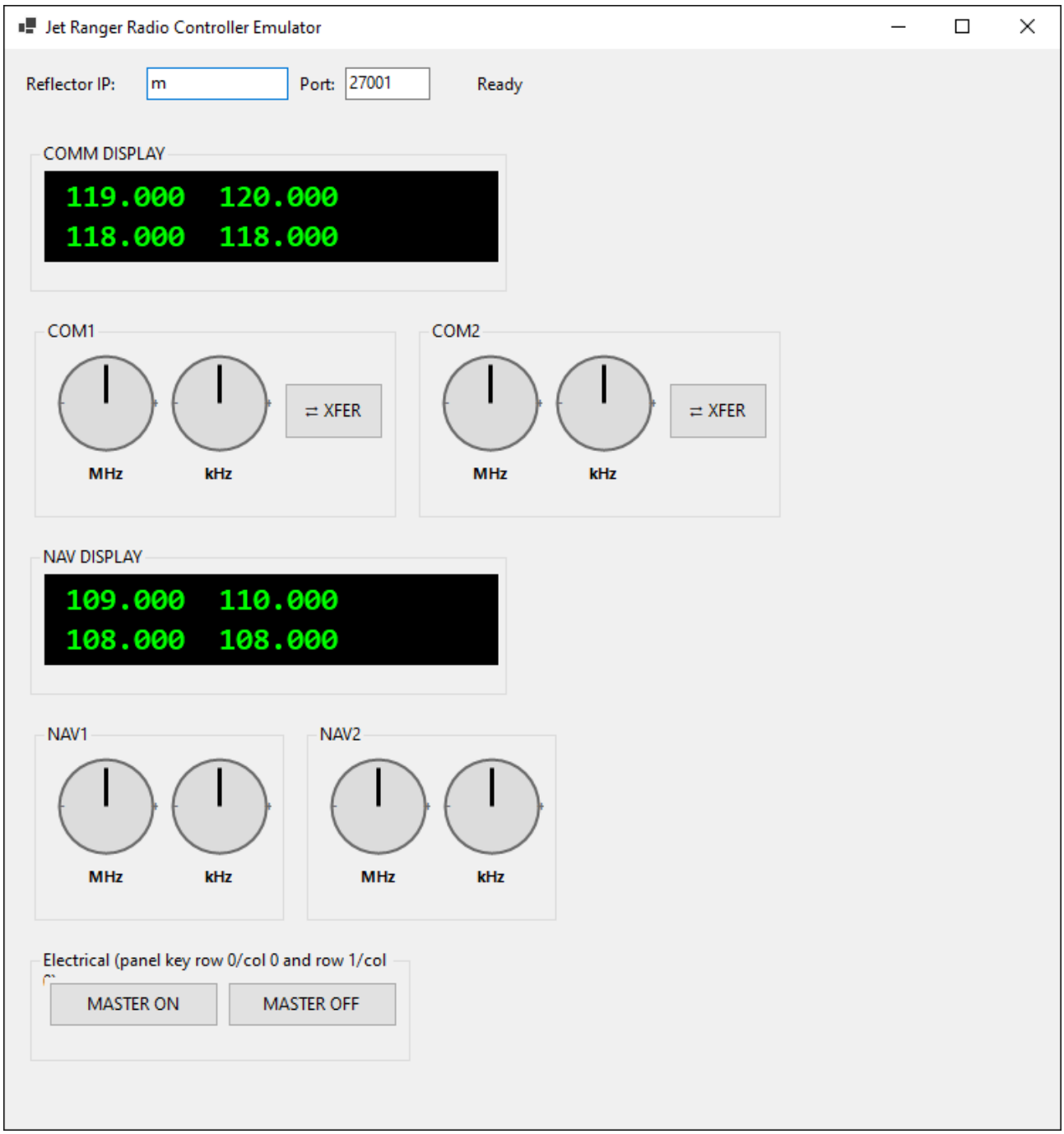



*Figure 22 — JetRangerRadioControllerEmulator*

The **Radio controller emulator** stands in for the radio board. Set the reflector (bridge) address and port at the top. The COMM and NAV displays show the active and standby frequencies; each radio has a **MHz** and a **kHz** knob (click the right half to turn it up one detent, the left half to turn it down) and the COM radios have an **XFER** swap button. **MASTER ON** and **MASTER OFF** send the battery and alternator commands. Every control sends its packet to the bridge on UDP 27001 at once.

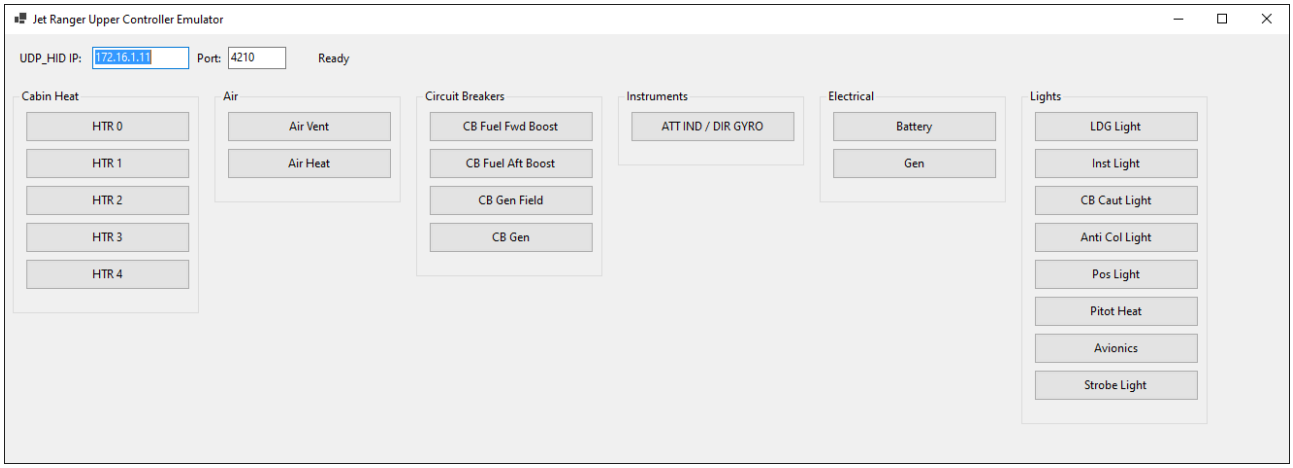



*Figure 23 — JetRangerUpperControllerEmulator*

The **Upper controller emulator** stands in for the upper panel (now on the combined ADF and Upper board; the switch numbers are unchanged). Set the USB HID board's address and port at the top (172.16.1.11, 4210). There is one toggle button per switch the firmware uses, in groups: Cabin Heat, Air, Circuit Breakers, Instruments, Electrical and Lights. A button turns green when its switch is on, and each press sends the switch change to the USB HID board.

## 5.6 GPS interface

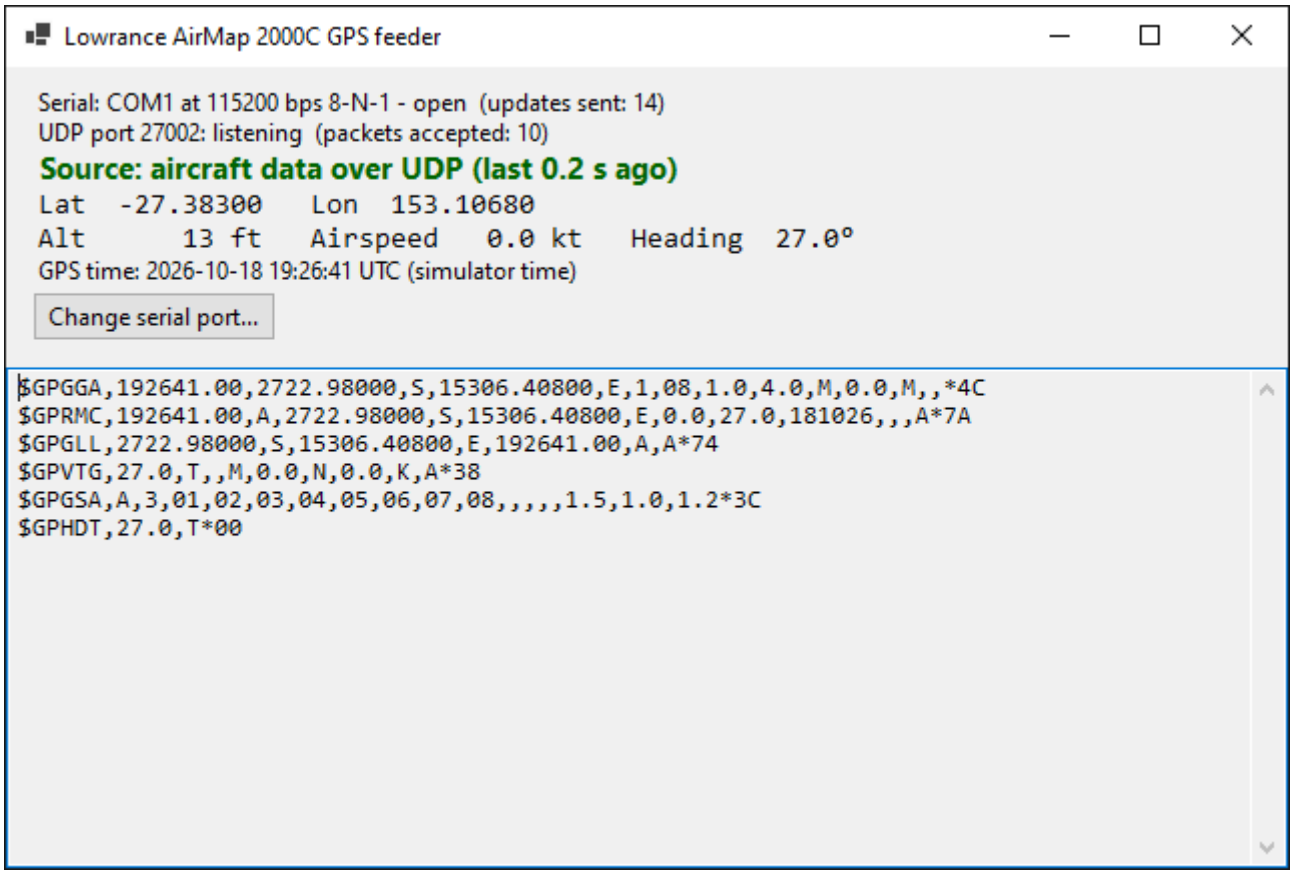



*Figure 24 — LowranceAirMapGps*

The window shows, from the top: the serial port, its speed and state and how many updates have been sent; the UDP port and the number of packets accepted; the **source** of the position (green *aircraft data over UDP*, orange *default - Brisbane Airport*); the latitude, longitude, altitude, airspeed and heading being sent; the **GPS time** and whether it comes from the simulator or the PC clock; the **Change serial port...** button; and the sentences being sent to the GPS.

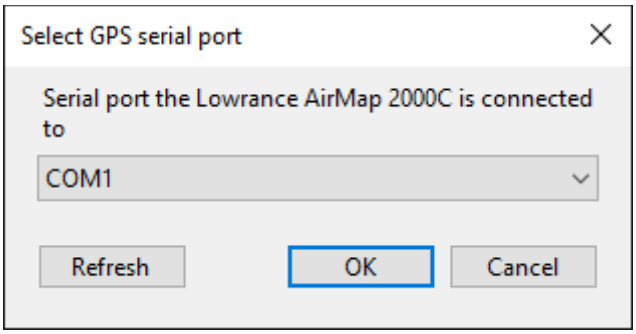



*Figure 25 — The serial port dialog*

The first time the program runs, or when the saved port is no longer present, this dialog asks which port the GPS is connected to. **Refresh** rereads the list of ports, **OK** saves the choice and **Cancel** leaves the port unselected (the program still runs, without sending to a GPS). **Change serial port...** opens the same dialog later.

## 5.7 GPS bench tester

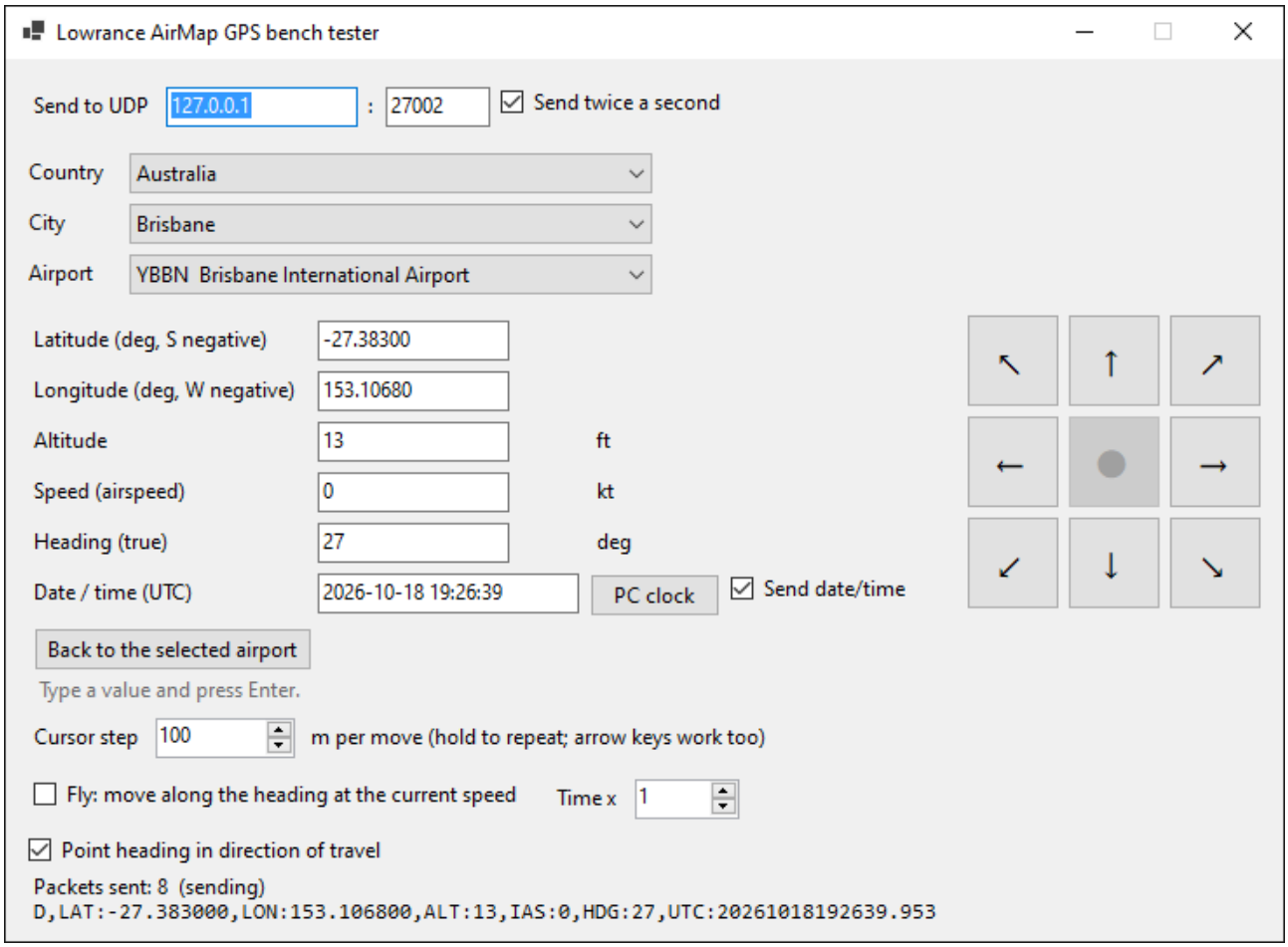
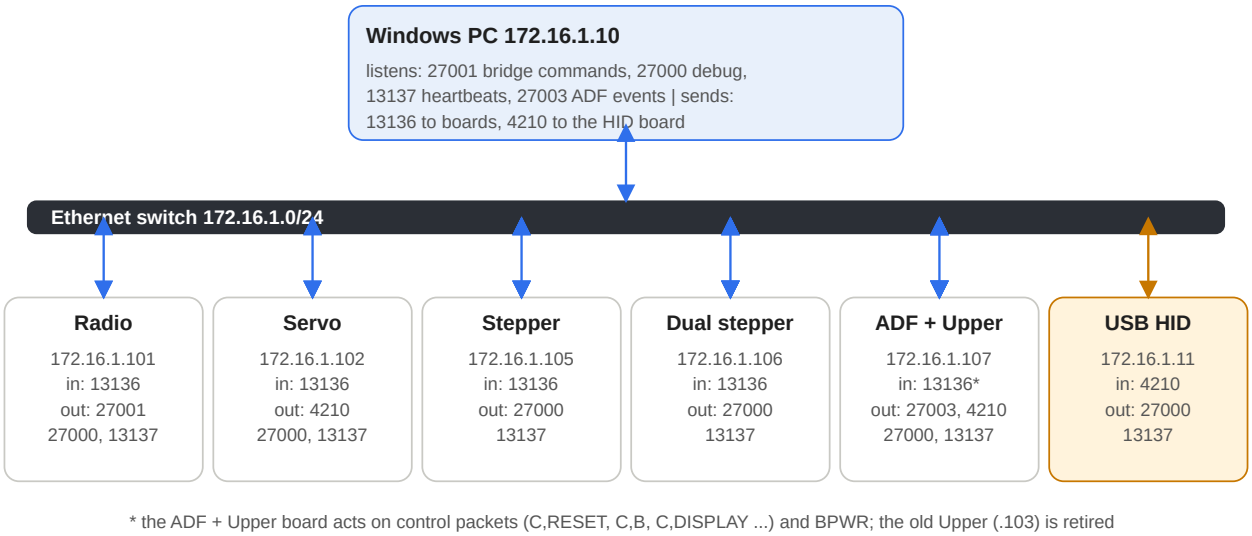



*Figure 26 — LowranceAirMapGpsBenchTester*

- **Send to UDP:** the address and port of the GPS interface (the same PC and port 27002 by default) and a check box to turn the sending on and off.
- **Country, City and Airport:** choose an airport to put the aircraft on the ground there.
- **Latitude, Longitude, Altitude, Speed and Heading:** type a value and press Enter (a value that cannot be read turns the box pink).
- **Date / time (UTC):** the date and time the GPS should show, **PC clock** to use the PC's, and **Send date/time** to include it.
- **Back to the selected airport** returns to the airport chosen above.
- **Eight-way cursor** (right): moves the position by the **Cursor step** in metres; the arrow keys work too. **Point heading in direction of travel** sets the heading to the direction moved.
- **Fly** moves the aircraft along its heading at its speed; **Time x** speeds it up.
- **Bottom:** the number of packets sent and the packet being sent.

# 6 Network

All devices sit on one wired `172.16.1.x` subnet with fixed addresses; there is no DHCP on the boards.




* the ADF + Upper board acts on control packets (C,RESET, C,B, C,DISPLAY ...) and BPWR; the old Upper (.103) is retired

**What the ports carry**

| | |
|---|---|
| 13136 | data (D,...) and control (C,...) packets from the PC to a board |
| 27001 | radio, battery and alternator commands from the Radio board to the bridge |
| 4210 | switch events (index,state) and key events to the USB HID board; also C,RESET:1 for it |
| 27002 | aircraft information from a bridge (or the bench tester) to the Lowrance GPS interface |
| 27003 | ADF frequency, radar follow ID and power status from the ADF controller to the PC (no bridge reads it yet) |
| 27000 | debug text from every board to the UDP reflector |
| 13137 | 10 second heartbeat text from every board to the Health Monitor |

*Figure 27 — Who talks to whom*

| Device | IP address | MAC | Keep-alive text |
|---|---|---|---|
| Primary PC (simulator, bridge, debug listener, Health Monitor) | 172.16.1.10 | | |
| USB HID board | 172.16.1.11 | DE:AD:BE:EF:FE:ED | `JOYSTICK` |
| Radio controller | 172.16.1.101 | A8:61:0A:9E:83:01 | `COMM_NAV` |
| Servo board | 172.16.1.102 | A8:61:0A:9E:83:02 | `SERVO` |
| Stepper board | 172.16.1.105 | A8:61:0A:67:83:69 | `STEPPER` |
| Dual stepper board | 172.16.1.106 | A8:61:0A:67:83:6A | `DUAL_STEPPER` |
| ADF and Upper controller | 172.16.1.107 | A8:61:0A:67:83:6B | `ADF_PANEL` and `UPPER_INPUT` |

Every live board has its own IP and MAC address. The old Upper controller (172.16.1.103) was merged into the ADF controller on 31 October 2026 and no longer exists; the merged board sends both heartbeat names so the Health Monitor still shows an Upper light. The bridges and the SwingTester still send some packets to .103, which go nowhere. The firmware sends its debug and keep-alive traffic to `172.16.1.10` (`reflectorIP`), so the primary PC must hold that address, or `reflectorIP` must be changed and the sketch uploaded again.

| UDP port | Direction | Purpose |
|---|---|---|
| 13136 | PC to boards | Gauge, lamp and radio data (`D,`) and control (`C,`) packets |
| 27001 | Radio and ADF controllers to PC bridge | Standby frequencies, radio swaps, and the Auto Pilot, Audio and ADF panel controls (for example `AP_MASTER`, `GEAR_DOWN`) |
| 27002 | Bridge to the GPS interface (same PC) | Aircraft position, altitude, airspeed, heading and simulator time, every 500 ms |

| UDP port | Direction | Purpose |
|---|---|---|
| 27003 | ADF controller to the PC | ADF frequency, radar follow ID and power status (no bridge reads it yet) |
| 4210 | Servo, Radio and ADF and Upper controllers to USB HID board | Switch changes (`index,state`) and keypad keys (`KEY,key,state`) |
| 13137 | Boards to Health Monitor | Keep-alive text every 10 seconds |
| 27000 | Boards to debug listener | Debug messages, each prefixed with the board name |

## 6.1 Packet format

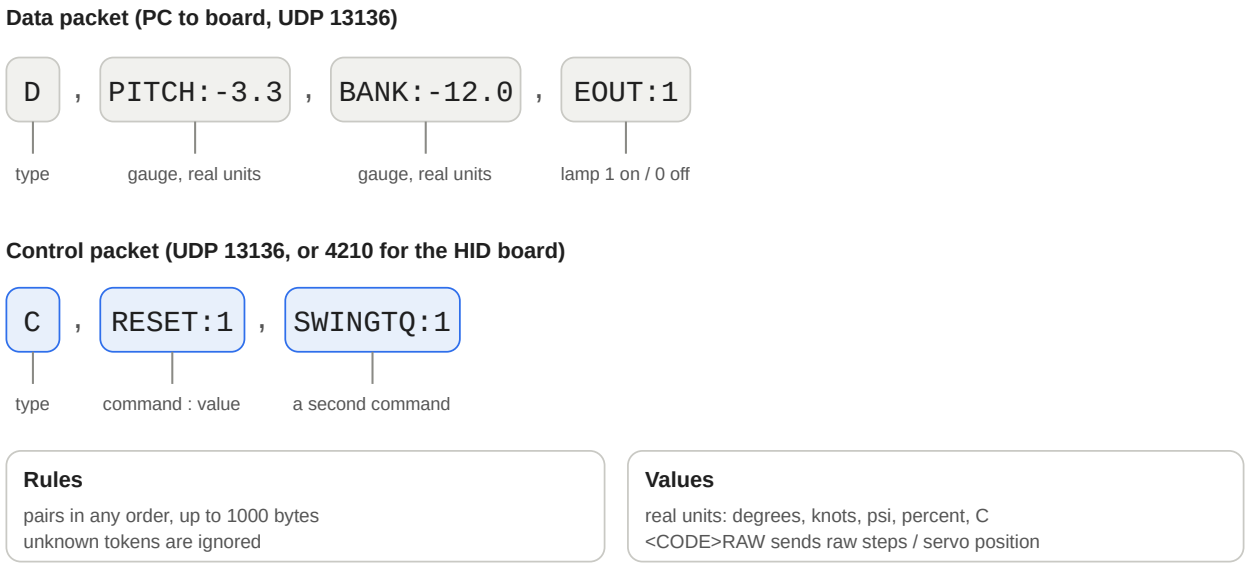



*Figure 28 — Data and control packets*

A packet can carry any number of pairs, in any order, up to 1000 bytes. Unknown tokens are ignored, so the PC can send only the values that changed. For example `D,PITCH:5,BANK:-10,LOWF:0` sent to the servo board sets 5 degrees pitch, minus 10 degrees bank and turns off Low Fuel.

To test without the simulator, send a packet from a PC on the subnet, for example with PowerShell:

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes('D,PITCH:5,RLOW:1')
$u.Send($b, $b.Length, '172.16.1.102', 13136)
```

## 6.2 What a board does

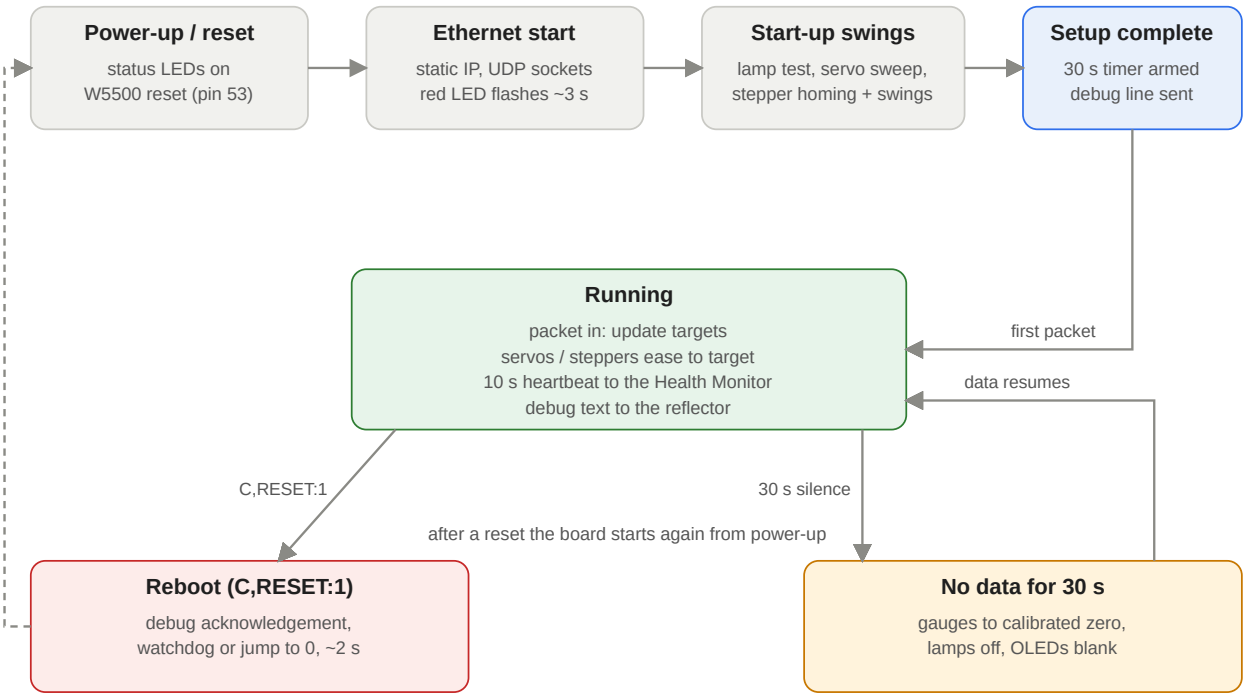



*Figure 29 — Board life cycle*

After start-up and its self-test a gauge board checks UDP 13136 every 5 ms and moves each needle smoothly towards its target. It sends its keep-alive every 10 seconds, with its 5 V rail after the board name (and the ADF controller its 12 V rail as well) for the Health Monitor. If no packet arrives for **30 seconds** every needle returns to zero (the Servo board also switches its lamps off, the Dual Stepper blanks its OLEDs and the Radio blanks its displays); the next packet brings everything back.

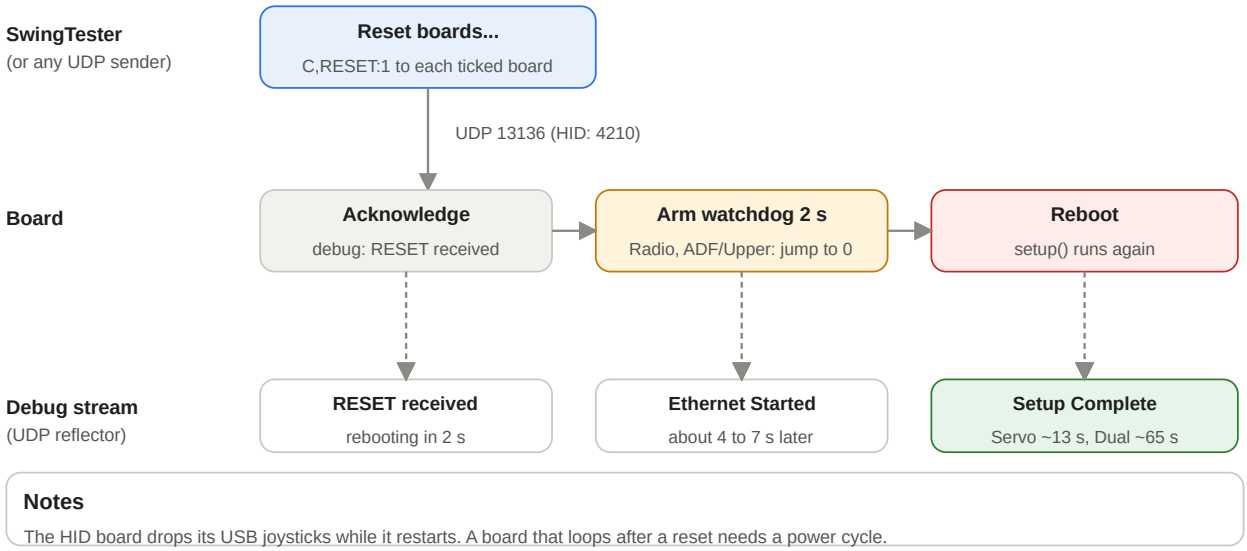



*Figure 30 — Remote reset*

The control packet `C,RESET:1` reboots a board about 2 seconds later. The SwingTester's *Reset boards...* button sends it to the boards you tick.

## 6.3 Unreachable hosts and the fast Ethernet timeout

Every sketch (Radio, Servo, Stepper, Dual Stepper, ADF and Upper and the USB HID board) sends UDP packets to other devices: debug text and keep-alives to the PC, commands from the Radio to the bridge, switch changes from the ADF and Upper controller to the USB HID board, and brightness from the ADF controller to the other panels. Before the Ethernet module can send to an address on the network it has to learn that address's MAC address by an ARP request. If the destination is switched off, unplugged or not yet running (the PC, or another board that is still starting), nobody answers.

The W5500 Ethernet chip then waits and asks again. Its default is a 200 ms timeout with 8 attempts, so one send to a missing device can hold the sketch up for about **1.6 seconds**. The sketch cannot do anything else while it waits: encoders and switches are not scanned, needles do not move, and every debug line sent to a PC that is off would cost another 1.6 seconds.

To stop that, each sketch calls `Ethernet.setRetransmissionTimeout(10)` straight after `Ethernet.begin()`. The timeout becomes **10 ms** and the number of attempts stays at 8, so the worst case for one send to a missing device is about **80 ms**. The panel keeps responding when the PC is off, a bridge is not running, or a board is unplugged, and the boards that are present are not slowed in any way you would notice.

- A missing device is not free: each send to it still costs up to 80 ms, so a sketch that sends many packets to a dead address (for example heavy debug traffic with the PC off) will run a little slower than normal. Switch the PC on, or turn debug traffic down, if a board feels sluggish with no PC.
- 10 ms is far longer than a real device on the cockpit network needs to answer an ARP request, so working devices are not affected. If you ever add a device on a slow or long network link, and packets to it are lost at first, raise the value in that sketch.
- The ADF controller sends its brightness to one board per pass through its loop for the same reason: a switched-off board never holds up the knobs and keypad for more than that one send.

## 6.4 When the panel loses power

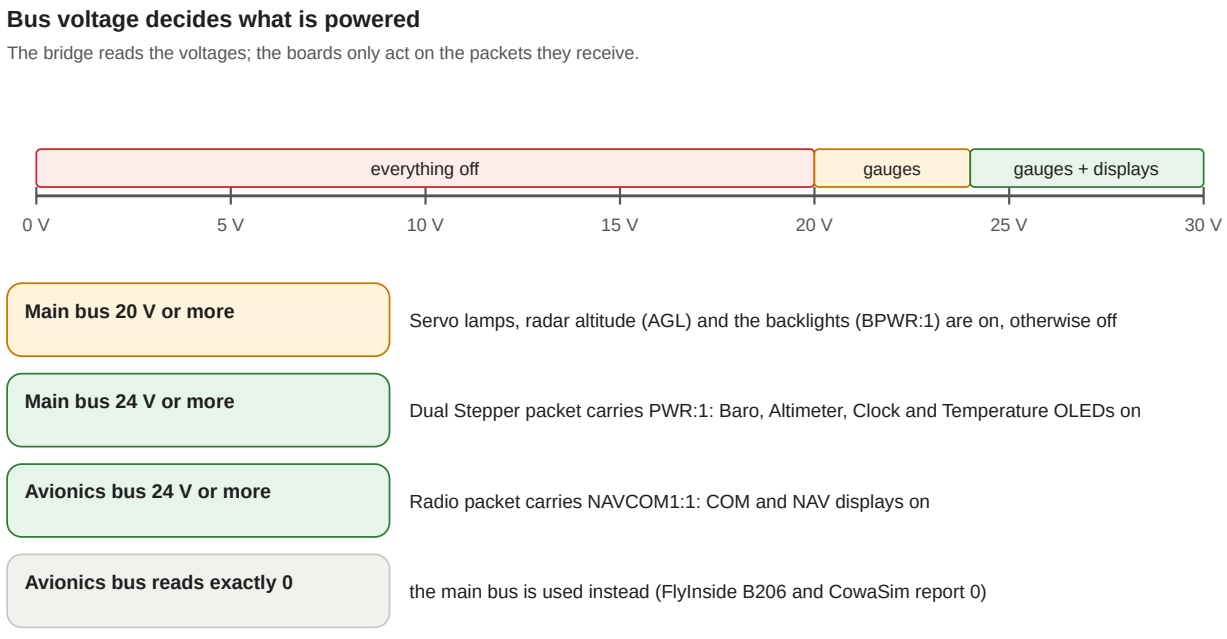



*Figure 31 — Bus voltage rules*

The bridge turns parts of the panel off when the simulated electrics drop: lamps, radar altitude and the backlights below 20 V on the main bus, the Dual Stepper's four OLEDs below 24 V on the main bus, and the Radio's COM and NAV displays below 24 V on the avionics bus (the main bus is used if the aircraft reports the avionics bus as 0).

## 6.5 Backlight

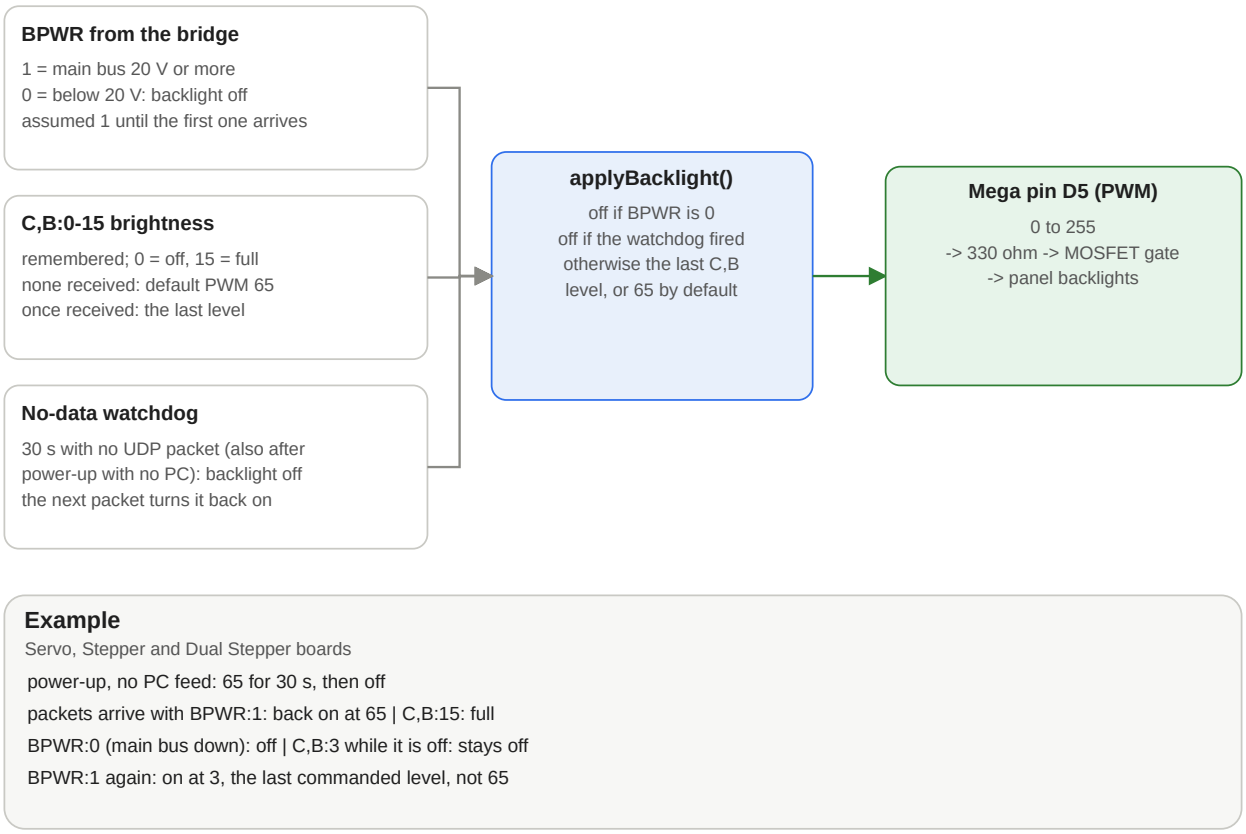



*Figure 32 — How the backlight level is decided*

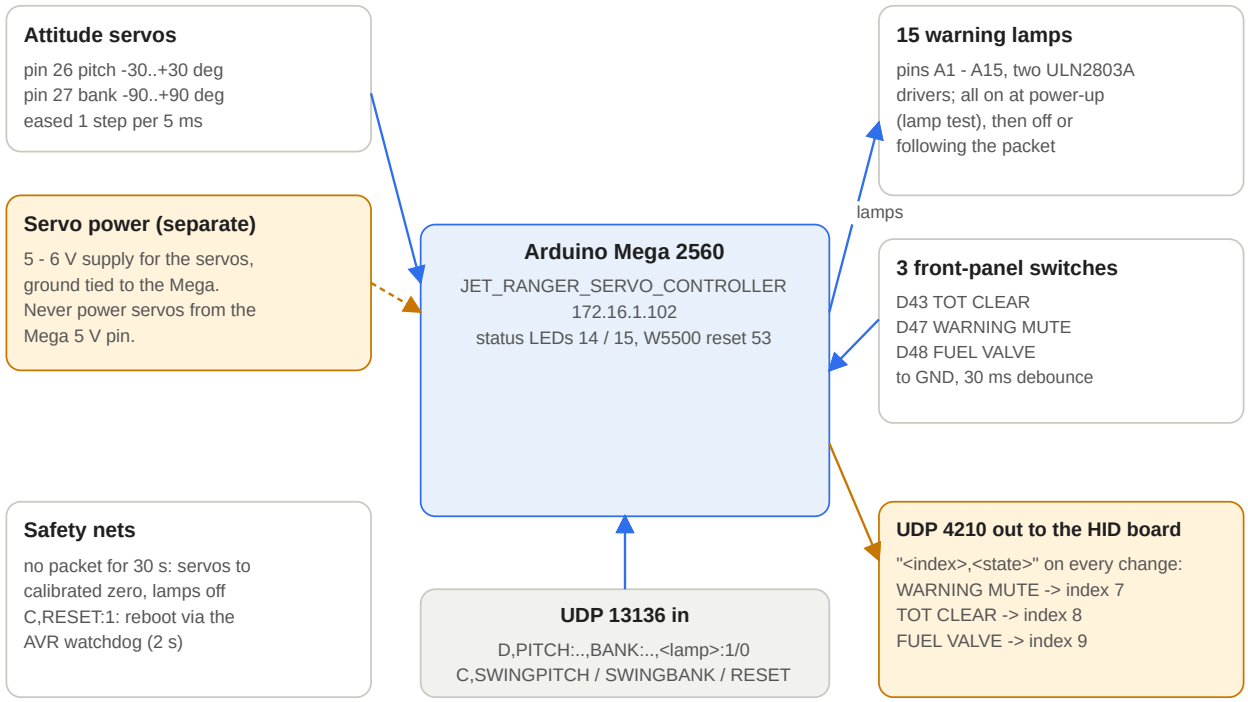
The Radio, Servo, Stepper, Dual Stepper and ADF and Upper boards drive their panel backlights from Mega pin D5 through a MOSFET on the PCB (on the Radio these are the Audio, Auto Pilot and COM and NAV panels; its "no data" rule is the same 30 seconds that blanks its displays, restarted by any knob or key; on the ADF and Upper board a switch, key or knob restarts it). Three things decide the level:

- **Main bus power (`BPWR`).** The bridge sends `BPWR:1` while the main bus is 20 V or more and `BPWR:0` below that, which switches the backlight off. Until the first `BPWR` arrives the board leaves the backlight on.
- **Brightness (`C,B:0` to `C,B:15`).** 0 is off and 15 is full. The board remembers the last level. If no brightness command has been sent the backlight uses a default level (PWM 65); once one has been sent, every time power returns the backlight comes back at the **last level you set**, not the default. A level sent while the bus is down is applied when power returns.
- **No data.** After 30 seconds with no UDP packet, including after the board is powered up with no PC sending, the backlight goes off along with the gauges. The next packet turns it back on.

The **SwingTester** has a brightness slider that sends the level to every board except the USB HID board, and *Backlight power ON / OFF* buttons for testing the main bus rule (including the ADF and Upper board). The **ADF controller** can send the level from its brightness pot to the other boards, but the pot is switched off for the moment. The Radio also uses the level for its panel LEDs. The ADF controller also blanks its display and lamps with the Radio's displays.

# 7 Arduino servo controller

The servo board is an Arduino Mega 2560 with an Ethernet module, on the Jet Ranger Front Panel Servo PCB at 172.16.1.102. It drives the attitude indicator's two servos, switches the 15 caution and warning lamps and reads three front-panel switches. It runs `JET_RANGER_SERVO_CONTROLLER.ino`. The other gauges that were once servo-driven now run on the two stepper boards.




Joystick 1 buttons 8, 9 and 10 in Windows (the firmware counts from 0). FSUIPC needs them bound to DS206 events.

Calibration: PITCH_DEG_TABLE -30 deg 179, 0 deg 113, +30 deg 70; BANK_DEG_TABLE -90 deg 5, 0 deg 93, +90 deg 179 (servo positions).

*Figure 33 — Servo board*

## 7.1 Uploading the firmware

1. Open the sketch in the Arduino IDE and select *Board: Arduino Mega or Mega 2560* (or use `arduino-cli`, see Flashing a board).
2. Install the libraries it uses: Ethernet (with SPI), Servo, Ewma (jonnieZG) and DCS-BIOS.
3. Upload, then watch the start-up self-test: both servos sweep and all 15 lamps light.

## 7.2 Servos

| Instrument | Mega pin | UDP token | Range |
|---|---|---|---|
| Attitude bank (roll) | 27 | `BANK` | -90 to +90 degrees |
| Attitude pitch | 26 | `PITCH` | -30 to +30 degrees |

Both tokens take attitude in degrees, with one decimal place allowed (`D,PITCH:4.5,BANK:-12`). The board turns degrees into a servo position with the tables in Calibration. `BANKRAW` and `PITCHRAW` take a raw servo position (0 to 180) and skip the tables. The PCB still has 16 servo headers from the earlier design; only pins 26 and 27 are used.

## 7.3 Other pins

| Function | Pin | Behaviour |
|---|---|---|
| Green status LED | 14 | Flashes alternately with red every 200 ms while running |
| Red status LED | 15 | Fast flash during the 3 second Ethernet start-up, then alternates with green |
| Ethernet module reset | 53 | Pulsed low for 2 ms at power-up |
| Backlight | 5 | PWM to the backlight MOSFET (`BACKLIGHT_PWM`); see Backlight |
| Lamps | A1 to A15 | Digital outputs, HIGH = lit (see Reference) |
| TOT clear switch | 43 | Switch to GND, debounced 30 ms; each change goes to the USB HID board as Joystick 1 button 9 (index 8) |
| Warning mute switch | 47 | The same, Joystick 1 button 8 (index 7) |
| Fuel valve switch | 48 | The same, Joystick 1 button 10 (index 9) |

**The 15 warning lamps on the Servo board**

Green: the bridge drives it from the simulator. Grey: needs aircraft-specific variables, so it stays off.

| | | |
|---|---|---|
| **ENGINE OUT**<br>pin A1 token EOUT<br>engine speed < 60 % | **ROTOR RPM LOW**<br>pin A2 token RLOW<br>rotor <= 95 % | **XMSN OIL PRESS**<br>pin A3 token TOPW<br>pressure < 40 psi |
| **XMSN OIL TEMP**<br>pin A4 token TOTW<br>always off | **BATTERY HOT**<br>pin A5 token BHOT<br>always off | **BATTERY TEMP**<br>pin A6 token BTMP<br>always off |
| **ENGINE CHIP**<br>pin A7 token EC<br>always off | **TAIL ROTOR CHIP**<br>pin A8 token TRC<br>always off | **XMSN CHIP**<br>pin A9 token TC<br>always off |
| **BAGGAGE DOOR**<br>pin A10 token BD<br>always off | **AFT FUEL FILTER**<br>pin A11 token FFLTR<br>always off | **FUEL PUMP**<br>pin A12 token FPMP<br>always off |
| **GENERATOR FAIL**<br>pin A13 token GENF<br>engine speed < 10 % | **LOW FUEL**<br>pin A14 token LOWF<br>fuel <= 17 gal | **SC FAIL**<br>pin A15 token SCF<br>always off |

All lamps go out when the main bus is below 20 V. Sent as <token>:1 / <token>:0 in every Servo packet.

The lamp test at power-up lights all 15; a no-data timeout turns them all off.

*Figure 34 — The 15 lamps and their pins*

## 7.4 What happens at power-up

1. All 15 lamps switch on as a lamp test.
2. The Ethernet module is reset and starts at 172.16.1.102; the red LED flashes for 3 seconds.
3. *Ethernet Started* goes to the debug listener.
4. Pitch, then bank, sweeps from minimum to maximum and back (about 2 seconds each) and parks at its calibrated level position.
5. All lamps switch off.
6. The starting state of the three switches is sent to the USB HID board, *Setup Complete* is logged and the 30 second watchdog starts.

## 7.5 Normal running

- Every 5 ms the board checks UDP 13136 and stores each value as a new target; each servo steps one position towards its target, so the attitude indicator moves smoothly.
- A servo that has not moved for 1 second is detached to stop buzz; the next movement re-attaches it.

- Lamps change as soon as a 0 or 1 arrives.
- `C,SWINGPITCH:1` or `C,SWINGBANK:1` re-runs one sweep on demand (the board pauses for about 2 seconds).

# 8 Stepper controller (13 Stepper board)

A second Arduino Mega 2560, on the Jet Ranger 13 Stepper PCB at 172.16.1.105, drives X27-style stepper gauges directly from its pins. Unlike the servo board it takes real units and converts them to needle steps with calibration tables in `JET_RANGER_STEPPER_CONTROLLER.ino`.

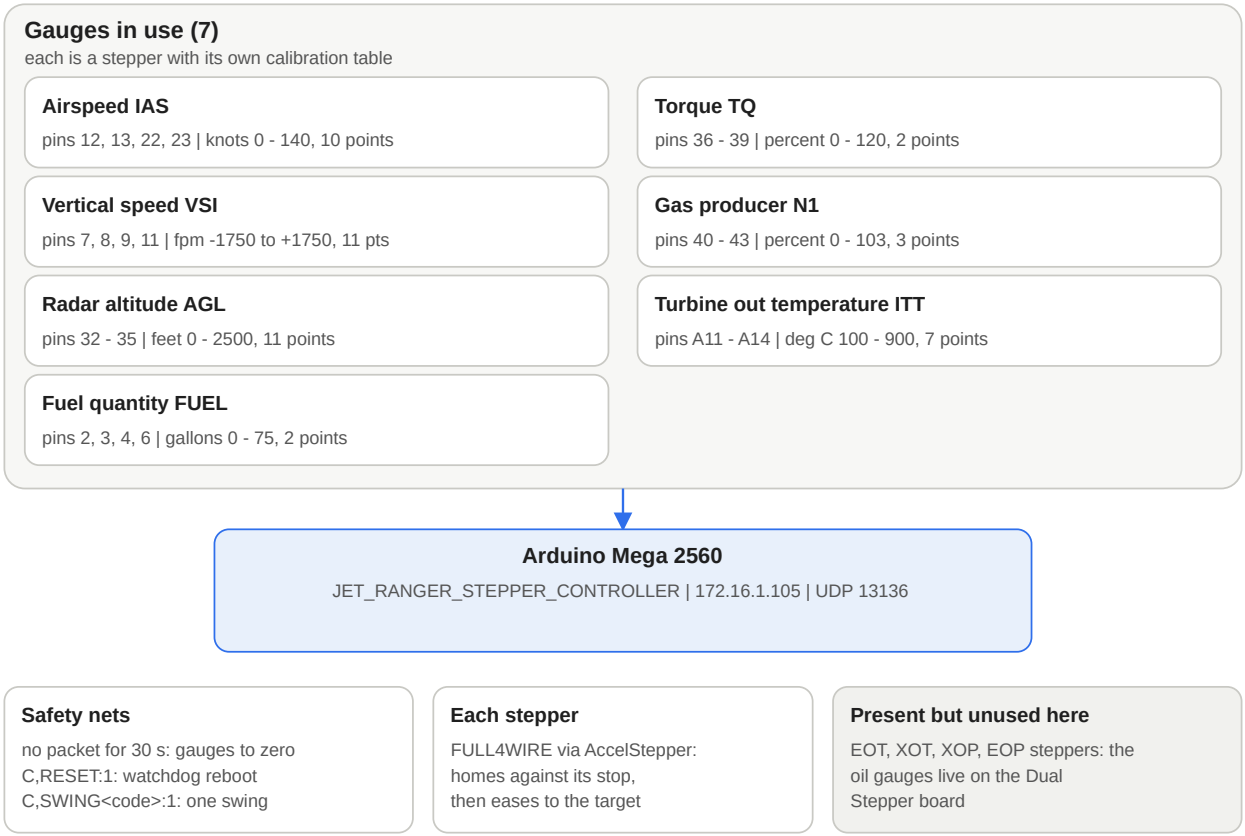



*Figure 35 — Stepper board*

| Gauge | Token | Mega pins | Units | Calibration |
|---|---|---|---|---|
| Airspeed | `IAS` | 12, 13, 22, 23 | knots 0 to 140 | 10-point table |
| Vertical speed | `VSI` | 7, 8, 9, 11 | ft/min, plus and minus 1750 | 11-point table |
| Radar altitude | `AGL` | 32 to 35 | feet 0 to 2500 | 11-point table |
| Torque | `TQ` | 36 to 39 | percent 0 to 120 | 2-point table |
| Gas producer | `N1` | 40 to 43 | percent 0 to 103 | 3-point table |
| Turbine out temperature | `ITT` | A11 to A14 | degrees C 100 to 900 | 7-point table |
| Fuel quantity | `FUEL` | 2, 3, 4, 6 | US gallons 0 to 75 | 2-point table |

Every token has a raw twin with `RAW` on the end (for example `IASRAW:300`). It moves the needle by raw steps and skips the table, which is how new calibration points are measured on the bench. The board also has four oil and transmission steppers with placeholder scales; those gauges are driven from the Dual Stepper, so wire each physical gauge to one board only. Pins 24 to 31 are free.

## 8.1 Behaviour

- **Power-up:** status LEDs flash, Ethernet starts, backlighting ramps up, then each gauge does a self-test sweep in turn: fuel, torque, turbine out, gas producer, VSI, airspeed, radar altitude. Each sweep has its own on/off switch (`SwingFA`, `SwingTQ` and so on) near the top of the sketch.
- **Running:** every 5 ms the board checks UDP 13136 and moves each needle towards its target with smooth acceleration.
- **No-data watchdog:** after 30 seconds without a packet every needle returns to its zero.

- **Zero trims:** each gauge has a `...ZeroOffset` value to nudge its true zero without touching the table.
- **Keep-alive:** `STEPPER` to the Health Monitor every 10 seconds; debug text goes to UDP 27000.

## 8.2 Testing a gauge on demand

Send a control packet to re-run one gauge's sweep without rebooting, for example `C,SWINGTQ:1`.

| Command | Gauge |
|---|---|
| `SWINGFUEL` | Fuel quantity |
| `SWINGTQ` | Torque |
| `SWINGITT` | Turbine out temperature |
| `SWINGN1` | Gas producer |
| `SWINGVSI` | Vertical speed |
| `SWINGIAS` | Airspeed |
| `SWINGAGL` | Radar altitude |

The **StepperVSITester** app can send every token and raw token to this board without the simulator. The **SwingTester** app sends these commands, and the Dual Stepper and servo sweeps, one at a time or all in a fixed order:

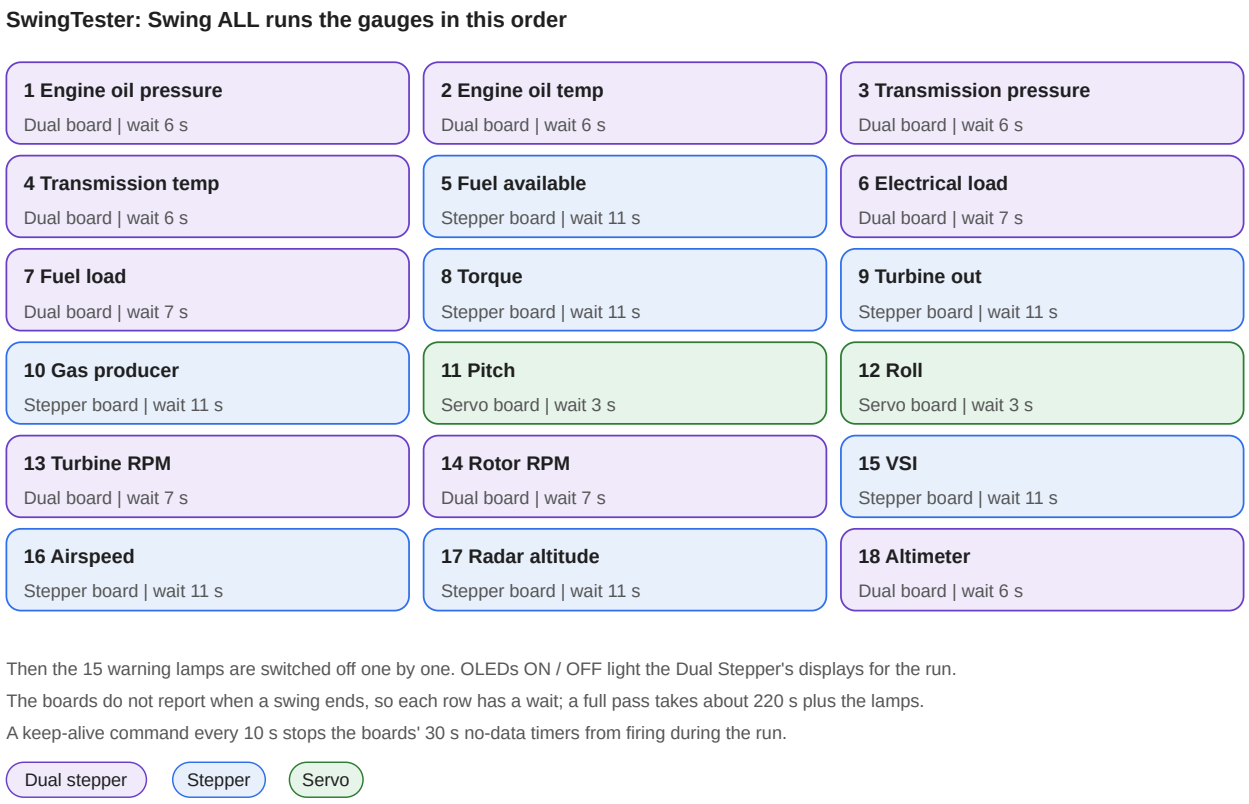



*Figure 36 — Swing ALL sequence*

It lights the OLEDs and the lamps during the run and sends a keep-alive so the boards' 30 second watchdogs do not fire. *Reset boards...* reboots the boards you tick (Radio, Servo, Upper (the retired .103 address), Stepper, Dual Stepper, USB HID) by sending each one `C,RESET:1` (the ADF and Upper board .107 is not in the list yet); each restarts about 2 seconds later and repeats its power-up test.

## 8.3 Known issues

- The backlight PWM was once on pin 8, which the PCB routes to VSI coil B. It is now pin 5, matching the PCB's `BACKLIGHT_PWM` net, in both stepper sketches, and it follows `BPWR`, `C,B:n` and the 30 second no-data watchdog (see Backlight).
- Pin 32 is also claimed by an unused roll stepper in the code.

- The oil, transmission and placeholder gauges use a shared linear scale, not bench-measured tables.

# 9 OLED dual stepper controller

A third Mega, on the Jet Ranger OLED Dual Stepper PCB at 172.16.1.106, drives stepper gauges plus four small OLED displays. It runs `JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER.ino` and, like the 13 Stepper board, takes real units.

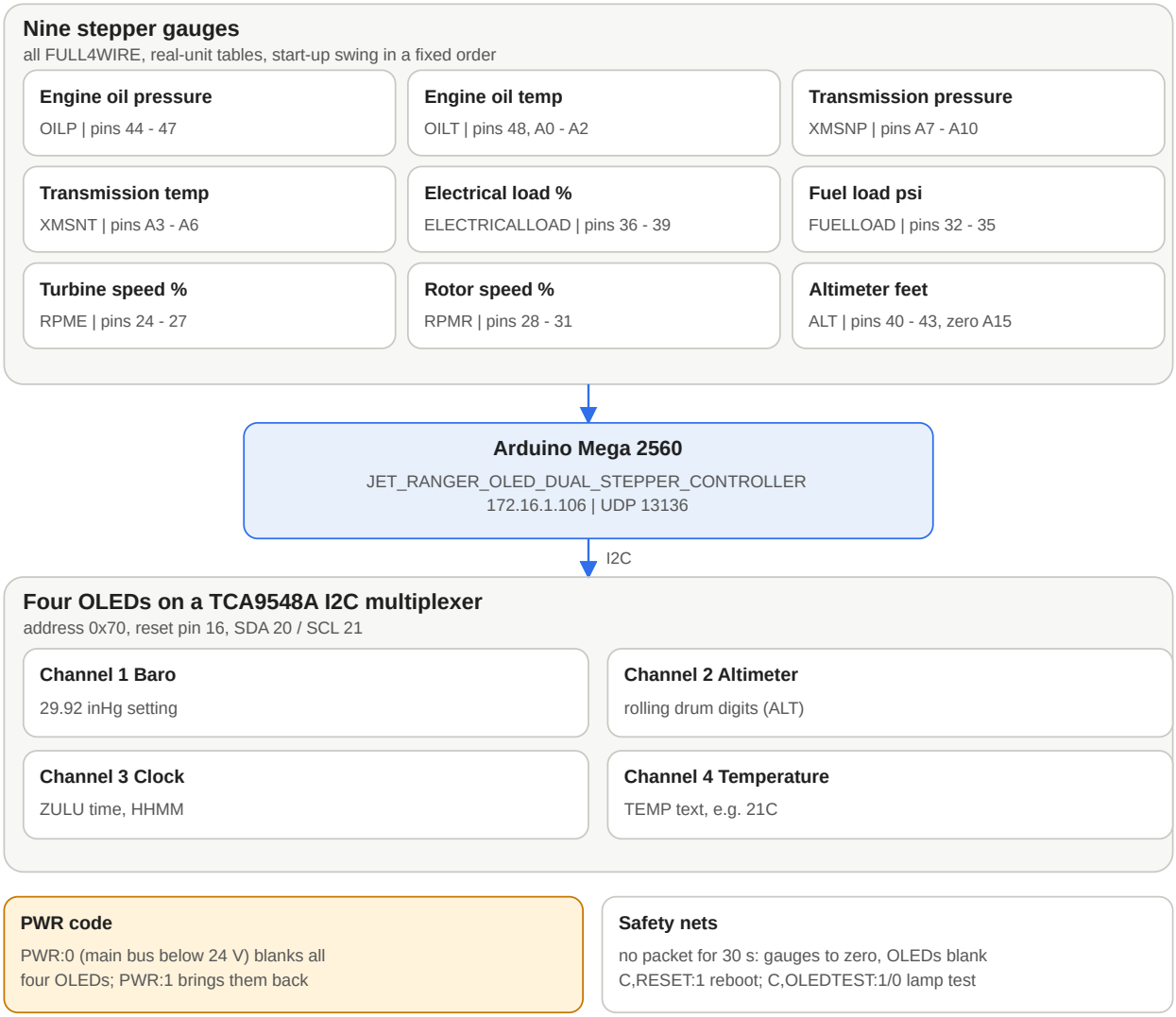



*Figure 37 — Dual stepper board*

| Gauge | Token | Mega pins | Units | Calibration |
|---|---|---|---|---|
| Engine speed (N2) | `RPME` | 24 to 27 | percent, one decimal | 4-point table |
| Rotor speed | `RPMR` | 28 to 31 | percent, one decimal | 4-point table |
| Altimeter | `ALT` | 40 to 43, zero sensor A15 | feet | 720 steps per 1000 ft |
| Fuel load | `FUELLOAD` | 32 to 35 | psi | 2-point table (0 to 30 psi) |
| Electrical load | `ELECTRICALLOAD` | 36 to 39 | percent | 2-point table |
| Engine oil temperature | `OILT` | 48, A0 to A2 | degrees C | 4-point table |
| Engine oil pressure | `OILP` | 44 to 47 | psi | 4-point table |
| Transmission oil temperature | `XMSNT` | A3 to A6 | degrees C | 4-point table |
| Transmission oil pressure | `XMSNP` | A7 to A10 | psi | 5-point table |

Turbine out temperature (`ITT`) and fuel quantity (`FUEL`) are also accepted, as on the 13 Stepper board. Wire each physical gauge to one board only; the matching channel on the other board then simply has nothing connected. Each token also has a `RAW` twin for bench work. `IAS` is not handled here; the FSUIPC and MSFS bridges send airspeed to the 13 Stepper board with one decimal place so the needle moves smoothly.

## 9.1 OLED displays

The four 128 by 32 SSD1306 OLEDs share the Mega's I2C pins (20 SDA, 21 SCL) through a TCA9548A multiplexer at address 0x70. The firmware pulses the mux reset on pin 16 at start-up.

| Display | Mux channel | UDP token | Shows |
|---|---|---|---|
| Barometric setting | 1 | none yet | `2992` at boot; not updated |
| Altimeter digits | 2 | `ALT` | Rolling-drum altitude, raw MSL (barometric correction off) |
| Clock | 3 | `ZULU` | Zulu time from HHMM, for example `ZULU:1430` |
| Temperature | 4 | `TEMP` | Text as sent, for example `TEMP:25C` |

Send `C,OLEDTEST:1` to light all four displays solid and `C,OLEDTEST:0` to restore them. When the main bus is below 24 V the bridge sends `PWR:0` and all four displays go blank; `PWR:1` turns them back on and they repaint within about a second. The altimeter needle keeps following altitude. The lamp test still lights all four displays whatever `PWR` says.

## 9.2 Behaviour

- **Power-up:** OLEDs start first, then each gauge sweeps in turn: engine oil pressure, engine oil temperature, transmission pressure, transmission temperature, electrical load, fuel load, engine RPM, rotor RPM, altimeter. The altimeter seeks its zero sensor before sweeping. Start-up takes about 65 seconds.
- **No-data watchdog:** after 30 seconds without a packet the needles return to zero and all OLEDs blank.
- **On-demand sweeps:** `C,SWINGOILP:1`, `SWINGOILT`, `SWINGXMSNP`, `SWINGXMSNT`, `SWINGELECTRICALLOAD`, `SWINGFUELLOAD`, `SWINGRPME`, `SWINGRPMR`, `SWINGALT`. Each blocks the board for 5 to 7 seconds.
- **Keep-alive:** `DUAL_STEPPER` to the Health Monitor every 10 seconds, so it shows separately from the 13 Stepper board.

## 9.3 Known issues

- RAM use is 75 percent or more; avoid adding more OLEDs or string-heavy code.
- The latest bench I2C scan found OLEDs on mux channels 1 to 4; check by eye that each shows its text.
- Engine and rotor RPM tables assume 0 percent is 0 steps; `RSoffset` is not yet measured.
- The altimeter zero seek travels up to 10,080 steps (about 14 turns) before giving up.

# 10 Radio controller (COM and NAV)

The radio panel is another Mega at 172.16.1.101, running `JET_RANGER_RADIO_CONTROLLER.ino`. It shows the COM and NAV frequencies on two OLEDs, reads eight tuning knobs and a 4 by 5 button matrix, serves the Auto Pilot and Audio panels (switches, knobs and 19 LEDs), and sends knob and button actions back to the PC.

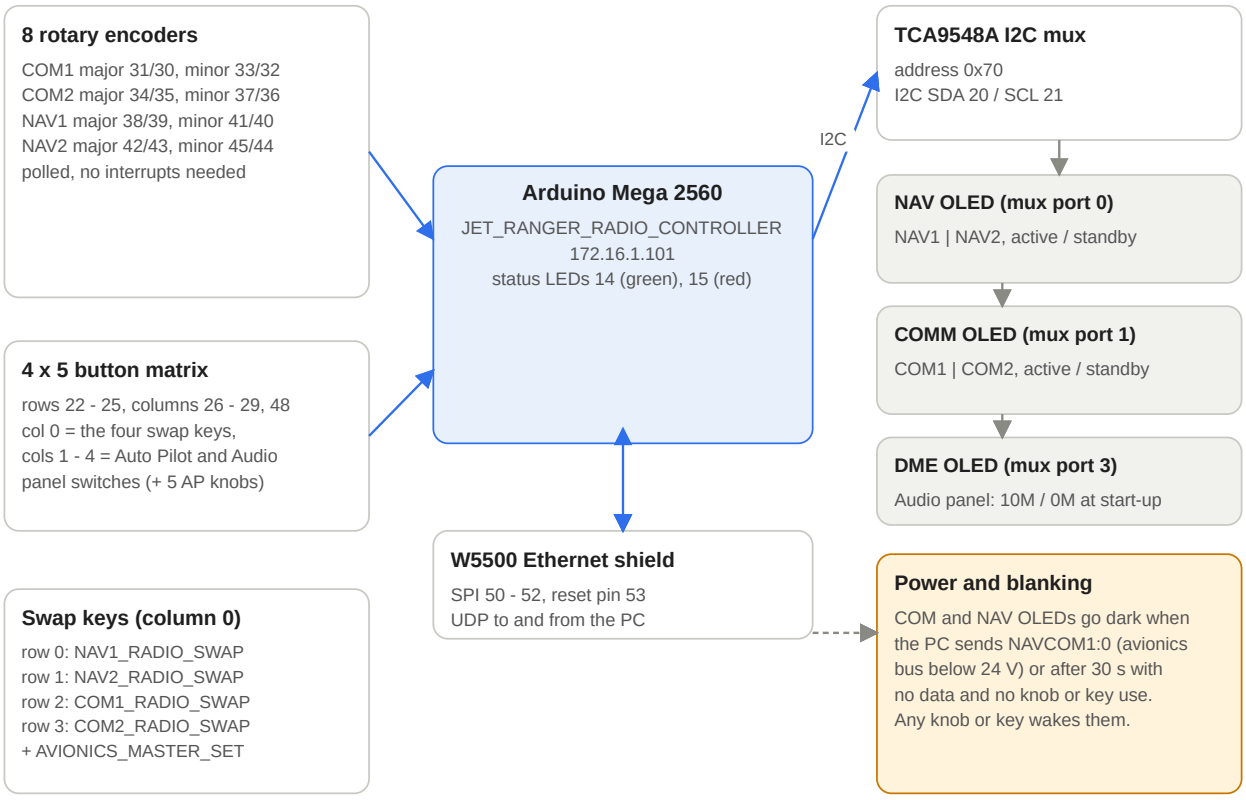



*Figure 38 — Radio panel*

## 10.1 Boards

| PCB | Role | Main parts |
|---|---|---|
| Jet Ranger Radio Auto Pilot | Radio controller board | Mega 2560 shield, Ethernet module, TCA9548A I2C mux |
| Jet Ranger COM and NAV | Front panel board behind the radio faceplate | 8 rotary encoders, push buttons, four frequency-swap switches |

## 10.2 Inputs and outputs

| Function | Mega pins | Notes |
|---|---|---|
| COM1 major and minor knob | 31, 30 and 33, 32 | Encoder A, B |
| COM2 major and minor knob | 34, 35 and 37, 36 | Encoder A, B |
| NAV1 major and minor knob | 38, 39 and 41, 40 | Encoder A, B |
| NAV2 major and minor knob | 42, 43 and 45, 44 | Encoder A, B |
| Button matrix | rows 22 to 25, columns 26, 27, 28, 29 and 48 | 4 by 5: column 0 the COM and NAV swap switches, columns 1 and 2 the Auto Pilot panel, |

| Function | Mega pins | Notes |
|---|---|---|
| | | columns 3 and 4 the Audio panel |
| Auto Pilot knobs | CRS 11, 12; ALT A4, A5; VS 46, 47; IAS 7, 6; HDG 8, 9 | Encoder A, B |
| Panel LEDs (MAX7219) | DIN A15, CLK A14, LOAD A13 | 19 LEDs on the Auto Pilot and Audio panels and the 20 Huey Warning Panel LEDs |
| Huey Warning Panel switches | A9 TEST, A10 RESET, A8 BRT / DIM | Switches to ground; the panel is hand wired at the moment (see Huey Warning Panel) |
| Backlight | D5 | IRF540N MOSFET for the Audio, Auto Pilot and COM and NAV panel backlights |
| OLEDs | I2C through the TCA9548A (0x70) | Port 0 = NAV OLED, port 1 = COM OLED, both at 0x3C |
| Status LEDs | 14 green, 15 red | Red flashes while Ethernet starts |
| Ethernet module | SPI 50 to 52, reset 53 | |

## 10.3 Data from the PC (UDP 13136)

| Token | Meaning | Example |
|---|---|---|
| `C1A`, `C1S` | COM1 active and standby frequency | `C1A:119.500` |
| `C2A`, `C2S` | COM2 active and standby frequency | `C2S:121.900` |
| `NAVCOM1` | Radio power (0 = off): the COM and NAV displays and the panel LEDs go off | `NAVCOM1:1` |
| `MAINBUS` | Main bus voltage | |
| `LED_xxx` | One of the 19 Auto Pilot and Audio panel LEDs (1 = on) | `LED_HDG:1` |
| `LED_H_xxx` | One of the 20 Huey Warning Panel LEDs (1 = on); the bridges do not send these yet | `LED_H_ENG_OIL:1` |
| `APHDG`, `APCRS`, `APIAS`, `APVS`, `APALT` | The five values on the Auto Pilot panel's OLED | `APHDG:090` |
| `BPWR` | Backlight power: 1 = main bus 20 V or more, 0 = backlight off | `BPWR:1` |
| `C,B:n` | Brightness 0 to 15: the panel backlight level (0 off) and the panel LEDs' intensity | `C,B:5` |
| `C,LEDTEST:n` | 1 lights all 39 panel LEDs, 0 returns them to the simulator's state | `C,LEDTEST:1` |

# 10.4 Commands to the PC (UDP 27001)

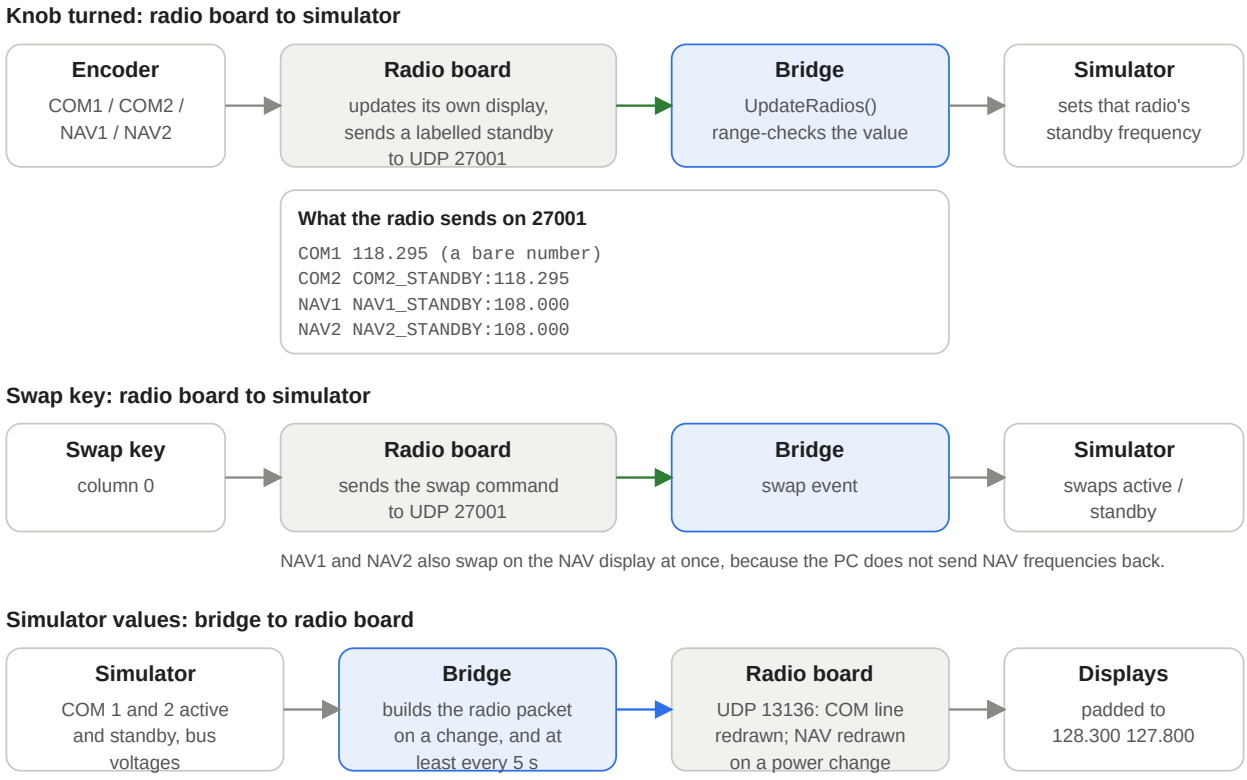



*Figure 39 — Where each input goes*

Turning a knob changes the standby frequency on the OLED and sends the new value to the PC. The four swap switches share matrix column 0:

| Switch (row, column 0) | Command sent |
|---|---|
| Row 0: NAV1 FREQ | `NAV1_RADIO_SWAP` |
| Row 1: NAV2 FREQ | `NAV2_RADIO_SWAP` |
| Row 2: COMM1 | `COM1_RADIO_SWAP` |
| Row 3: COMM2 | `COM2_RADIO_SWAP`, then `AVIONICS_MASTER_SET` |

The bridge listens on 27001 and turns these into simulator events. NAV1 and NAV2 also swap their active and standby frequencies on the NAV OLED straight away, because the PC does not send NAV frequencies back; the COM swaps change the display only when the simulator returns the swapped frequencies. Other buttons are only logged to the debug listener.

Each knob sends its standby frequency with the radio's name: COM1 as the bare number (for example `118.295`), the others as `COM2_STANDBY:118.295`, `NAV1_STANDBY:108.000` or `NAV2_STANDBY:108.000`. Nothing is sent during the first 1.5 seconds after power-up, so the knobs' start positions do not change the simulator. Each OLED line always shows two frequencies in a fixed 16-column layout.

**Tuning steps** follow the standard channel spacing. The COM knobs run 118.000 to 136.975 MHz in 25 kHz channels (the major knob picks 118 to 136, the minor knob 000, 025, 050 and 075). The NAV knobs run 108.00 to 117.95 MHz in 50 kHz steps (major 108 to 117, minor 000, 050, 100 and so on). Each minor knob wraps round at the ends. 8.33 kHz channels are not offered. FSUIPC holds COM frequencies to 10 kHz, so in that simulator a channel such as 118.025 may appear as 118.02 or 118.03.

## 10.5 Auto Pilot and Audio panels

The Auto Pilot and Audio panels plug into the radio controller. Each panel has eight switches, and the Auto Pilot panel also has five knobs (CRS, ALT, VS, IAS and HDG). Every switch and knob is sent **two ways**:

1. as a **joystick button** to the USB HID board (**Joystick 3** for the Auto Pilot panel, **Joystick 4** for the Audio and ADF panels), which you can bind in FSUIPC or MSFS like any other button. A switch holds its button down while pressed; a knob click is a short press, clockwise on one button and anticlockwise on the next;
2. as a **command to the simulator bridge** on UDP 27001, which the FSUIPC and MSFS bridges turn straight into the simulator control of that name. A switch sends it when pressed; a knob sends one for each click (with a repeat count when you turn it quickly). Nothing needs binding for these.

Nothing is sent for the first 1.5 seconds after power-up. The full list, with the ADF panel's switches, is below. The joystick buttons used (Joystick 3 and 4) are not used by any other switch.

| Panel | Control | Matrix or pin | Joystick button (index) | Command to the bridge (UDP 27001) |
|---|---|---|---|---|
| Auto Pilot | AP ENGAGE | row 0, col 1 | Joystick 3, button 1 (64) | `AP_MASTER` |
| Auto Pilot | HDG | row 1, col 1 | Joystick 3, button 2 (65) | `AP_HDG_HOLD` |
| Auto Pilot | CRS | row 2, col 1 | Joystick 3, button 3 (66) | `AP_NAV1_HOLD` |
| Auto Pilot | ALT | row 3, col 1 | Joystick 3, button 4 (67) | `AP_ALT_HOLD` |
| Auto Pilot | APPR | row 0, col 2 | Joystick 3, button 5 (68) | `AP_APR_HOLD` |
| Auto Pilot | FD | row 1, col 2 | Joystick 3, button 6 (69) | `TOGGLE_FLIGHT_DIRECTOR` |
| Auto Pilot | BC | row 2, col 2 | Joystick 3, button 7 (70) | `AP_BC_HOLD` |
| Auto Pilot | IAS | row 3, col 2 | Joystick 3, button 8 (71) | `AP_PANEL_SPEED_HOLD` |
| Auto Pilot | CRS knob clockwise, anticlockwise | D11, D12 | Joystick 3, buttons 9, 10 (72, 73) | `VOR1_OBI_INC`, `VOR1_OBI_DEC` |
| Auto Pilot | ALT knob | A4, A5 | Joystick 3, buttons 11, 12 (74, 75) | `AP_ALT_VAR_INC`, `AP_ALT_VAR_DEC` |
| Auto Pilot | VS knob | D46, D47 | Joystick 3, buttons 13, 14 (76, 77) | `AP_VS_VAR_INC`, `AP_VS_VAR_DEC` |
| Auto Pilot | IAS knob | D7, D6 | Joystick 3, buttons 15, 16 (78, 79) | `AP_SPD_VAR_INC`, `AP_SPD_VAR_DEC` |
| Auto Pilot | HDG knob | D8, D9 | Joystick 3, buttons 17, 18 (80, 81) | `HEADING_BUG_INC`, `HEADING_BUG_DEC` |
| Audio | COM1 | row 0, col 3 | Joystick 4, button 1 (96) | `COM1_TRANSMIT_SELECT` |
| Audio | BOTH | row 1, col 3 | Joystick 4, button 2 (97) | `COM_RECEIVE_ALL_TOGGLE` |
| Audio | NAV2 | row 2, col 3 | Joystick 4, button 3 (98) | `RADIO_VOR2_IDENT_TOGGLE` |
| Audio | DME | row 3, col 3 | Joystick 4, button 4 (99) | `RADIO_DME1_IDENT_TOGGLE` |
| Audio | COM2 | row 0, col 4 | Joystick 4, button 5 (100) | `COM2_TRANSMIT_SELECT` |
| Audio | NAV1 | row 1, col 4 | Joystick 4, button 6 (101) | `RADIO_VOR1_IDENT_TOGGLE` |
| Audio | MKR | row 2, col 4 | Joystick 4, button 7 (102) | `MARKER_SOUND_TOGGLE` |
| Audio | ADF | row 3, col 4 | Joystick 4, button 8 (103) | `RADIO_ADF_IDENT_TOGGLE` |
| ADF | Landing gear switch | ADF controller D31 | Joystick 4, button 9 (104) | `GEAR_SET:1` when closed, `GEAR_SET:0` when open |
| ADF | Flaps switch | ADF controller D32 | Joystick 4, button 10 (105) | `FLAPS_SET:1` when closed, `FLAPS_SET:0` when open |
| ADF | Speed brakes switch | ADF controller D33 | Joystick 4, button 11 (106) | `SPOILERS_SET:1` when closed, `SPOILERS_SET:0` when open |
| ADF | Brakes button | ADF controller D34 | Joystick 4, button 12 (107) | `PARKING_BRAKES` when pressed |
| ADF | ADF major knob clockwise, anticlockwise | ADF controller D6, D7 | Joystick 4, buttons 13, 14 (108, 109) | none (the knob tunes the ADF frequency, `ADF_FREQ`) |

| Panel | Control | Matrix or pin | Joystick button (index) | Command to the bridge (UDP 27001) |
|---|---|---|---|---|
| ADF | ADF minor knob clockwise, anticlockwise | ADF controller D8, D9 | Joystick 4, buttons 15, 16 (110, 111) | none |
| ADF | Anti-ice switch | ADF controller D36 | Joystick 4, button 17 (112) | `ANTI_ICE_SET:1` when closed, `ANTI_ICE_SET:0` when open |
| ADF | Hyd boost switch | ADF controller D35 | Joystick 4, button 18 (113) | none (bind the joystick button in FSUIPC for the Dodo) |
| ADF | Caution dim switch | ADF controller D37 | Joystick 4, button 19 (114) | none (bind the joystick button in FSUIPC for the Dodo) |
| Huey | TEST switch | Radio controller A9 | Joystick 4, button 20 (115) | none (bind the joystick button in the simulator) |
| Huey | RESET switch | Radio controller A10 | Joystick 4, button 21 (116) | none |
| Huey | BRT / DIM switch | Radio controller A8 | Joystick 4, button 22 (117) | none |

If a knob turns the wrong way in the simulator, swap its two encoder pins in the sketch. The rows and columns come from the panel schematics; check each switch on the bench with the debug listener, which prints a line for every press (for example `AP HDG pressed (joystick button index 65)`), and `C,MATRIX:1` for the raw matrix.

## 10.6 Auto Pilot and Audio panel LEDs

One MAX7219 chip on the Radio Auto Pilot PCB lights the 19 LEDs on the two panels (and, since the Huey Warning Panel was added to the sketch, 20 more: see Huey Warning Panel below). Each LED sits between one segment and one digit of the chip:

| LED | Token | MAX7219 digit | Segment | | LED | Token | MAX7219 digit | Segment |
|---|---|---|---|---|---|---|---|---|
| CRS | `LED_CRS` | 0 | A | | COM1 | `LED_COM1` | 2 | A |
| HDG | `LED_HDG` | 0 | B | | BOTH | `LED_BOTH` | 2 | B |
| IAS | `LED_IAS` | 0 | C | | NAV2 | `LED_NAV2` | 2 | C |
| ALT | `LED_ALT` | 0 | D | | DME | `LED_DME` | 2 | D |
| APPR | `LED_APPR` | 1 | A | | OUTER | `LED_OUTER` | 2 | E |
| FD | `LED_FD` | 1 | B | | INNER | `LED_INNER` | 2 | F |
| BC | `LED_BC` | 1 | C | | COM2 | `LED_COM2` | 3 | A |
| AP ENGAGE | `LED_APENG` | 1 | D | | NAV1 | `LED_NAV1` | 3 | B |
| | | | | | MKR | `LED_MKR` | 3 | C |
| | | | | | ADF | `LED_ADF` | 3 | D |
| | | | | | MIDDLE | `LED_MID` | 3 | E |

The bridges send each LED as a `LED_xxx:0/1` pair in the radio packet and the radio lights it, so the panel follows the simulator: the Auto Pilot LEDs show the autopilot and flight director modes (CRS is NAV1 lock), the COM1, COM2, BOTH, NAV1, NAV2, DME, ADF and MKR LEDs show the audio panel selections, and OUTER, MIDDLE and INNER light as you fly over a marker beacon. The LEDs go off with the radio power (avionics bus below 24 V, or no data and no use for 30 seconds). `C,LEDTEST:1` lights all 39 (including the Huey panel's) for checking the wiring and `C,LEDTEST:0` returns them to the simulator's state. The brightness command `C,B:0-15` (the SwingTester slider, or the ADF panel's pot) sets their brightness, and 0 switches them off. The **SwingTester** can light each LED on its own (see Program windows). The FSUIPC bridge reads standard FSUIPC offsets that the DodoSim aircraft may not all drive, and the MSFS bridge has not been checked against the simulator, so some LEDs may stay off until that is confirmed.

## 10.7 Huey Warning Panel

The Huey Warning Panel has 20 caution LEDs and three switches. **It is wired by hand to the Radio controller at the moment** (there is no PCB for it in this guide), and the Radio sketch supports it. The LEDs are on the same MAX7219 as the other two panels, digits 4 to 7, with the tokens `LED_H_ENG_OIL`, `LED_H_ENG_ICE`, `LED_H_ENG_ICE_DETECT`, `LED_H_ENG_CHIP`, `LED_H_LEFT_FUEL_BOOST`, `LED_H_ENG_FUEL_PUMP`, `LED_H_RIGHT_FUEL_BOOST`, `LED_H_20_MINUTE_FUEL`, `LED_H_AUX_FUEL_LOW`,

`LED_H_XMSN_OIL_PRESS`, `LED_H_XMSN_OIL_HOT`, `LED_H_HYD_PRESS`, `LED_H_ENG_INLET_AIR`, `LED_H_INST_INVERTER`, `LED_H_DC_GENERATOR`, `LED_H_EXT_POWER`, `LED_H_FUEL_FILTER`, `LED_H_GOV_EMER`, `LED_H_CHIP_DETECTOR` and `LED_H_IFF`. The three switches (TEST on A9, RESET on A10, BRT / DIM on A8; TEST and RESET are the two positions of one switch) are joystick buttons 20 to 22 on Joystick 4 (indexes 115 to 117). **While TEST is closed all 20 LEDs light** (only while the radio has power). The bridges do not send the `LED_H_xxx` tokens yet, so for now the LEDs light only from the SwingTester (the *Panel LEDs* group has a toggle for each), the lamp test or a test packet. Scanning eight digits instead of four makes the Auto Pilot and Audio LEDs dimmer than before. The Radio on COM20 was flashed with this code on 31 October 2026; the switches and LEDs have not been checked on the panel yet.

## 10.8 Auto Pilot display

The Auto Pilot panel's OLED sits behind a mask, so each value has its own window on the two 16-character lines: **HDG** at the left hand end of the top row and **VS** at the right hand end, **CRS** at the left hand end of the bottom row, **IAS** in the centre of the bottom row and **ALT** at its right hand end. At start-up it shows heading 090, course 110, airspeed 80, vertical speed 0 and altitude 3200:

```
090           0
110    80   3200
```

The bridges then keep the five values up to date from the simulator's autopilot: the heading bug, the NAV1 course (OBS), the autopilot airspeed, vertical speed and altitude settings (heading and course as three digits, north is 360). The display goes blank with the radio power and comes back with it, like the COM and NAV displays. The **SwingTester** has an *Auto Pilot display* group to send test values.

## 10.9 When the USB HID board is not connected

Every Audio and Auto Pilot switch and knob click is also sent to the USB HID board. If that board is switched off or unplugged, the radio now notices when a send fails, stops sending joystick buttons for 30 seconds, and says so in the monitor (`USB HID board 172.16.1.11 not answering`), so the panel stays responsive. When the board returns the buttons that are held down are sent again.

## 10.10 Backlight

The radio controller switches the backlights of the Audio, Auto Pilot and COM and NAV panels from Mega pin D5, using the same rules as the other panels (see Network): on only while the main bus is 20 V or more (`BPWR`), at the last brightness sent (`C,B:0-15`) or a default until one arrives, and off when the radio has had no data and no knob or key use for 30 seconds (a knob or key brings it back). The SwingTester's brightness slider and *Backlight power ON / OFF* buttons test it, and the ADF panel's brightness pot sets it. The brightness also sets the intensity of the Auto Pilot and Audio panel LEDs.

## 10.11 Display behaviour

- If no data arrives from the PC for 30 seconds and no knob or button is used, the COM, NAV, Auto Pilot and DME displays blank (as if the radio had no power) and the radio tells the ADF controller to blank its display and lamps too. Turning a knob, pressing a button or new data switches them back on.
- When the avionics bus is below 24 V the bridge tells the radio the power is off (`NAVCOM1:0`) and the COM, NAV, Auto Pilot and DME displays (and the ADF panel's display) go blank together; they light together again when the bus rises.
- At power-up the matrix is ignored for the first 1.5 seconds so a floating input cannot send false button presses. Each switch is checked every millisecond and reported about 12 ms after it changes, and a noisy reading cannot hold a release back.
- At start-up the radio scans the I2C bus and sends what it finds, behind each multiplexer channel, to the debug monitor (lines starting `I2C`); send `C,I2CSCAN:1` to scan again. The Audio panel's OLED is the **DME display**: it shows `10M` on the top line and `0M` on the bottom line after start-up.
- The radio sends `COMM_NAV` to the Health Monitor every 10 seconds.

## 10.12 Known issues

- Only COM1 and COM2 frequencies arrive from the PC; there are no NAV frequency tokens yet, although the NAV knobs are read.
- The Radio Auto Pilot PCB has volume pots that the radio sketch does not read yet.
- Four encoders (COM1 major and minor, COM2 minor, NAV1 minor) have A and B the other way round from the PCB labels; it only matters if one of those knobs tunes backwards.
- If every button in the matrix's first column stops working, send `C,MATRIX:1` to port 13136: the radio prints the raw H or L state of each column to the debug listener. A column that reads L with nothing pressed is a hardware fault on that Mega pin or board (a faulty or badly seated Mega did exactly this on the bench).

# 11 Joystick (HID) interface

Overhead and console switches reach the simulator as ordinary USB joystick buttons. The ADF and Upper controller scans the overhead switches (the old Upper controller's job) and sends each change over UDP to a small USB board, `UDP_TO_HID`, which Windows sees as four joysticks and a keyboard. The joysticks have buttons only: no axes and no hat switches. Bind them like any other controller, in FSUIPC for Prepar3D and FSX or in the MSFS controls menu.

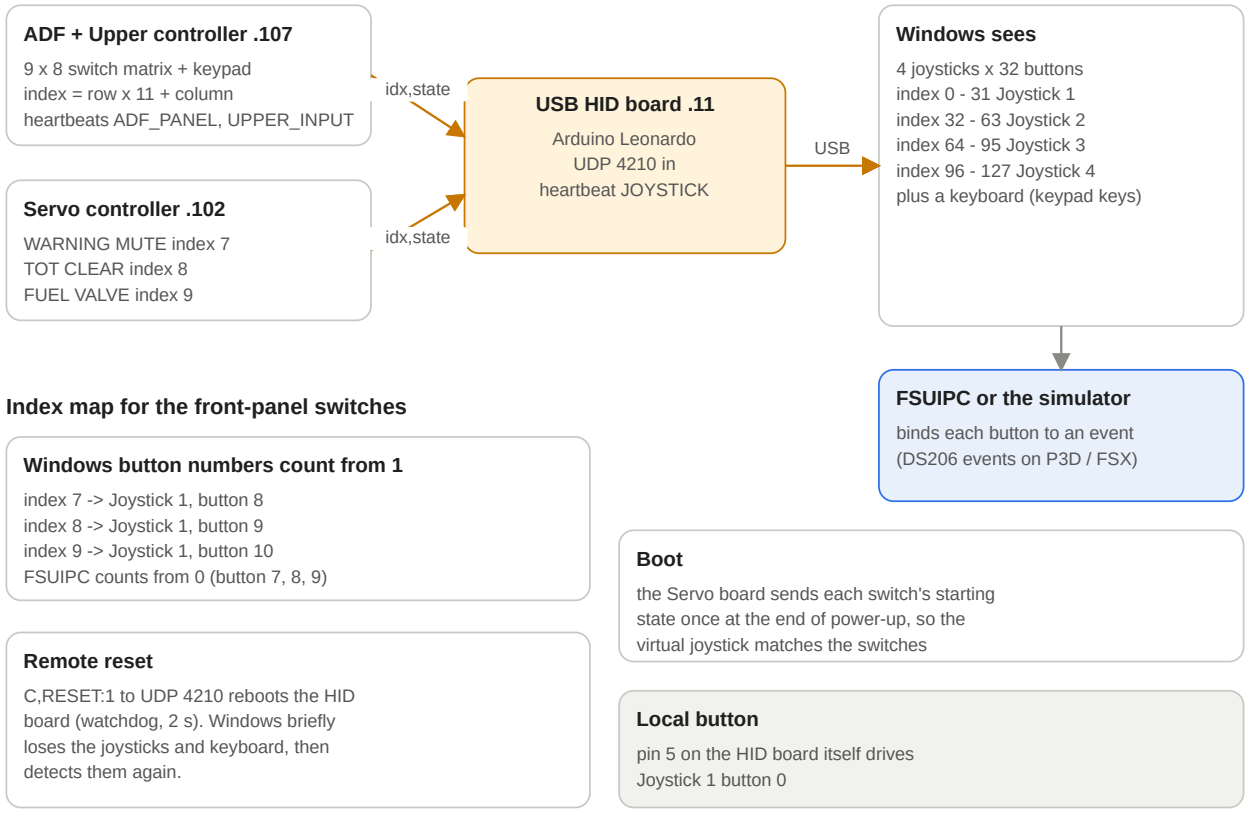



*Figure 40 — From switch to joystick button*

| Board | Sketch | Address | Role |
|---|---|---|---|
| ADF and Upper controller (Mega 2560) | `JET_RANGER_ADF_CONTROLLER.ino` | 172.16.1.107 | Scans a 9 by 8 switch matrix (rows on pins 22 to 30, columns on 38 to 45) and a 4 by 3 keypad (A9 to A15), as well as the ADF panel. It replaces the old Upper controller (`JET_RANGER_UPPER_CONTROLLER.ino`, 172.16.1.103) |
| USB HID board (Leonardo class) | `UDP_TO_HID.ino` | 172.16.1.11 | Listens on UDP 4210 and presents 4 joysticks of 32 buttons plus a keyboard |
| Servo board (Mega 2560) | `JET_RANGER_SERVO_CONTROLLER.ino` | 172.16.1.102 | Also sends three front-panel switches (warning mute, TOT clear, fuel valve) as index 7, 8 and 9 |

## 11.1 How a switch becomes a joystick button

1. The ADF and Upper controller numbers each matrix position as `index = row x 11 + column` (0 to 95). The numbering is the old Upper board's, so existing bindings still apply; the matrix is now 9 rows by 8 columns instead of 16 by 11.
2. When a switch changes it sends `index,state` to `172.16.1.11:4210`, with state 1 = switch on and 0 = off.

3. The HID board picks joystick = index div 32 and button = index mod 32, then holds that button down while the switch is on.
4. Keypad keys are sent as `KEY,<key>,<state>` and typed as keyboard keys 0 to 9, `*` and `#`.
5. The servo board sends the same format for its three switches: index 7 (warning mute, pin 47), 8 (TOT clear, pin 43) and 9 (fuel valve, pin 48), with 1 = closed.

| Index range | Windows device |
|---|---|
| 0 to 31 | Joystick 1 |
| 32 to 63 | Joystick 2 |
| 64 to 95 | Joystick 3 |
| 96 to 127 | Joystick 4 |
| 128 and above | Not sent to Windows (out of range); nothing sends these any more |

## 11.2 Auto Pilot, Audio and ADF panel buttons

The radio controller (Auto Pilot, Audio and Huey panels) and the ADF controller send their switches and knob clicks in **Joystick 3** (indexes 64 to 95, Auto Pilot panel) and **Joystick 4** (96 to 127, Audio, ADF and Huey panels), buttons nothing else uses (the Upper matrix's rows 6 to 8, indexes 66 to 95, are wired but have no switches; do not fit any at 66 to 73 or 77 to 81). Windows numbers them from 1:

| Index | Windows button | Source |
|---|---|---|
| 64 to 71 | Joystick 3, buttons 1 to 8 | Auto Pilot switches: ENGAGE, HDG, CRS, ALT, APPR, FD, BC, IAS |
| 72 to 81 | Joystick 3, buttons 9 to 18 | Auto Pilot knobs (clockwise, anticlockwise): CRS, ALT, VS, IAS, HDG |
| 96 to 103 | Joystick 4, buttons 1 to 8 | Audio switches: COM1, BOTH, NAV2, DME, COM2, NAV1, MKR, ADF |
| 104 to 107 | Joystick 4, buttons 9 to 12 | ADF panel: landing gear, flaps, speed brakes, brakes |
| 108 to 111 | Joystick 4, buttons 13 to 16 | ADF knobs (clockwise, anticlockwise): major, minor |
| 112 to 114 | Joystick 4, buttons 17 to 19 | ADF panel: anti-ice, hyd boost, caution dim |
| 115 to 117 | Joystick 4, buttons 20 to 22 | Huey Warning Panel: TEST, RESET, BRT / DIM |

Every one of these is also sent to the simulator bridge as a control (see the radio controller chapter), so you only need to bind the joystick buttons if you want to use them differently.

## 11.3 Switch mapping in use

From `UpperSwitchMapping.ods`. Button numbers are as Windows and the simulators show them (1 to 32); the firmware counts from 0. A latching switch holds its button down while on, so bind both press and release: in FSUIPC use the `_SET` event with parameter 1 on press and 0 on release.

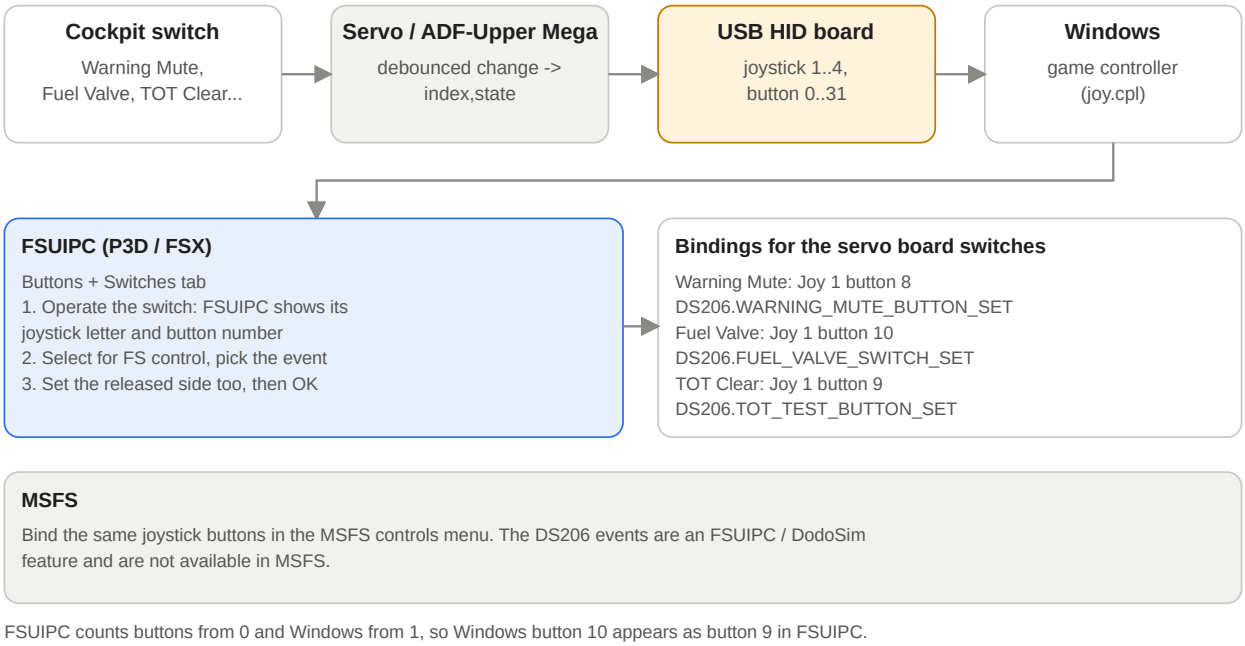
| Index | Source | Switch | Joystick, button | Suggested DodoSim event |
|---|---|---|---|---|
| 0 to 4 | Upper 0, 0-4 | Heater rotary HTR 0 to 4 | J1, 1 to 5 | |
| 5 | Upper 0, 5 | Air vent | J1, 6 | `HEAT_VENT_SWITCH_SET` |
| 6 | Upper 0, 6 | Air heat | J1, 7 | `HEAT_VENT_SWITCH_SET` |
| 7 | Servo D47 | Warning mute | J1, 8 | `DS206.WARNING_MUTE_BUTTON_SET` |
| 8 | Servo D43 | TOT clear (TOT light test) | J1, 9 | `DS206.TOT_TEST_BUTTON_SET` |
| 9 | Servo D48 | Fuel valve | J1, 10 | `DS206.FUEL_VALVE_SWITCH_SET` |
| 11 | Upper 1, 0 | Forward fuel boost breaker | J1, 12 | `FORE_FUEL_PUMP_CB_SET` |
| 12 | Upper 1, 1 | Aft fuel boost breaker | J1, 13 | `AFT_FUEL_PUMP_CB_SET` |
| 13 | Upper 1, 2 | Generator field breaker | J1, 14 | `GENERATOR_FIELD_CB_SET` |
| 14 | Upper 1, 3 | Generator breaker | J1, 15 | `GENERATOR_RESET_CB_SET` |
| 15 | Upper 1, 4 | Directional gyro / attitude indicator | J1, 16 | `DIR_GYRO_SWITCH_SET` |
| 16 | Upper 1, 5 | Battery | J1, 17 | `BATTERY_SWITCH_SET` |
| 17 | Upper 1, 6 | Generator (on / start) | J1, 18 | `GENERATOR_SWITCH_SET` |
| 22 | Upper 2, 0 | Landing light | J1, 23 | `LANDING_LIGHT_SWITCH_SET` |
| 23 | Upper 2, 1 | Instrument lights | J1, 24 | `PANEL_LIGHTS_SWITCH_SET` |
| 24 | Upper 2, 2 | Caution light breaker | J1, 25 | `CAUTION_LT_CB_SET` |
| 25 | Upper 2, 3 | Anti-collision light | J1, 26 | `COLLISION_LIGHTS_SET` |

| Index | Source | Switch | Joystick, button | Suggested DodoSim event |
|---|---|---|---|---|
| 26 | Upper 2, 4 | Position lights | J1, 27 | `POSITION_LIGHTS_SET` |
| 27 | Upper 2, 5 | Pitot heat | J1, 28 | `PITOT_HEAT_SWITCH_SET` |
| 28 | Upper 2, 6 | Avionics | J1, 29 | `AVIONICS_SWITCH_SET` |
| 39 | Upper 3, 6 | Strobe light | J2, 8 | none yet |

The suggested events are the closest match in `DODSIM206V2.EVT`; the bindings actually saved in FSUIPC or MSFS are not in the repository. All other matrix positions are wired but unassigned.

## 11.4 Setting FSUIPC events from the joystick buttons

A joystick button does nothing in the simulator until it is bound to an event in FSUIPC. The **Warning Mute** and **Fuel Valve** switches on the servo board must be set this way before they work; the **TOT clear** switch beside them is bound the same way. This applies to Prepar3D and FSX with FSUIPC and `DODSIM206V2.EVT` installed. In MSFS, bind the buttons in the controls menu instead; the DS206 events are not available there.




FSUIPC counts buttons from 0 and Windows from 1, so Windows button 10 appears as button 9 in FSUIPC.

*Figure 41 — Binding a switch*

1. Start the simulator with the USB HID board connected. Windows should list the four joysticks (Game Controllers, `joy.cpl`); pressing a switch lights the matching button there.
2. In the simulator open FSUIPC from the Add-ons menu and choose the **Buttons + Switches** tab.
3. Operate the switch you want to bind (for example flip the Fuel Valve switch). FSUIPC shows the joystick letter and button number it saw. FSUIPC counts buttons from 0 and Windows from 1, so Windows button 10 shows as button 9 in FSUIPC.
4. Under the pressed (or closed) side, tick *Select for FS control*, choose the event from the list and type the parameter in the box beside it. The DS206 events are listed alphabetically.
5. A switch that stays on needs the released side set as well: tick *Select for FS control* there, choose the same event and enter the opposite parameter, so the cockpit follows the switch in both directions.
6. Click OK, flip the switch and check the result in the cockpit. After an event file update FSUIPC re-indexes the events alphabetically, so check these bindings again.

- **Warning Mute:** Joystick 1 button 8 (FSUIPC button 7). Event `DS206.WARNING_MUTE_BUTTON_SET`, parameter 1 when pressed and 0 when released. Alternatively bind `DS206.WARNING_MUTE_BUTTON_TRIGGER` (no parameter) to the pressed side only.

- **Fuel Valve:** Joystick 1 button 10 (FSUIPC button 9). Event `DS206.FUEL_VALVE_SWITCH_SET`, parameter 1 when closed and 0 when released. The valve in the simulator follows the switch; if it ends up the wrong way round, swap the two parameters.
- **TOT clear (TOT light test):** Joystick 1 button 9 (FSUIPC button 8). Event `DS206.TOT_TEST_BUTTON_SET`, parameter 1 when pressed and 0 when released.

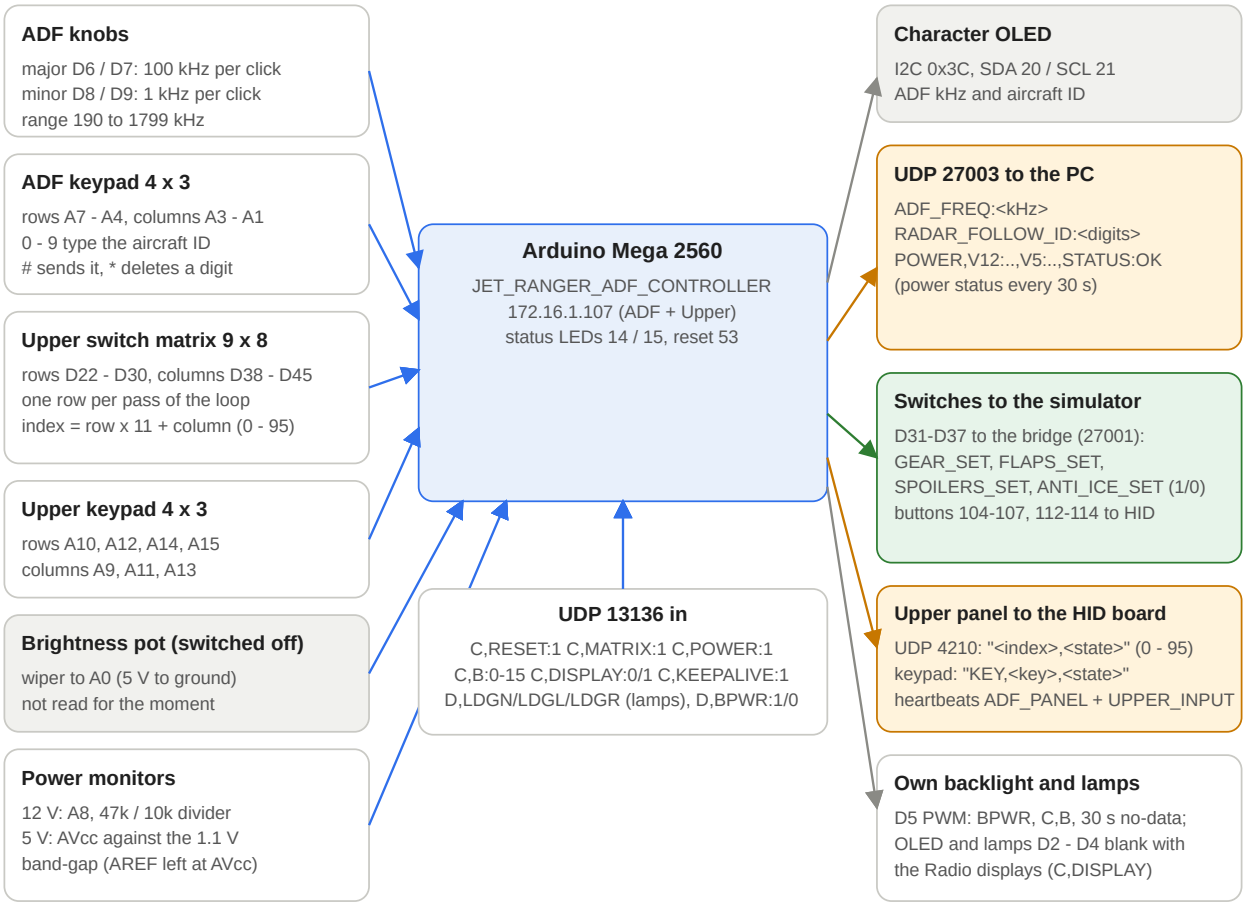
If FSUIPC does not see a switch, check that Windows shows the button moving in `joy.cpl` first. If it does not, the problem is on the board side: the servo board's debug messages report each change and the HID board logs the button it sets.

## 11.5 Known issues

- The HID board's own test button on pin 5 also drives Joystick 1 button 1, the same button as HTR 0.
- The old Upper sketch's DCS-BIOS stubs were not carried into the merged ADF and Upper sketch, so nothing is sent to DCS; only the joystick path works.
- The HID board reboots when it receives `C,RESET:1` (or `RESET`) on UDP 4210. Its USB connection drops while it restarts, so Windows briefly loses the four joysticks and the keyboard; re-check any game that has them open.
- The servo board's three switches use indexes 7, 8 and 9, which Windows shows as Joystick 1 buttons 8, 9 and 10. If they were meant to be Windows buttons 7, 8 and 9, button 7 is already the Air heat switch (index 6), so the indexes would have to move.

# 12 ADF and Upper controller

The ADF panel is another Mega, at 172.16.1.107, running `JET_RANGER_ADF_CONTROLLER.ino` on the Jet Ranger Upper ADF Controller PCB (revision 1.0), the combined ADF and Upper board. Since 31 October 2026 it also does the job of the old Upper controller (172.16.1.103): it scans the **overhead switch matrix** and the **Upper keypad** and sends them to the USB HID board (see Upper panel switches and keypad below), so there is no separate Upper board any more. It tunes the ADF with two knobs, takes the ID of an aircraft to follow on the radar from a keypad, sends its landing gear, flaps, speed brakes, brakes, anti-ice, hyd boost and caution dim switches to the simulator, lights the three landing gear lamps, blanks its display, lamps and backlight with the other panels, and watches the 12 V and 5 V supplies. The PCB has a brightness pot, but it is **switched off for the moment**. The sketch is flashed (COM26); the 12 V reading, the landing gear lamps and the ADF keypad have been checked on the bench, but the Upper matrix and keypad, the switches at their new pins and the blanking and backlight rules have not.




The toggle switches send their position as a value (every change and every 5 s); the brakes push button sends PARKING_BRAKES.
The Upper panel (switch matrix and keypad) was the separate Upper controller, 172.16.1.103, until it was merged into this sketch.

*Figure 42 — ADF panel*

## 12.1 Using the panel

| Control | What it does |
|---|---|
| Major knob | Changes the ADF frequency by 100 kHz per click |
| Minor knob | Changes it by 1 kHz per click (faster when you turn it quickly). The range is 190 to 1799 kHz |
| Keypad 0 to 9 | Types the ID of the aircraft to follow (up to 8 digits) |
| Keypad # | Sends the ID |

| Control | What it does |
|---|---|
| Keypad * | Deletes the last digit; on an empty entry it stops following |
| Landing gear, flaps and speed brakes switches | Set the simulator to match the switch position (see Panel switches) |
| Brakes button | Toggles the parking brake |
| Anti-ice, hyd boost and caution dim switches | Joystick 4 buttons 17 to 19; anti-ice also sets the simulator's anti-ice (see Panel switches) |
| Brightness pot | Not used for the moment: the panel backlight stays at its default and no brightness is sent to the other panels |

The OLED shows the frequency on the first line (`ADF  340 kHz`) and the ID on the second: `ID: 1234_` while you type, `FOLLOW 1234` once it is sent, or `FOLLOW OFF`. An entry you leave unfinished for 20 seconds is dropped.

**Brightness pot.** The pot on the PCB (A0) can set the backlight of this panel and of the Radio, Servo, Stepper and Dual Stepper panels, but `USE_BRIGHTNESS_POT` in the sketch is 0, so it does nothing: this panel's backlight stays at its default (level 4) until a `C,B` command sets it, and the other panels follow the SwingTester slider or their own defaults. Set it to 1 and reflash to bring the pot back.

## 12.2 Inputs and outputs

| Function | Mega pins | Notes |
|---|---|---|
| ADF major knob | D6, D7 | Encoder A, B |
| ADF minor knob | D8, D9 | Encoder A, B |
| ADF keypad | scan rows A7, A6, A5, A4, scan columns A3, A2, A1 | 4 by 3; the lines arrive in reverse order on this PCB; see the key table below |
| Upper switch matrix | rows D22 to D30, columns D38 to D45 | 9 by 8, 72 switches (see Upper panel switches and keypad) |
| Upper keypad | rows A10, A12, A14, A15, columns A9, A11, A13 | 4 by 3 |
| Panel brightness pot | A0 | Wiper, 5 V to ground; not read while the pot is switched off |
| 12 V monitor | A8 | 47 k and 10 k divider with 0.1 uF |
| 5 V monitor | AREF / AVcc | Measured against the internal 1.1 V reference; the AREF pin is left at its default |
| Backlight | D5 | IRF540N MOSFET: level 4 by default, then `C,B`; off with `BPWR:0`, the no-data timeout or no data (see Panel blanking and backlight) |
| OLED | I2C D20, D21 (0x3C) | 2 lines of 16 characters |
| Status LEDs | D14 green, D15 red | Red flashes while Ethernet starts |
| Landing gear lamps | D2, D3, D4 | Nose, left, right: lit by data packets (see below) |
| Landing gear, flaps, speed brakes and brakes switches | D31 to D34 | Switches to ground; see Panel switches |
| Hyd boost, anti-ice and caution dim switches | D35, D36, D37 | Switches to ground (connector J20); see Panel switches |
| Ethernet module | SPI 50 to 52, reset 53 | |

**Keypad keys.** The ADF keypad's seven lines come out of pins 18 to 24 of the ADF panel connector (J5) to A1 to A7 (connector J41). On this PCB they arrive in reverse order, so the sketch scans A7, A6, A5 and A4 as the rows and A3, A2 and A1 as the columns (the old ADF board used D22 to D28). With the first guess at the pins keys 1, 2 and 3 did nothing and most others reported as the wrong key; after the swap every key reports correctly. The key labels in the sketch were set by pressing every key and moving each label to where it was reported:

| Scan row | Column 0 (A3) | Column 1 (A2) | Column 2 (A1) |
|---|---|---|---|
| 0 (A7) | 6 | 4 | 5 |
| 1 (A6) | 9 | 7 | 8 |
| 2 (A5) | # | * | 0 |
| 3 (A4) | 3 | 1 | 2 |

If the keys ever need checking again, send `C,MATRIX:1` while holding a key: the debug listener shows which row and column changes.

## 12.3 Panel switches and joystick buttons

The seven switches are read when they change (30 ms debounce, to ground when on). Each change is sent two ways, like the Auto Pilot and Audio panels:

| Switch | Joystick button (index) | What goes to the bridge (UDP 27001) |
|---|---|---|
| Landing gear (toggle) | Joystick 4, button 9 (104) | The position: `GEAR_SET:1` when closed (gear down), `GEAR_SET:0` when open |
| Flaps (toggle) | Joystick 4, button 10 (105) | `FLAPS_SET:1` (flaps fully down) or `FLAPS_SET:0` |
| Speed brakes (toggle) | Joystick 4, button 11 (106) | `SPOILERS_SET:1` (fully out) or `SPOILERS_SET:0` |
| Brakes (push button) | Joystick 4, button 12 (107) | `PARKING_BRAKES` when pressed |
| Anti-ice (toggle) | Joystick 4, button 17 (112) | `ANTI_ICE_SET:1` or `ANTI_ICE_SET:0` |
| Hyd boost (toggle) | Joystick 4, button 18 (113) | none |
| Caution dim (toggle) | Joystick 4, button 19 (114) | none |
| ADF major knob (clockwise, anticlockwise) | Joystick 4, buttons 13, 14 (108, 109) | none: the knob tunes the ADF frequency |
| ADF minor knob (clockwise, anticlockwise) | Joystick 4, buttons 15, 16 (110, 111) | none |

The gear, flaps, speed brakes and anti-ice toggle switches send their **position as a value** (not an event the simulator could toggle), every time they change and again **every 5 seconds**, so the simulator always follows the switches. That includes the position they were in at power-up: about 5 seconds after the panel is switched on, the aircraft's gear, flaps and speed brakes move to match the switches. For 1.5 seconds after power-up the switches are only read. If a switch works the wrong way round in the simulator, swap its 1 and 0 in the sketch (`adfSwitches[]`). A knob click is a short joystick press.

**Anti-ice, hyd boost and caution dim on the Dodo.** The Dodo Jet Ranger has its own events for these switches: `ANTI_ICE_SWITCH_SET`, `HYDRAULICS_SWITCH_SET` (hyd boost) and `CAUTION_LIGHT_SWITCH_SET` (caution lights bright or dim), each taking 1 or 0. In FSUIPC's Buttons + Switches page, bind Joystick 4 buttons 17, 18 and 19 to them (the switch pins are D36 anti-ice, D35 hyd boost and D37 caution dim), with 1 on press and 0 on release (and turn on the Dodo's "Sync controls on load" option). The simulator's own controls for these are toggles, so the bridges have no reliable command for hyd boost and caution dim, and the Dodo may ignore the bridge's `ANTI_ICE_SET`: use the FSUIPC binding **or** the bridge command for anti-ice, not both.

## 12.4 Upper panel switches and keypad

The overhead switch matrix and the numeric keypad of the old Upper controller are scanned by this board. Each of the 72 matrix switches (9 rows by 8 columns) is sent to the USB HID board as a joystick button, numbered `row x 11 + column` (0 to 95) as before, so the FSUIPC and MSFS button assignments and the switch table in the Joystick section do not change. The matrix is smaller than the old board's 16 by 11 (176 positions). The Upper keypad types keys 0 to 9, `*` and `#` on the HID board's keyboard. The Warning Mute and TOT Test connectors on the PCB are part of the matrix (indexes 88 and 89), nothing is assigned to them yet. A switch that is closed when the board starts is reported straight away. Everything on the panel counts as input for the 30 second no-data timeout. The old Upper board's backlight is this board's backlight.

## 12.5 Landing gear lamps

The three lamps on the ADF panel (nose, left and right) are lit by a data packet to the ADF controller on UDP 13136: `D,LDGN:1,LDGL:1,LDGR:0` (1 = lit, 0 = off). The **SwingTester** has a toggle for each lamp and *All ON* and *All OFF* buttons (see Program windows). The simulator bridges do not send them yet, so for now they are a bench test. They go out with the rest of the panel and come back as last set.

## 12.6 Power status

Five seconds after start-up, and then every **30 seconds**, the board sends the two rail voltages to the PC (UDP 27003) and to the debug listener (UDP 27000):

`POWER,V12:12.41,V5:5.02,STATUS:OK`

`STATUS` is `OK`, or the rails that are out of range: `5V_LOW` (below 4.75 V), `5V_HIGH` (above 5.25 V), `12V_LOW` (below 10.8 V) or `12V_HIGH` (above 15.0 V). Send `C,POWER:1` to get a reading at once. The first time you run the board, compare the readings with a meter and trim `V12_CAL` and `V5_CAL` at the top of the sketch. The same two readings are in the heartbeat the board sends every 10 seconds (`ADF_PANEL V5:5.03 V12:12.41`, and again as `UPPER_INPUT` so the Health Monitor's Upper light stays green), and the **Health Monitor** shows them in its voltage table: this board is the only one with a 12 V reading. The 12 V divider is on A8 on this PCB (it was on A15 on the old ADF board, and a sketch reading A15 reported 5.33 V with 12.3 V on the board).

## 12.7 Messages to the PC (UDP 27003)

| Message | Meaning |
|---|---|
| `ADF_FREQ:830` | The ADF frequency in kHz, sent 250 ms after the knobs stop |
| `RADAR_FOLLOW_ID:4521` | The aircraft ID typed on the keypad; `RADAR_FOLLOW_ID:0` means stop following |
| `POWER,V12:12.41,V5:5.02,STATUS:OK` | Power status, every 30 seconds |

These use port 27003, not 27001, because the FSUIPC and MSFS bridges treat anything they do not recognise on 27001 as a COM1 standby frequency. (The panel switch commands above do go to 27001: the bridges recognise those names.) **Neither bridge reads port 27003 yet**, so the frequency and the follow ID do not reach the simulator.

## 12.8 Control packets

The board acts on control packets on UDP 13136: `C,RESET:1` reboots it, `C,MATRIX:1` prints the raw ADF keypad readings to the debug listener, `C,POWER:1` sends a power status, `C,B:0` to `C,B:15` sets the backlight brightness (while the pot is switched off), `C,KEEPALIVE:1` counts as data and `C,DISPLAY:0` / `C,DISPLAY:1` (sent by the Radio controller) blank and restore the panel. Of the data packets it uses the three landing gear lamp tokens and `BPWR`.

## 12.9 Panel blanking and backlight

The panel follows the other boards' rules:

- **With the Radio.** When the Radio controller blanks its displays (no avionics power, or no data or input for 30 seconds) it tells the ADF controller, which clears its OLED and puts the landing gear lamps out. They come back when the Radio's displays do.
- **Main bus.** The bridges send `BPWR` to the ADF controller like the other boards; `BPWR:0` switches the backlight off and `BPWR:1` brings it back.
- **Brightness.** `C,B:0` to `C,B:15` sets the backlight (the SwingTester slider sends it); the default is level 4.
- **No data.** 30 seconds with no data packet from the bridge and no knob, key or switch use blanks the OLED, the lamps and the backlight until something arrives, so the panel goes dark if the PC is not feeding it.

## 12.10 Checking the panel for the first time

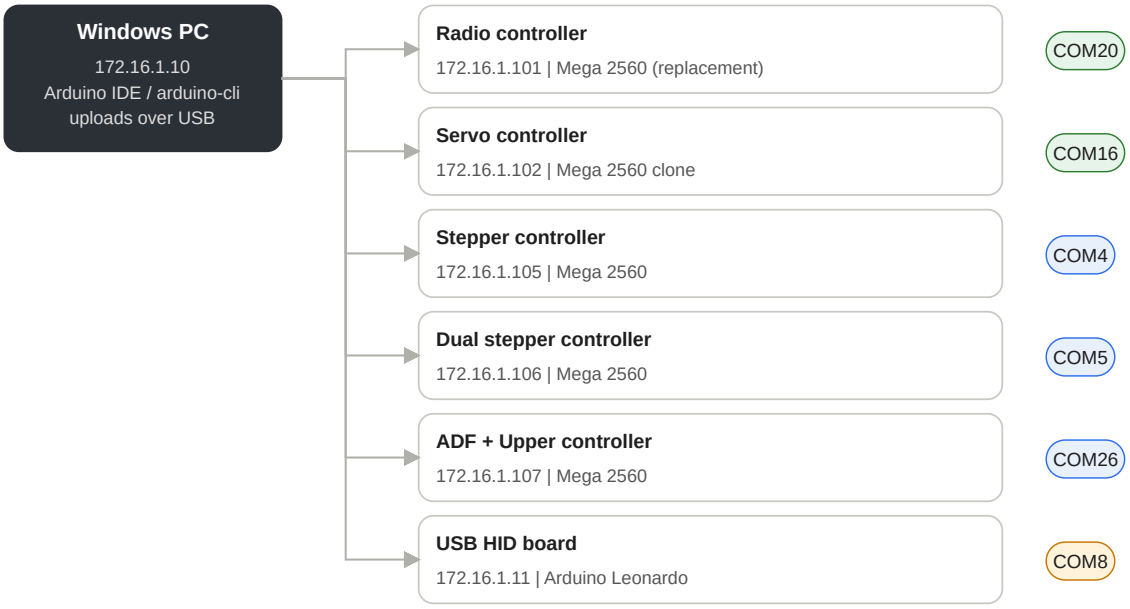
1. Flash the board (see COM ports and flashing), then watch the debug listener for `Jet Ranger ADF: Setup Complete`.
2. Press each ADF keypad key and check the OLED shows the key you pressed (`C,MATRIX:1` shows the raw readings if not). Operate a few Upper switches and keys and watch the debug listener for `Front Input - <index>:<state>` and `Upper key pressed: <key>`.
3. Turn each knob. If it tunes the wrong way, swap its two pins in the `RotaryEncoder` lines.
4. Light the three landing gear lamps from the SwingTester.

5. Check the OLED. The sketch drives the same kind of character OLED as the Radio controller; a graphics SSD1306 needs different code.
6. Compare the power status with a meter and trim `V12_CAL` and `V5_CAL`.

# 13 COM ports and flashing

Each Arduino is programmed over its own USB cable from the primary PC. The cockpit itself runs over Ethernet, so USB is only needed to upload a sketch.

## 13.1 COM port to Arduino mapping




COM numbers are the ones used at the last flash. They follow the USB socket and the board, so check them (see the guide) before an upload. Each board also has a network connection: USB is for programming only, the cockpit runs over UDP.

*Figure 43 — Which board is on which COM port*

| Board | Address | Hardware | COM port | Sketch |
|---|---|---|---|---|
| Radio controller | 172.16.1.101 | Arduino Mega 2560 (replacement board) | **COM20** | `JET_RANGER_RADIO_CONTROLLER` |
| Servo controller | 172.16.1.102 | Mega 2560 clone | **COM16** | `JET_RANGER_SERVO_CONTROLLER` |
| Stepper controller | 172.16.1.105 | Mega 2560 | **COM4** | `JET_RANGER_STEPPER_CONTROLLER` |
| Dual stepper controller | 172.16.1.106 | Mega 2560 | **COM5** | `JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER` |
| ADF and Upper controller (combined) | 172.16.1.107 | Mega 2560 | **COM26** | `JET_RANGER_ADF_CONTROLLER` (the old `JET_RANGER_UPPER_CONTROLLER` is retired) |
| USB HID board | 172.16.1.11 | Arduino Leonardo | **COM8** | `UDP_TO_HID` |

These are the ports used the last time each board was flashed. A COM number belongs to a USB socket and a board together, so it **can change**:

- The Servo controller is a clone Mega that reports no USB serial number, so Windows ties its COM number to the USB socket it is plugged into. Plug it into a different socket and it appears as a new port.
- Replacing a Mega (the Radio controller was replaced once) gives a new port number.
- The Lowrance GPS cable is separate: the GPS interface asks for its serial port the first time it runs and remembers it.

Always check the port before an upload: open Device Manager, Ports (COM and LPT), or run `arduino-cli board list`. If you are not sure which board is which, unplug one USB cable and see which port disappears.

Add the confirmed port to the table above (and to the sketch header comment) once you have uploaded to a board.

## 13.2 Flashing a board

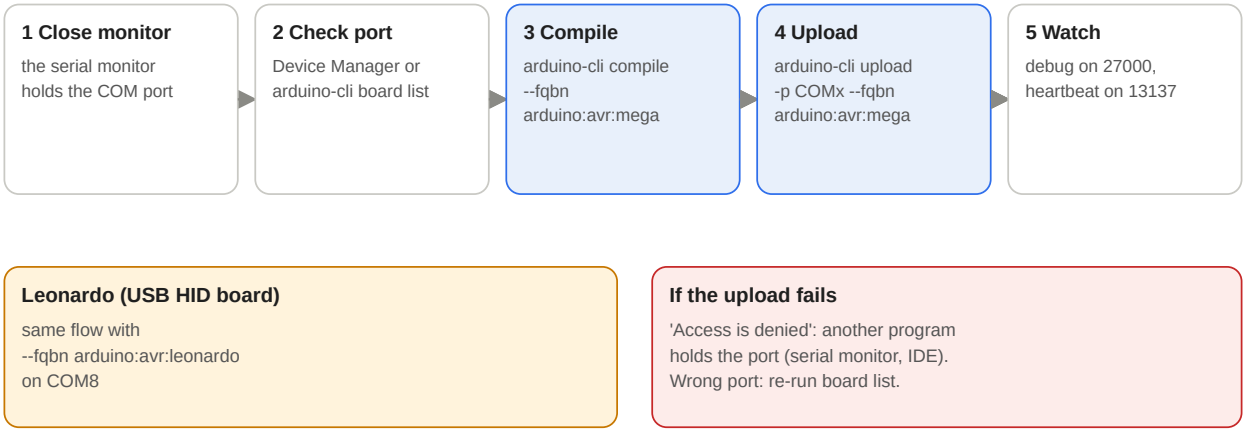



*Figure 44 — Flashing a board*

1. Close the Arduino IDE **serial monitor**. It holds the port, and an upload then fails with *Access is denied*.
2. Check the COM port (above).
3. Compile the sketch.
4. Upload to that port.
5. Watch the debug listener (UDP 27000) and the Health Monitor: the board prints *Ethernet Started* and *Setup Complete* and its light goes green within about 10 seconds.

```
arduino-cli compile --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
arduino-cli upload -p COM4 --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
```

For the Leonardo (USB HID board) use `--fqbn arduino:avr:leonardo` and its port, COM8. Flash only the board you are working on. If a board ever reboot-loops after a reset packet, power-cycle it and flash it again.

# 14 PCBs and wiring

The Mega sits on a carrier PCB that breaks its pins out to servo headers, lamp outputs, gauge sockets and the Ethernet module. KiCad projects for every board are in `Jet Ranger/PCBs` in the repository, with Gerbers in each `MANUFACTURING` folder. The list below was brought up to date on 18 October 2026.

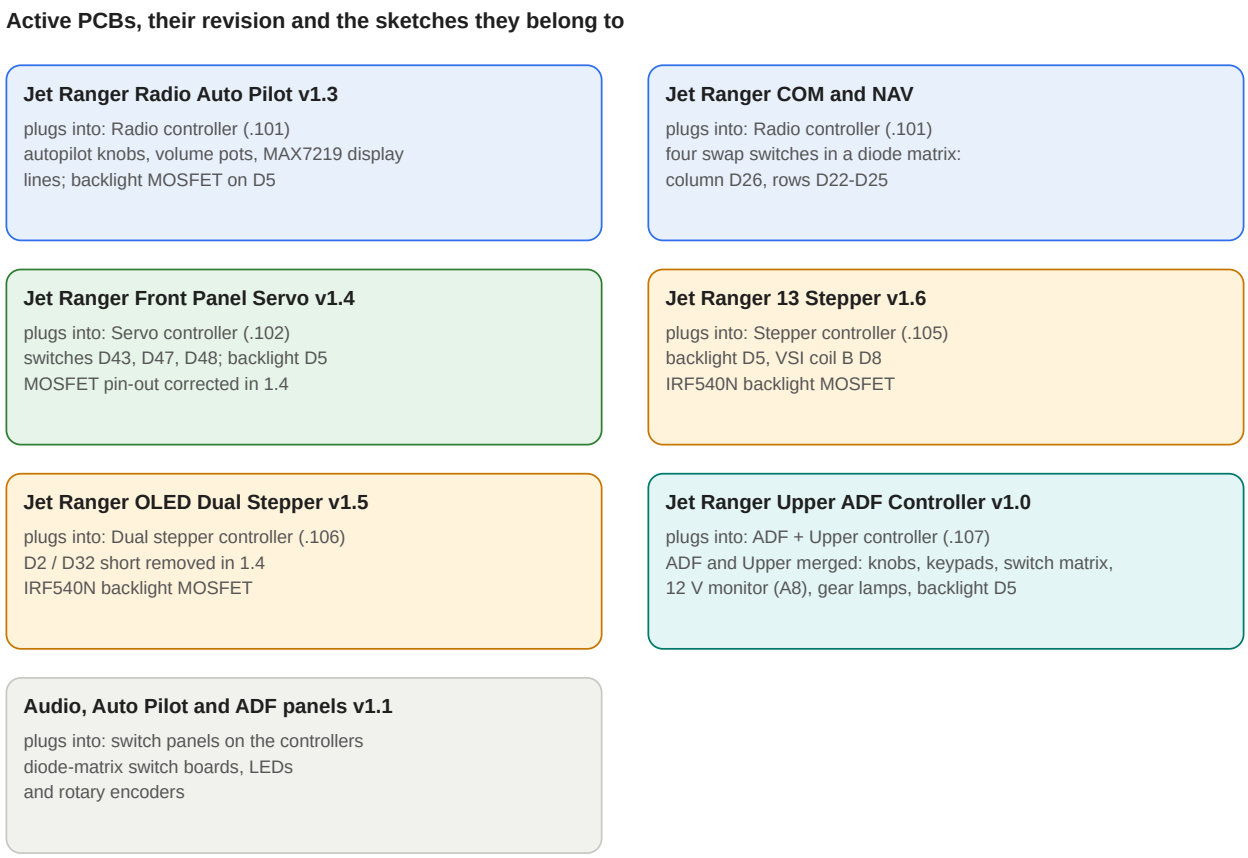


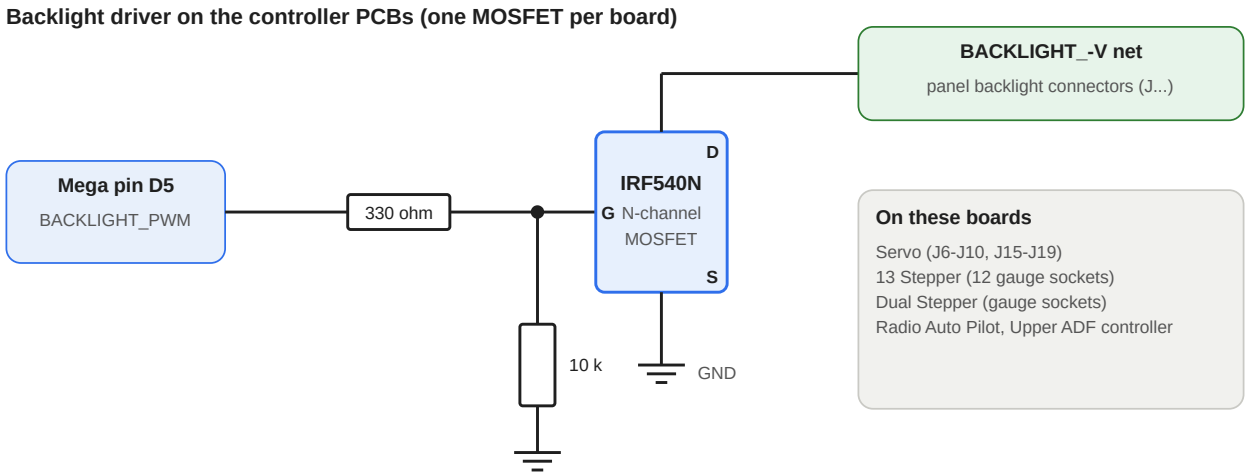

*Figure 45 — Active PCBs, their revision and their sketches*

| PCB | Revision | Used by | Main parts |
|---|---|---|---|
| Jet Ranger Front Panel Servo | 1.4 | Servo board | Mega shield, ULN2803A lamp drivers, servo headers, lamp outputs, three switch inputs, backlight MOSFET |
| Jet Ranger 13 Stepper | 1.6 | Stepper board | Mega shield, USR-ES1 Ethernet, 14 six-pin gauge sockets, 12 V lamp supply with polyfuses, backlight MOSFET |
| Jet Ranger OLED Dual Stepper | 1.5 | Dual stepper board | Mega shield, USR-ES1, six-pin gauge sockets, TCA9548A sockets for the OLEDs, backlight MOSFET |
| Jet Ranger X27 Small Stepper | not set | Each stepper gauge | One X27 or VID29 stepper, backlight LED, six-pin connector to the controller |
| Jet Ranger Radio Auto Pilot | 1.3 | Radio controller | Radio and autopilot controller with the backlight MOSFET |
| Jet Ranger COM and NAV | not set | Radio panel | 8 rotary encoders, push buttons, four frequency-swap switches |
| Jet Ranger Upper ADF Controller | 1.0 | ADF and Upper controller (replaces | ADF knobs, ADF keypad, overhead |

| PCB | Revision | Used by | Main parts |
|---|---|---|---|
| | | the Upper Controller 1.4 and ADF Controller 1.6 boards) | switch matrix and Upper keypad, panel brightness pot, 12 V monitor (A8), gear lamps, anti-ice, hyd boost and caution dim switch inputs, I2C OLED, backlight MOSFET |
| Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel | 1.1 | Switch panels | Diode-matrix switch boards with LEDs and rotary encoders |

The X27 steppers run straight from the Mega's pins, so the stepper boards need no driver chips; the 12 V input feeds lamps and backlights. The schematic and board layout of every PCB are in the *PCB schematics and layouts* chapter at the end of this guide.

## 14.1 Backlight MOSFET (revisions of 18 October 2026)




Revisions 1.3 to 1.5 (18 October 2026) replaced the generic MOSFET symbol, which had source and drain swapped, with the IRF540N.
A board built from an earlier revision has the transistor pinned wrongly: check it, or fit it the right way round.

*Figure 46 — The backlight driver*

Every controller board switches its panel backlights with one **IRF540N** MOSFET: Mega pin D5 (`BACKLIGHT_PWM`) goes through 330 ohm to the gate, a 10 k resistor holds the gate to ground, the drain is the shared backlight net and the source is ground. The earlier revisions used a generic MOSFET symbol with **source and drain swapped**, so the 1.4 Servo, 1.3 Radio Auto Pilot, 1.4 Upper, 1.3 ADF Controller and 1.5 Stepper and Dual Stepper boards were revised (the combined Upper ADF Controller started from the corrected ADF design) to use the IRF540N with the correct pin-out. If you have a board from an earlier revision, check the transistor before relying on the backlight.

### 14.1.1 Backlight resistor values

The backlight circuit uses two groups of resistors.

| Where | Resistor | Value |
|---|---|---|
| Controller board, Mega pin D5 to the MOSFET gate | Gate resistor | 330 ohm |
| Controller board, MOSFET gate to ground | Gate pull-down | 10 kohm |
| Backlight LEDs wired in a chain on the 12 V backlight supply, 3 LEDs in the chain | Series resistor | 150 ohm |
| Backlight LEDs wired in a chain on the 12 V backlight supply, 2 LEDs in the chain | Series resistor | 330 ohm |
| Backlight LEDs wired in a chain on the 12 V backlight supply, 1 LED in the chain | Series resistor | 560 ohm |

The more LEDs there are in a chain, the lower the series resistor, because the LEDs use up more of the supply voltage. The Jet Ranger X27 Small Stepper board has two backlight LEDs in one chain; its schematic currently shows a **470 ohm** resistor, which is not the 330 ohm in this table, so check the value fitted.

**Version 1.4 of the OLED Dual Stepper PCB (12 October 2026)** separated Mega pins 2 and 32. Earlier versions joined them in copper because of a copy-and-paste net name, so the fuel quantity and fuel load gauges shared coil A. On a board older than 1.4 fit only one of those two gauges or cut the trace between the pins.

**Lamp pins on the Servo PCB.** Ten of the 15 lamp outputs are on different Mega pins from the sketch (see Known issues); test each lamp on its own.

**Radio Auto Pilot knobs.** The PCB also carries the autopilot knobs, volume pots, display drivers and a fifth matrix column that the radio sketch does not read yet, and four of its encoders have A and B swapped relative to the sketch.

## 14.2 Connections on the servo carrier board

| Connector | Count | Pins | Wiring |
|---|---|---|---|
| Servo headers | 16 on the PCB, 2 used | 26 (pitch), 27 (bank) | Signal, +V, GND; +V from the servo rail |
| Lamp outputs | 15 | A1 to A15 | Pin HIGH = lamp on; through a driver for lamps above 20 mA |
| Spare outputs | 3 | not assigned | Reserved for future lights |
| Ethernet module | 1 | SPI 50 to 52, chip select, reset 53 | Reset is pulsed by the firmware at start-up |
| Status LEDs | 2 | 14 green, 15 red | With series resistors to GND |
| Servo power input | 1 | | 5 to 6 V; ground joined to Mega GND |
| Front-panel switch inputs | 3 | D43, D47, D48 | Each to GND through a switch (SW1 TOT clear, SW3 warning mute, SW2 fuel valve); internal pull-up, debounced in firmware |

The rule that matters most: **never power servos from the Mega's 5 V pin.** Servos draw current spikes when they move, which can reset the Mega and drop the Ethernet link.

## 14.3 Wiring checklist

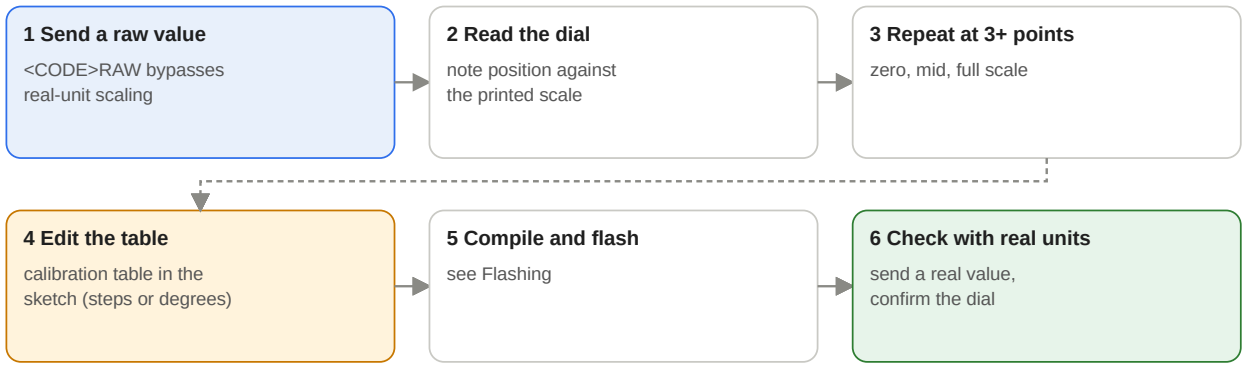
- Servo rail fed from its own 5 to 6 V supply, not the Mega.
- Supply ground joined to Mega GND (signals need a shared reference).
- Each servo plugged in signal-side matching the header silkscreen.
- Lamps checked during the power-up lamp test.
- Ethernet cable to the same switch as the primary PC.
- Board serial number noted in the sketch header.

# 15 Calibration

The servo board turns attitude degrees into a servo position (0 to 180) with two small tables in the sketch. The stepper boards do the same for steps. Between table points the boards interpolate in a straight line; beyond the ends they hold the end value.

| Instrument | Attitude | Servo position |
|---|---|---|
| Pitch (`PITCH_DEG_TABLE`) | -30 degrees | 179 |
| Pitch | 0 (level) | 113 |
| Pitch | +30 degrees | 70 |
| Bank (`BANK_DEG_TABLE`) | -90 degrees | 5 |
| Bank | 0 (level) | 93 |
| Bank | +90 degrees | 179 |

The self-test sweep uses `aServMinPosition[]` and `aServMaxPosition[]` (bank 5 to 179, pitch 166 to 70) and parks at `aServZeroPosition[]` (bank 93, pitch 113).




Typical tables: pitch -30 / 0 / +30 degrees, bank -90 / 0 / +90 degrees, one row per stepper or servo gauge.

Sending <CODE>RAW:n moves the motor to step n directly, so any dial can be calibrated without a simulator.

*Figure 47 — The calibration loop*

## 15.1 Calibrating a servo

1. Send raw positions with `BANKRAW` or `PITCHRAW` (for example `D,PITCHRAW:113`) until the instrument shows a known attitude.
2. Note the position for each attitude you check: at least both ends and level.
3. Enter them in `PITCH_DEG_TABLE` or `BANK_DEG_TABLE`, and set `aServZeroPosition[]` to the level position so the self-test parks there.
4. Upload, then check with real values, for example `D,PITCH:0` and `D,BANK:30`.

## 15.2 Calibrating a stepper gauge

1. Send raw steps with the `RAW` token (for example `D,TQRAW:300`) and read the dial.
2. Repeat at zero, mid scale and full scale (more points where the scale is not linear).
3. Edit the gauge's table in the sketch (`TQ_PCT_TABLE`, `IAS_KT_TABLE` and so on) and its `...ZeroOffset` trim if the zero is off.
4. Upload and check with real values (`D,TQ:60`).

The StepperVSITester sends every token and raw token without a simulator. Do not drive a gauge hard against its mechanical stop: a stalled servo or stepper draws high current and gets hot.

# 16 Troubleshooting

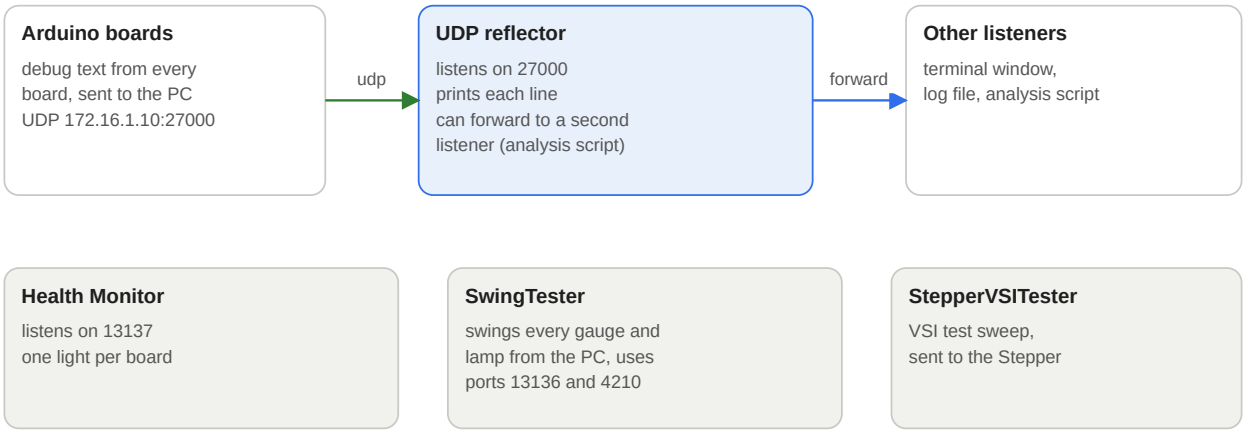



*Figure 48 — Debug and test tools*

| Symptom | Likely cause | What to check |
|---|---|---|
| Red LED keeps fast-flashing, no gauge sweep | Board stuck in Ethernet start-up | Ethernet module seated; reset line on pin 53; module powered |
| Sweep runs but gauges never move in flight | Packets not reaching UDP 13136 | PC on 172.16.1.x; Windows firewall allows outbound UDP; the bridge sends to the right boards (servo .102, steppers .105 and .106); only one bridge running |
| No debug messages, or the Health Monitor shows a board as down | Listener or Health Monitor not running, or PC address wrong | PC at 172.16.1.10; listener bound to UDP 27000, Health Monitor to UDP 13137; firewall allows inbound UDP on both |
| One gauge pinned at an end stop | Value outside the calibration table, or table points wrong | Send the `RAW` token to find the true limits, then re-measure the table (see Calibration) |
| All gauges drop to zero after 30 seconds | No packets arriving (watchdog) | Bridge running and connected; simulator not paused; network link |
| A board feels slower, or its knobs and needles stutter, while the PC (or another board) is off | Each packet sent to a missing device waits up to 80 ms for an ARP answer (the fast Ethernet timeout) | Switch the missing device on, or reduce debug traffic; see *Unreachable hosts and the fast Ethernet timeout* in the Network chapter |
| Attitude indicator lags fast changes | 1 degree per 5 ms step limit | Expected; reduce `servoCheckInterval` to speed up |
| Pressing an Audio or Auto Pilot switch is slow, or a joystick button seems delayed | The USB HID board is switched off or unplugged: each send to it waits for an answer that never comes | The radio now pauses joystick sends for 30 seconds after one fails and says so in the monitor (`USB HID board ... not answering`); connect the HID board and the buttons return |
| Servo buzzes or jitters | Weak servo supply | Separate 5 to 6 V supply for servos, common ground with the Mega |
| A backlight is off | Main bus below 20 V (`BPWR:0`), no packets for 30 seconds, or brightness set to 0 | Check the main bus voltage in the bridge window; check the bridge is running; send a brightness with the SwingTester slider |
| A backlight stays on at power-up with no PC | Old firmware without the no-data rule, or the board has never been flashed with the backlight code | Flash the current Radio, Servo, Stepper and Dual Stepper sketches; the backlight should go out about 30 seconds after power-up |
| The backlight is always on or flickers | MOSFET pinned wrongly on a board from an earlier revision (source and drain | Check the transistor against the revision notes (PCBs and wiring) |

| Symptom | Likely cause | What to check |
|---|---|---|
| | swapped) | |
| The GPS shows Brisbane Airport | No aircraft data for 30 seconds | Check a bridge or the bench tester is running on the same PC and sending to UDP 27002; check no other program owns the port |
| The GPS shows the PC time, not the simulator time | The packets carry no `UTC:` (for example the FSUIPC time read failed) | The window says *PC clock*; check the bridge version |
| The GPS interface cannot open its port | Wrong COM port, or another program holds it | Use *Change serial port...*; close other serial programs |
| A lamp never lights | Wiring or the lamp's driver | It should light at the power-up lamp test; if not, check the A pin and its driver |
| A radio knob tunes the wrong radio | Bare frequency sent instead of a labelled one | Use the real board (it labels COM2, NAV1 and NAV2); the Radio emulator sends bare numbers, so only COM1 tunes |
| COM and NAV displays blank | Avionics bus below 24 V, or no data and no knob use for 30 seconds | Switch avionics on; turn a knob to wake them |
| Dual stepper OLEDs blank | Main bus below 24 V (`PWR:0`) | Switch the battery on; check the main bus voltage in the bridge window |
| Radio swap switches do nothing | Column 0 of the matrix stuck | Send `C,MATRIX:1` and read the debug listener (see Radio controller) |
| FSUIPC binding fires the wrong event | EVT re-indexed after an update | Re-map the binding in FSUIPC |
| Joystick buttons vanish after a reset | USB HID board rebooted | Windows re-detects them within seconds; re-check games that had them open |
| Upload fails with *Access is denied* | Another program holds the COM port | Close the serial monitor and the Arduino IDE port; check the port with `arduino-cli board list` |
| Upload goes to the wrong board | COM port moved | Re-check Device Manager; see COM ports and flashing |
| Data reaches the boards only in bursts, with gaps of several seconds | The PC's Ethernet link to the board switch is dropping in and out | In Windows, check the network adapter stays connected; check the 5 V supply of the network switch stays above 4.75 V (it once dipped below and the switch dropped its ports), then the cable and switch port |

## 16.1 Known issues in the code

**Servo firmware (`JET_RANGER_SERVO_CONTROLLER.ino`)**

- `powerAvailable` is set from `NAVCOM1` but nothing uses it, so the attitude indicator stays live when the simulator's power is off (the bridge sends all lamps off below 20 V).
- Several lamps log *Transmission Temperature changed* to the debug listener, whichever lamp changed.
- The PCB labels 10 of the 15 lamp outputs on different pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). Test each lamp on its own, for example `D,RLOW:1`, and correct whichever side is wrong.

**FSUIPC event file (`DODSIM206V2.EVT`)**

- The line before event 133 is missing its `//`, so FSUIPC may reject or mis-read it.
- Events 189 and 190 are named the wrong way round: 189 is called `_SET` but toggles, 190 is called `_TOGGLE` but takes 1 or 2.
- Events 69 to 71 mix up the instrument and cockpit light breakers: 69 is named `COCKPIT_LT_CB_TOGGLE`, and `INST_LT_CB_SET` appears twice.

## 16.2 Open questions

- Power supply rating for the servo rail and the 12 V lamp supply.
- Whether the estimates for the FlyInside and CowaSim aircraft hold at other power settings.

# 17 Reference: UDP tokens

From the current sketches. Gauge tokens take real units; lamp tokens take 0 (off) or 1 (on). `UDP_Interface_Values.xlsx` still lists the old 16-servo pin plan, so use these tables instead.

## 17.1 Gauge tokens

| Token | Gauge | Board | Units |
|---|---|---|---|
| `BANK` | Attitude bank | Servo (.102) | degrees, plus and minus 90 |
| `PITCH` | Attitude pitch | Servo (.102) | degrees, plus and minus 30 |
| `IAS` | Airspeed | Stepper (.105) | knots, one decimal allowed |
| `VSI` | Vertical speed | Stepper (.105) | ft/min |
| `AGL` | Radar altitude | Stepper (.105) | feet |
| `TQ` | Engine torque | Stepper (.105) | % |
| `N1` | Gas producer | Stepper (.105) | % |
| `ITT` | Turbine out temperature | Stepper (.105) or Dual (.106) | degrees C |
| `FUEL` | Fuel quantity | Stepper (.105) or Dual (.106) | US gal |
| `OILT` | Engine oil temperature | Dual (.106) or Stepper | degrees C |
| `OILP` | Engine oil pressure | Dual (.106) or Stepper | psi |
| `XMSNT` | Transmission oil temperature | Dual (.106) or Stepper | degrees C |
| `XMSNP` | Transmission oil pressure | Dual (.106) or Stepper | psi |
| `RPME` | Engine RPM (N2) | Dual (.106) | %, one decimal |
| `RPMR` | Rotor RPM | Dual (.106) | %, one decimal |
| `ALT` | Altimeter | Dual (.106) | feet |
| `FUELLOAD` | Fuel load | Dual (.106) | psi |
| `ELECTRICALLOAD` | Electrical load | Dual (.106) | % |

Every gauge token also has a `RAW` twin (for example `PITCHRAW`, `IASRAW`) that takes a raw servo position or stepper step count and skips the calibration table.

## 17.2 Lamp tokens (servo board)

| Token | Lamp | Pin |
|---|---|---|
| `EOUT` | Engine out | A1 |
| `RLOW` | Rotor RPM low | A2 |
| `TOPW` | Transmission oil pressure | A3 |
| `TOTW` | Transmission oil temperature | A4 |
| `BHOT` | Battery hot | A5 |
| `BTMP` | Battery temperature | A6 |
| `EC` | Engine chip | A7 |
| `TRC` | Tail rotor chip | A8 |
| `TC` | Transmission chip | A9 |
| `BD` | Baggage door | A10 |
| `FFLTR` | Aft fuel filter | A12 |
| `FPMP` | Fuel pump | A11 |
| `GENF` | Generator fail | A13 |
| `LOWF` | Low fuel | A14 |
| `SCF` | SC fail (standby compass) | A15 |

## 17.3 Aircraft information to the GPS interface (UDP 27002)

`D,LAT:<deg>,LON:<deg>,ALT:<feet>,IAS:<knots>,HDG:<degrees true>,UTC:<yyyyMMddHHmmss[.fff]>`, sent every 500 ms by a bridge or the bench tester. Any subset of keys works, and the CSV form `lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc]` is accepted too. South and west are negative.

## 17.4 Status and control tokens

| Token | Packet | Meaning |
|---|---|---|
| NAVCOM1 | D | 0 = avionics bus below 24 V (radio COM and NAV displays blank, panel LEDs off), 1 = on |
| LED_CRS, LED_HDG, LED_IAS, LED_ALT, LED_APPR, LED_FD, LED_BC, LED_APENG | D | Auto Pilot panel LEDs on the radio controller (1 = on) |
| LED_COM1, LED_BOTH, LED_NAV2, LED_DME, LED_OUTER, LED_INNER, LED_COM2, LED_NAV1, LED_MKR, LED_ADF, LED_MID | D | Audio panel LEDs on the radio controller (1 = on) |
| APHDG, APCRS, APIAS, APVS, APALT | D | The values on the Auto Pilot panel's OLED (radio controller): heading and course as three digits (north is 360), airspeed in knots, vertical speed in feet per minute and altitude in feet |
| LDGN, LDGL, LDGR | D | The landing gear lamps on the ADF panel (nose, left, right): 1 = lit |
| BPWR | D | Backlight power for the Radio, Servo, Stepper, Dual Stepper and ADF and Upper controllers: 1 = main bus 20 V or more, 0 = backlight off. Sent first in the Servo and Stepper packets, after PWR in the Dual Stepper packet, after NAVCOM1 in the Radio packet and on its own to the ADF and Upper board |
| PWR | D | Dual stepper board: 1 = all four OLEDs on, 0 = blank them (main bus below 24 V). Sent first in the packet |
| ZULU | D | Zulu time as HHMM, for example 1430 (dual stepper clock OLED) |
| TEMP | D | Temperature text for the dual stepper's temperature OLED, for example 25C |
| B | C | Backlight brightness 0 to 15 (0 off, 15 full). Radio, Servo, Stepper, Dual Stepper and ADF and Upper (the Radio also sets the brightness of its Auto Pilot, Audio and Huey panel LEDs from it). The ADF controller can send it from its brightness pot, which is switched off for the moment |
| SWING<CODE> | C | Runs one gauge's start-up sweep on demand, for example C,SWINGTQ:1 (stepper boards) or C,SWINGPITCH:1 (servo board); the value is ignored |
| OLEDTEST | C | 1 fills the four OLEDs solid white, 0 restores them (dual stepper board) |
| KEEPALIVE | C | Ignored command that still counts as received data and restarts the 30 second no-data timer (SwingTester sends it during a run) |
| RESET | C | Reboots the board about 2 seconds later (servo, stepper, dual stepper, radio and ADF and Upper controllers on port 13136; USB HID board on port 4210). The radio and ADF and Upper controllers use a software reset because a watchdog reset hung the radio's bootloader; the upper controller's reset has not been bench-tested |
| MATRIX | C | Radio and ADF controllers: print the raw button matrix to the debug listener |
| POWER | C | ADF controller: send a power status (12 V and 5 V rails) now |
| LEDTEST | C | Radio controller: 1 lights all 19 Auto Pilot and Audio panel LEDs, 0 returns them to the simulator's state |

# 18 PCB schematics and layouts

The schematic (the top sheet of each project) and the board layout of every PCB project in `Jet Ranger/PCBs/`, exported from KiCad 9. The layout shows the front and back copper (red and blue), the silkscreen (yellow) and the board outline, viewed from the front. The images are bitmaps: zoom in on the PDF or the document to read the labels, or open the KiCad project for the full detail. The schematic and layout images are regenerated from the KiCad files, so they show the boards as they were on the day the document was built, including any changes not yet committed.

## 18.1 Jet Ranger Radio Auto Pilot

Revision 1.3 (change log; the title block shows 1.2, 2026-10-17). Mega shield for the Radio controller (172.16.1.101). Carries the USR-ES1 Ethernet module, the TCA9548A I2C multiplexer for the OLEDs, the MAX7219 that lights the 19 Auto Pilot and Audio panel LEDs, the backlight MOSFET (D5), the COM and NAV panel connector (`COMM_NAV`), the `AUTOPILOT` and `AUDIO` panel connectors, five matrix columns and the Auto Pilot knob inputs.

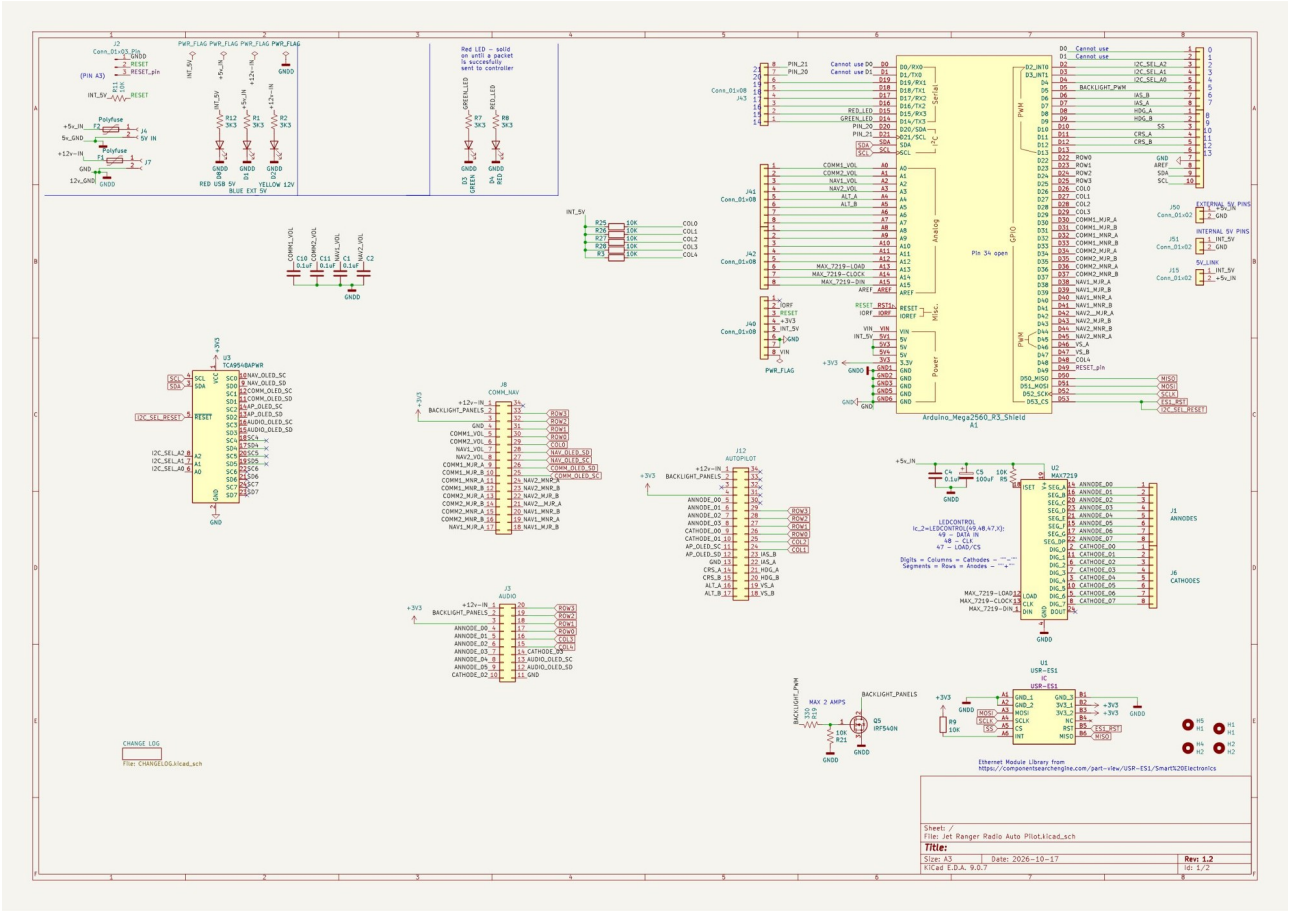


*Figure 49 — Jet Ranger Radio Auto Pilot schematic*

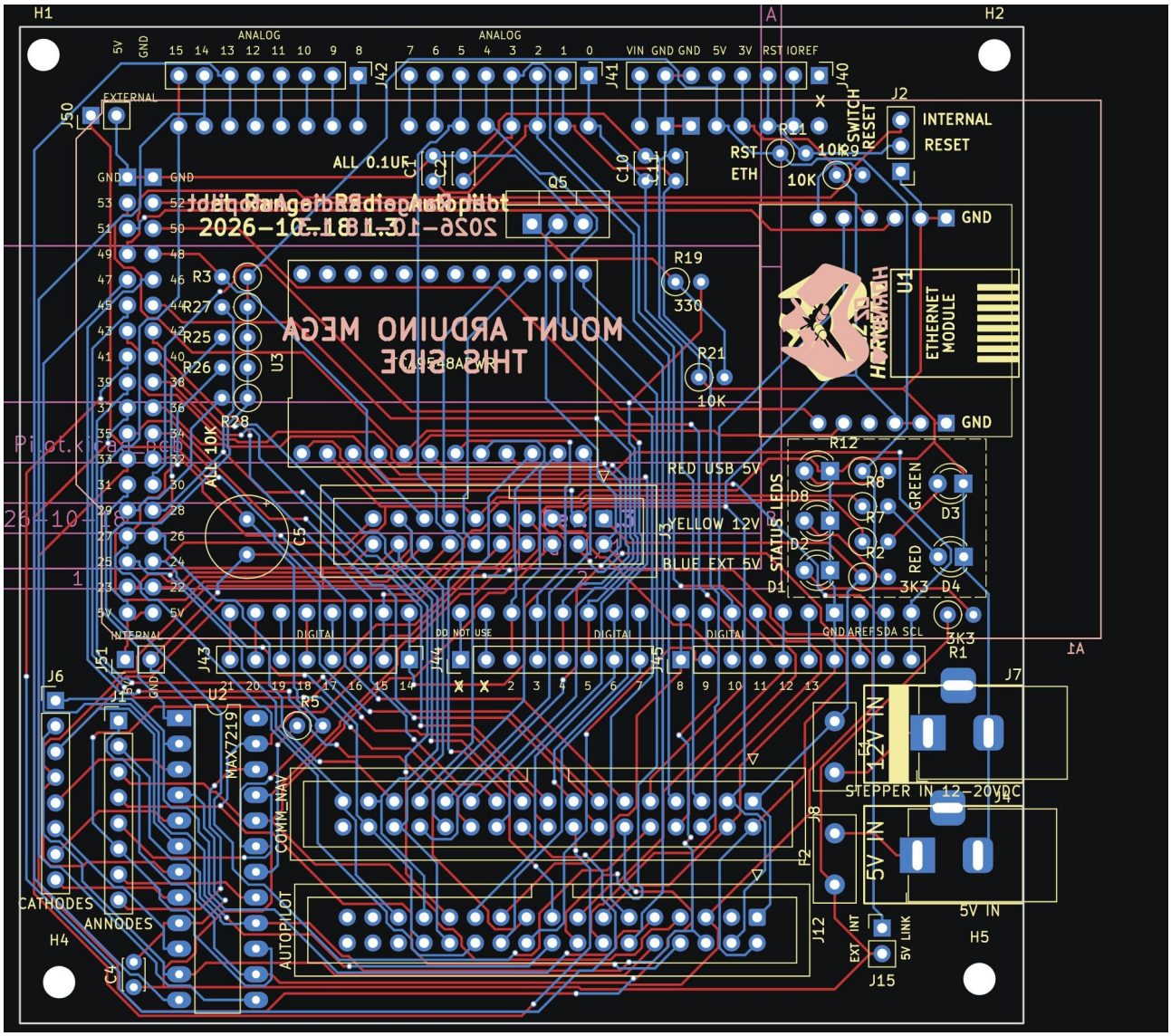



*Figure 50 — Jet Ranger Radio Auto Pilot board layout*

## 18.2 Jet Ranger COM and NAV

No revision set (title block dated 2024-08-10). Radio panel board behind the COM and NAV faceplate. Eight rotary encoders (a major and a minor knob for COM1, COM2, NAV1 and NAV2), the four frequency-swap switches in a diode matrix on column 0, four volume pots, and sockets for the two character OLEDs.

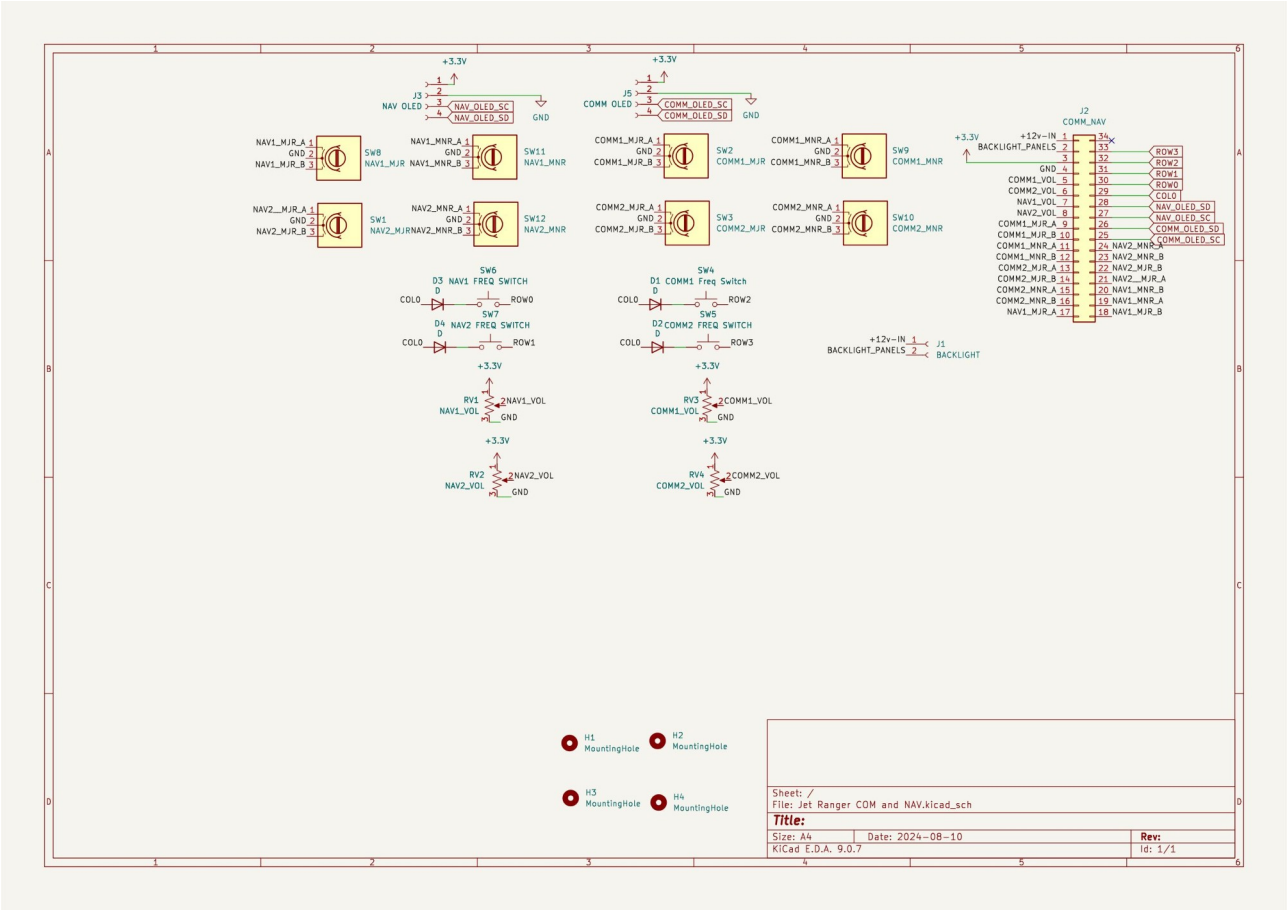



*Figure 51 — Jet Ranger COM and NAV schematic*

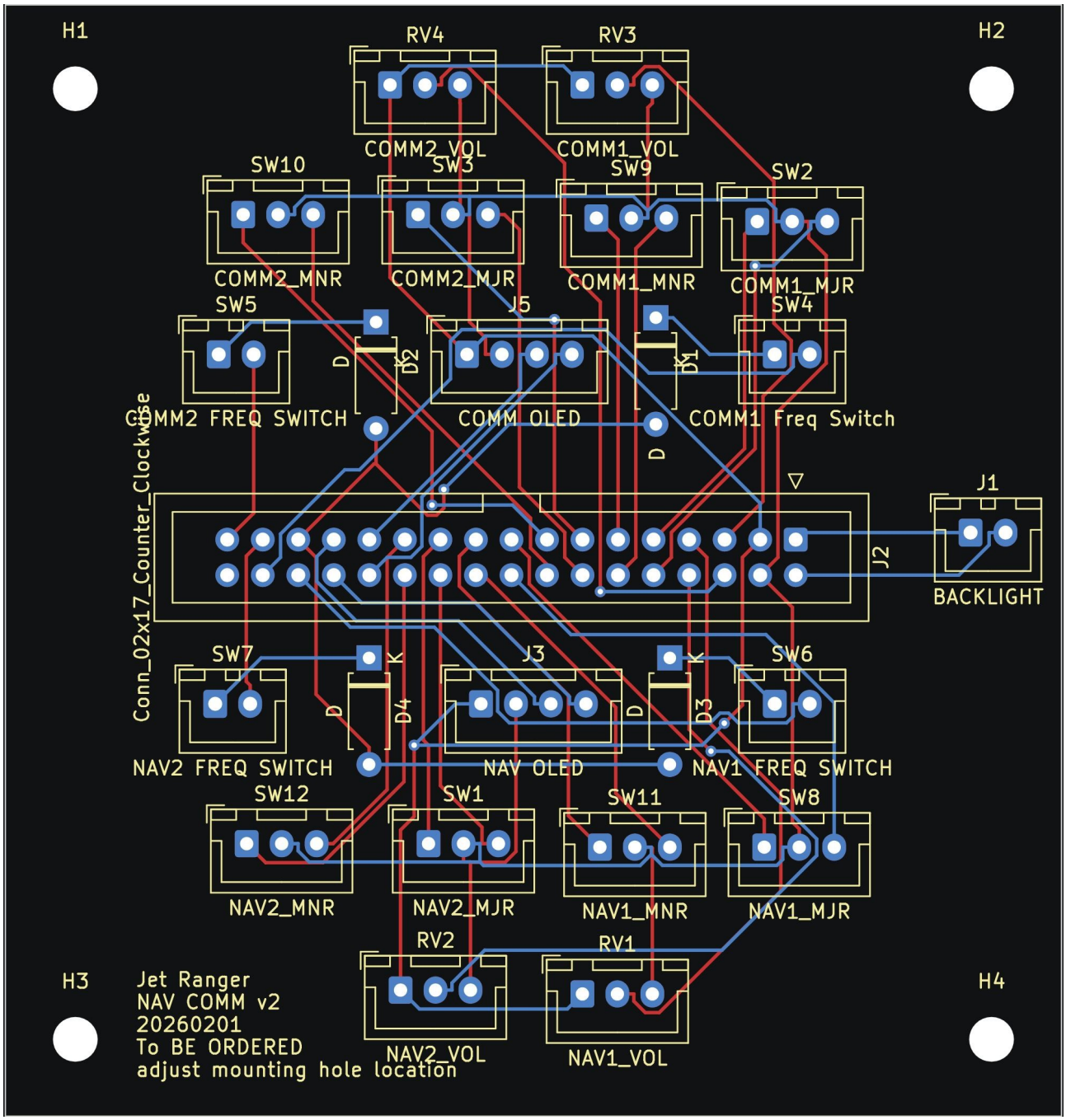



*Figure 52 — Jet Ranger COM and NAV board layout*

## 18.3 Jet Ranger Front Panel Servo

Revision 1.4 (2026-10-18). Mega shield for the Servo controller (172.16.1.102). ULN2803A lamp drivers for the 15 warning lamps, headers for the pitch and bank servos, three front-panel switch inputs, the USR-ES1 Ethernet module and the backlight MOSFET. Version 1.4 corrected the MOSFET pin-out.

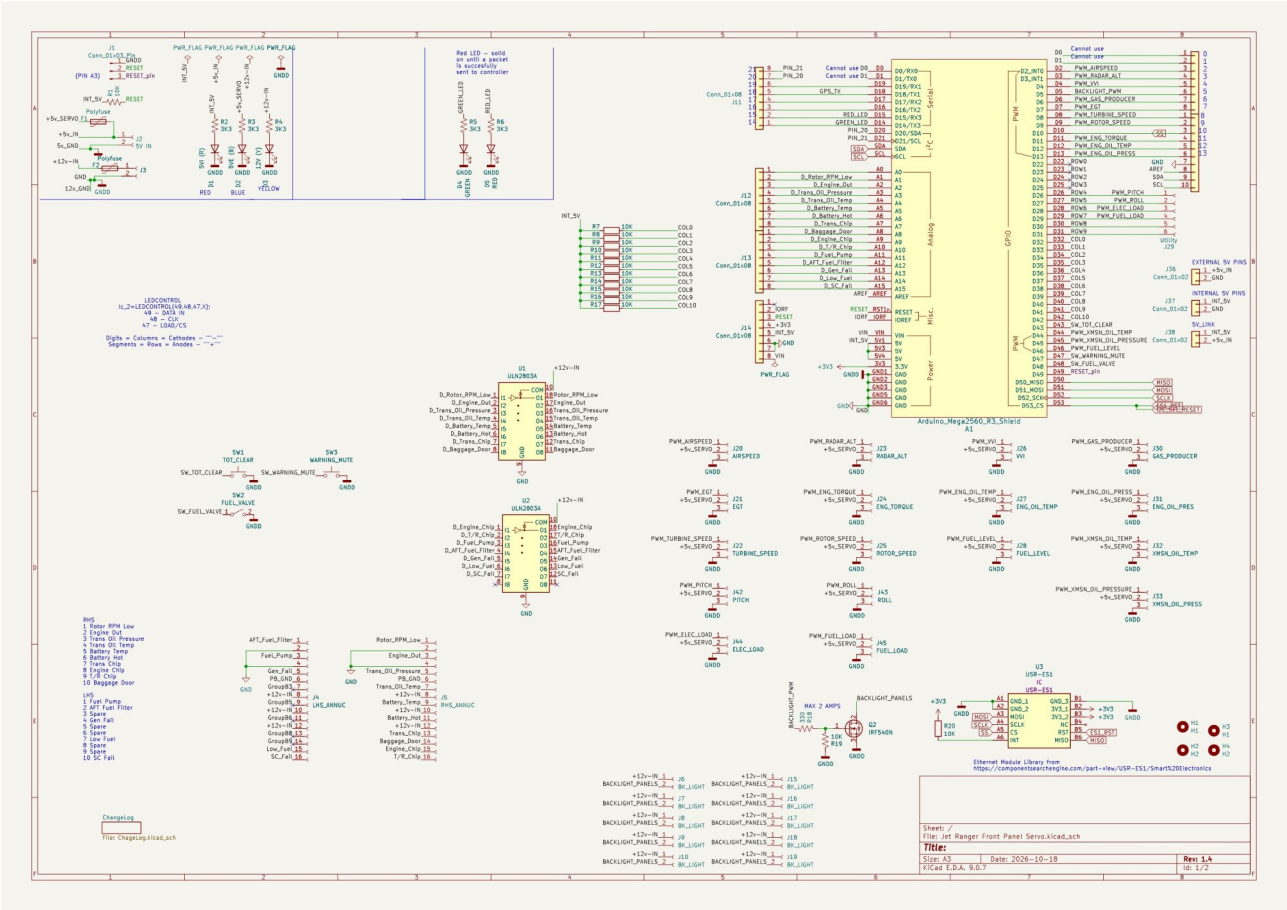


*Figure 53 — Jet Ranger Front Panel Servo schematic*

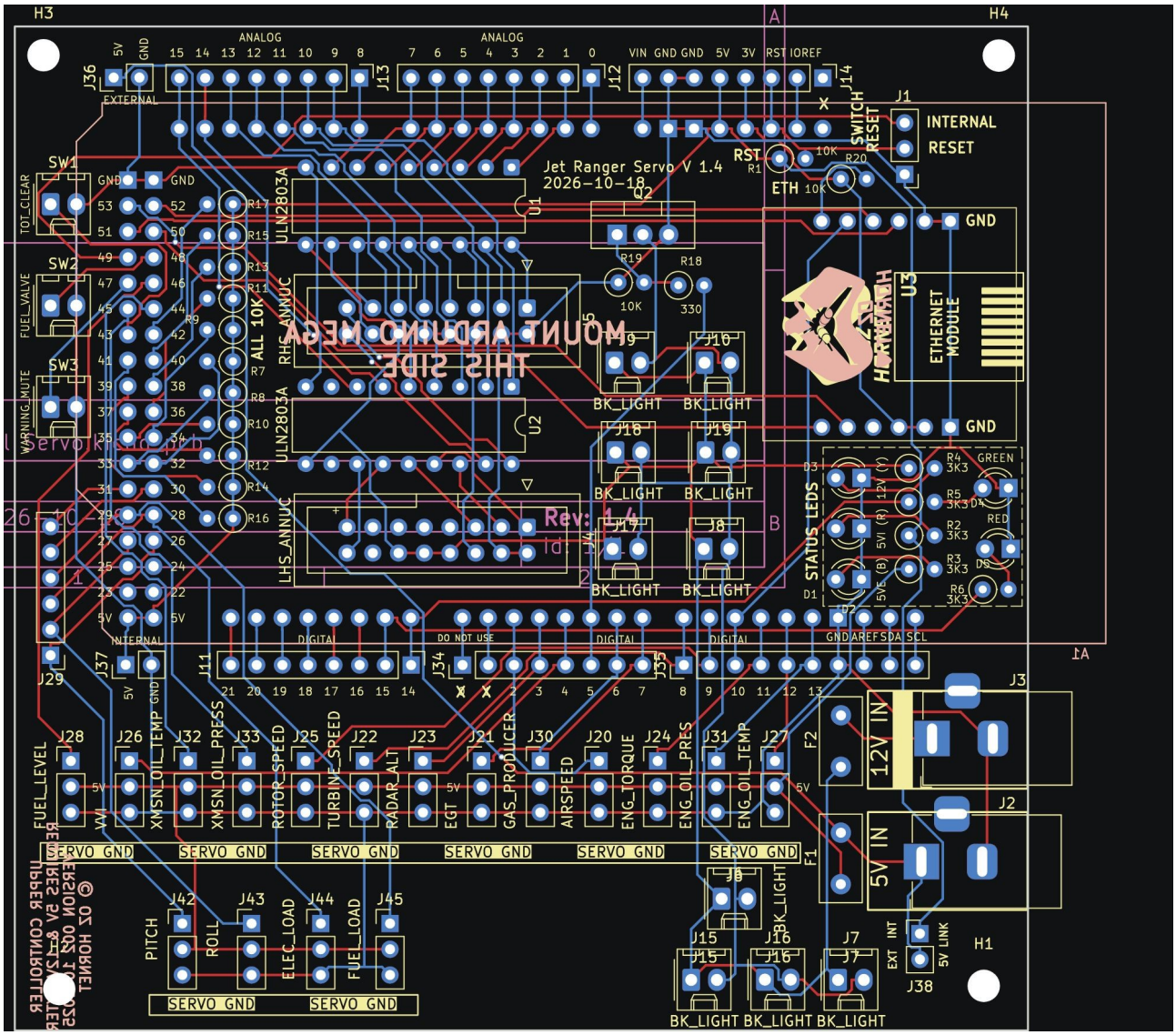



*Figure 54 — Jet Ranger Front Panel Servo board layout*

## 18.4 Jet Ranger 13 Stepper

Title block revision 1.6 (2026-10-18); the change log lists changes up to 1.5. Mega shield for the Stepper controller (172.16.1.105). Fourteen six-pin gauge sockets for the X27 steppers, a 12 V lamp supply with polyfuses, the USR-ES1 Ethernet module and the backlight MOSFET. 1.4 moved the LEDs away from the USB connectors and 1.5 selected the IRF540N.

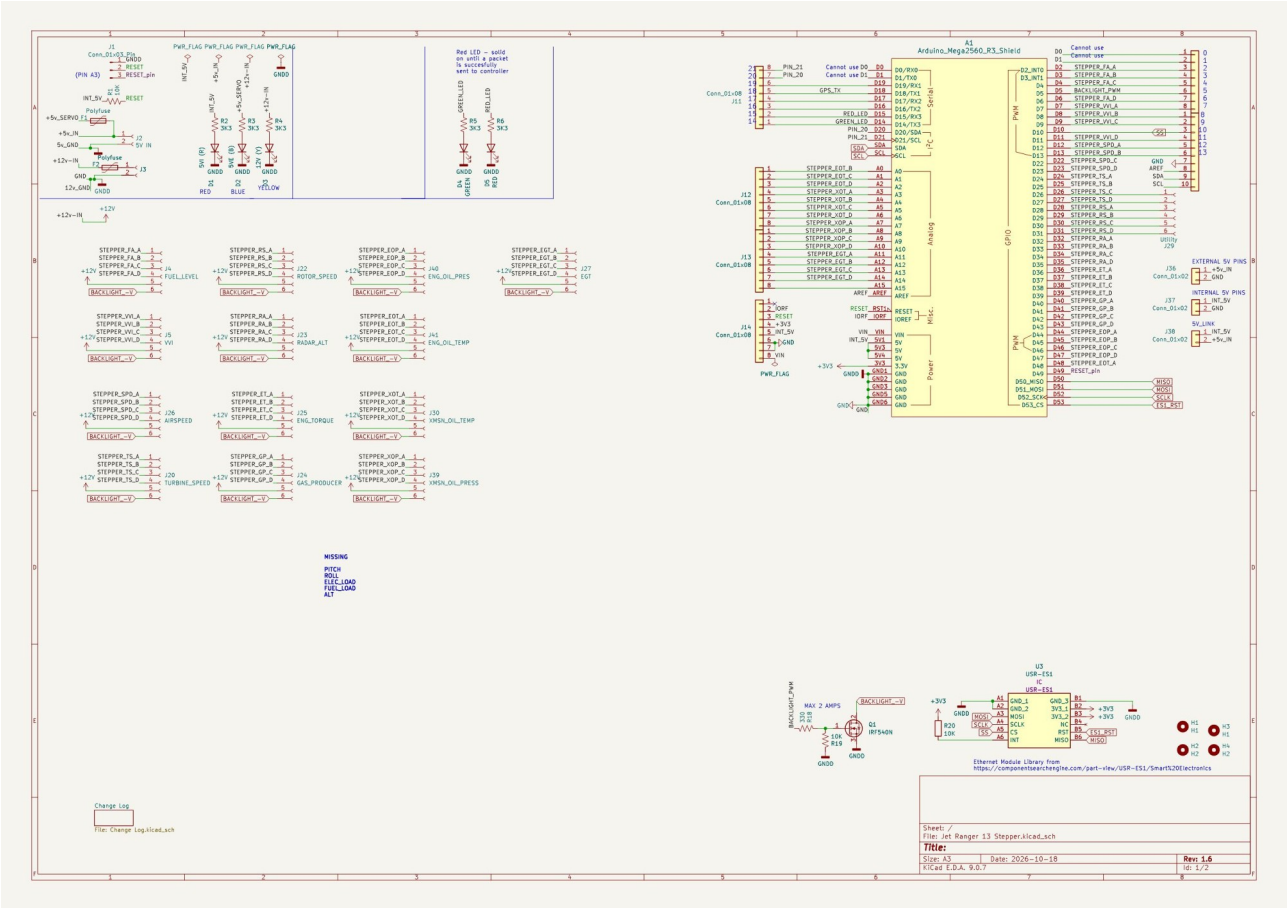



*Figure 55 — Jet Ranger 13 Stepper schematic*

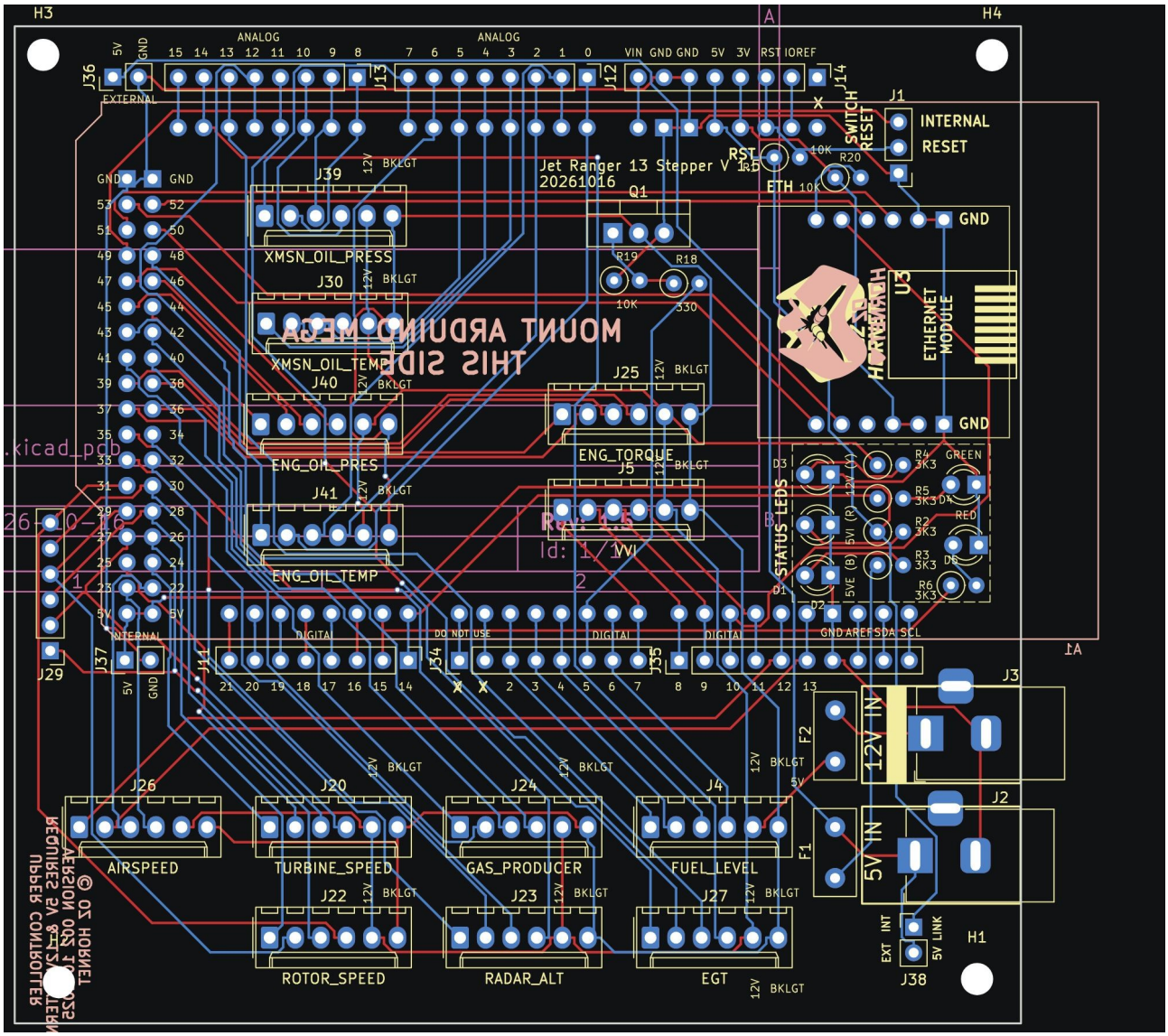



*Figure 56 — Jet Ranger 13 Stepper board layout*

## 18.5 Jet Ranger OLED Dual Stepper

Revision 1.5 (2026-10-18). Mega shield for the Dual Stepper controller (172.16.1.106). Six-pin gauge sockets for the oil, load, RPM and altimeter gauges, TCA9548A sockets for the four OLEDs (barometer, altimeter, clock, temperature), the USR-ES1 Ethernet module and the backlight MOSFET. 1.3 swapped power and ground on the four-pin OLED connections, 1.4 removed the short between Mega pins 2 and 32 and 1.5 corrected the MOSFET.

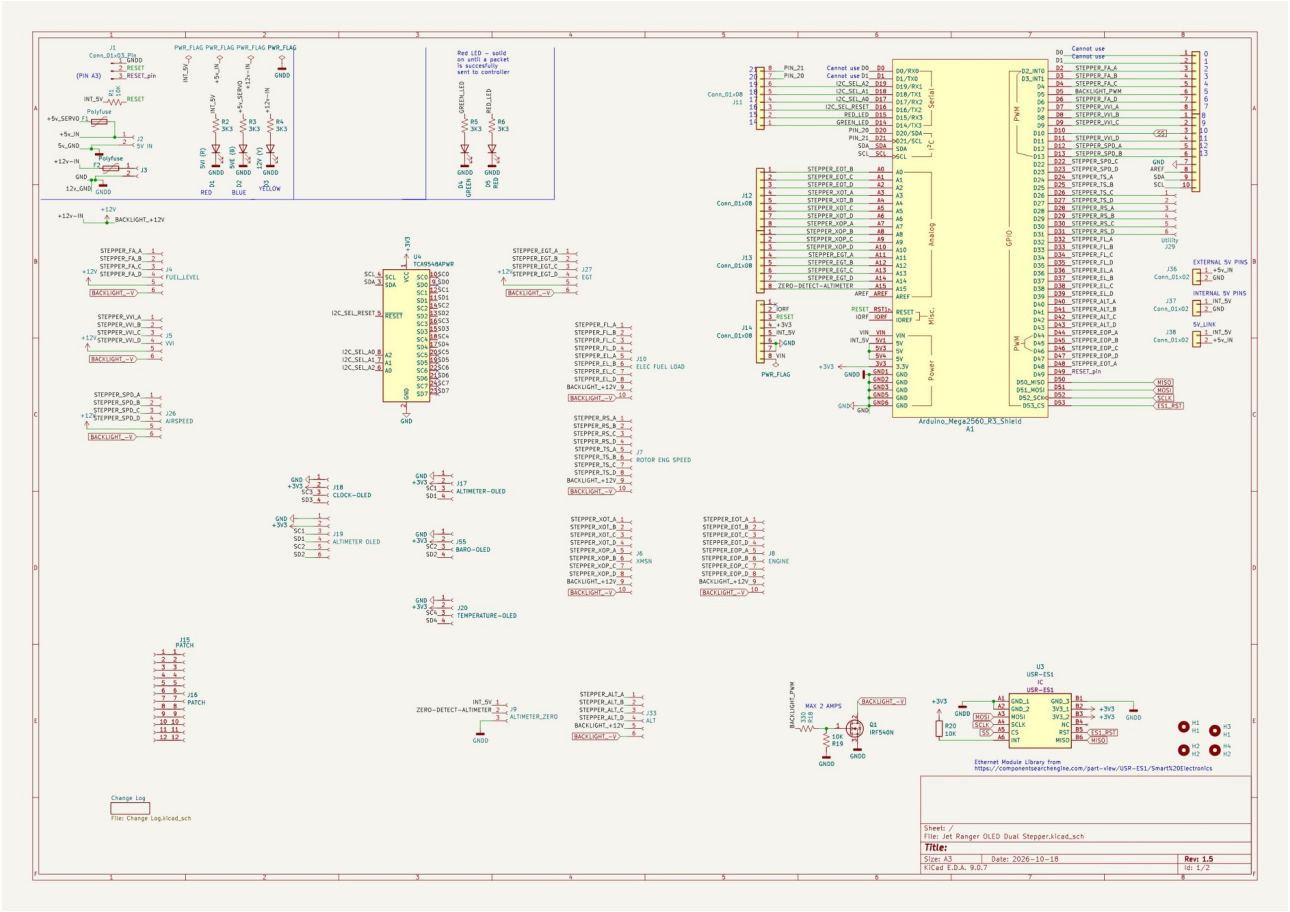


*Figure 57 — Jet Ranger OLED Dual Stepper schematic*

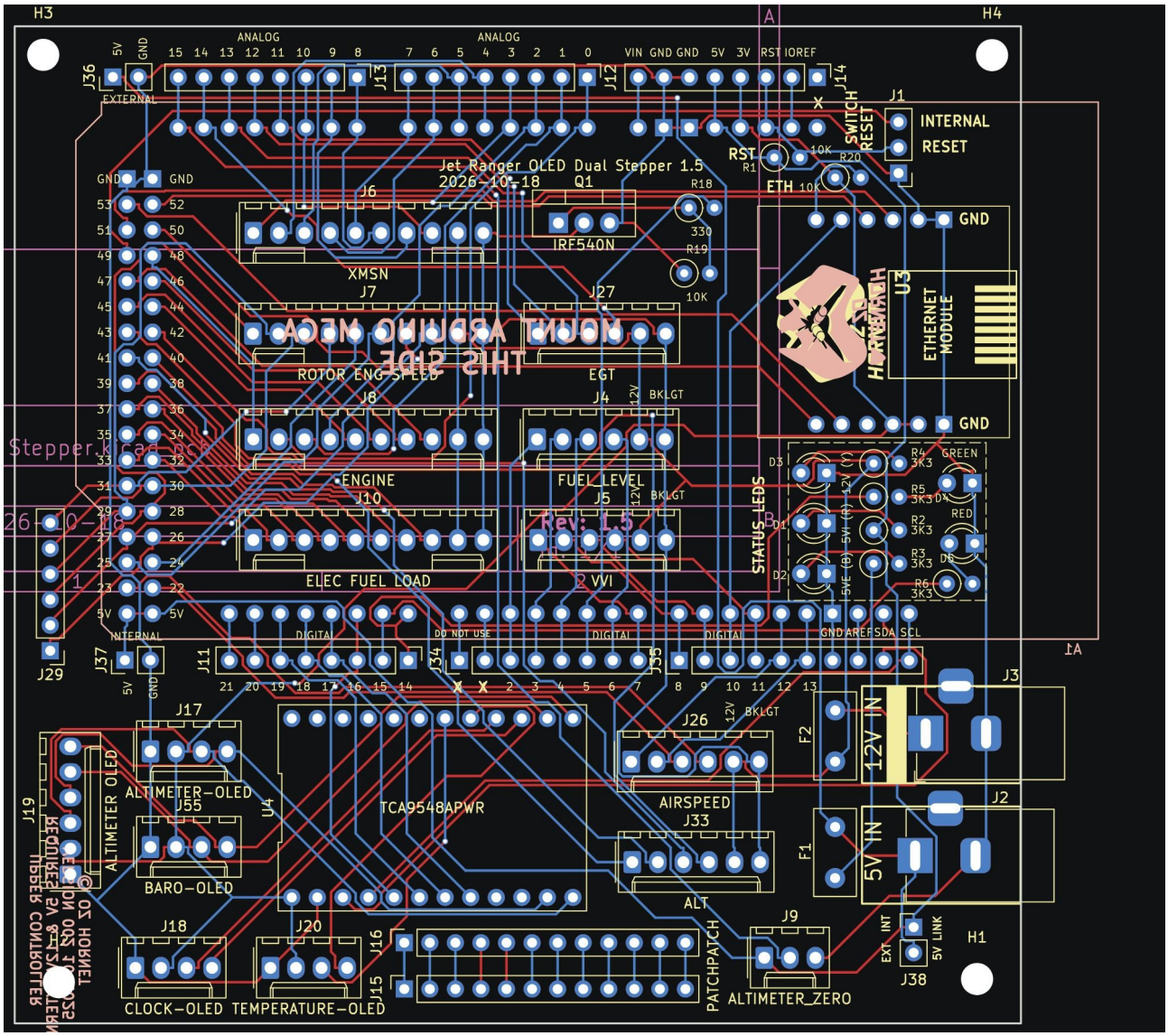



*Figure 58 — Jet Ranger OLED Dual Stepper board layout*

## 18.6 Jet Ranger X27 Small Stepper

Title block revision 2 (2022-08-04). One board for each stepper gauge. One X27 or VID29 stepper motor, two backlight LEDs with a 470 ohm resistor (the schematic value, see the backlight resistor table) and the six-pin connector to the controller.

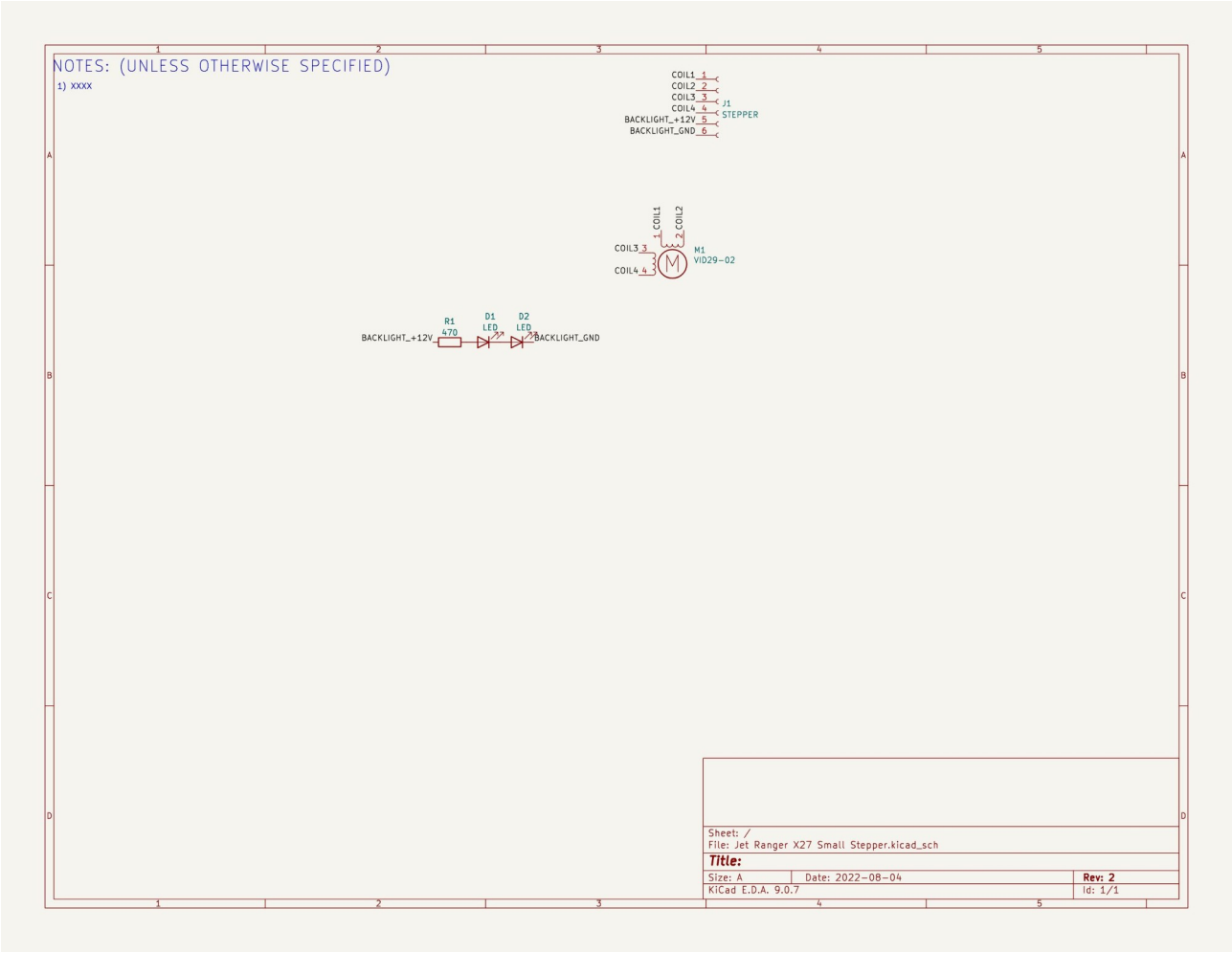



*Figure 59 — Jet Ranger X27 Small Stepper schematic*

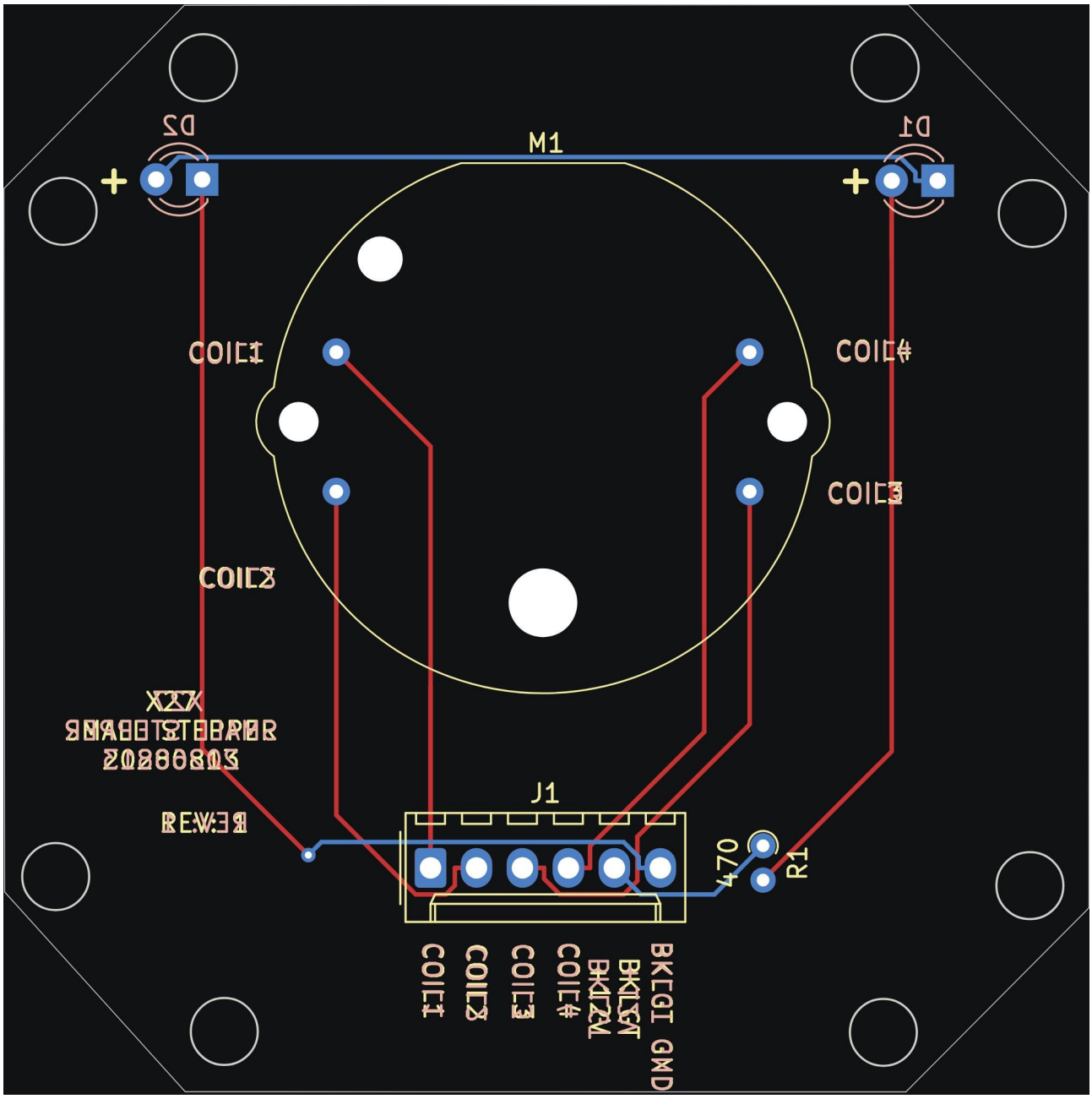



*Figure 60 — Jet Ranger X27 Small Stepper board layout*

## 18.7 Jet Ranger Upper ADF Controller

Revision 1.0 (2026-10-26). The combined ADF and Upper controller (172.16.1.107), the reference PCB for the ADF controller. It merges the Jet Ranger Upper Controller (1.4) and the Jet Ranger ADF Controller (1.6) onto one Mega, started from the ADF Controller. The ADF major and minor knobs (D6 to D9), the ADF keypad (A1 to A7, connector J41), the landing gear, flaps, speed brake and brake inputs (D31 to D34), the hyd boost, anti-ice and caution dim inputs (D35 to D37, J20), three landing gear lamps (D2 to D4), the I2C OLED, the Upper switch matrix (rows D22 to D30 on J30, columns D38 to D45 on J31 with 10 k pull-ups), the Upper keypad (A9 to A15, J32), the Warning Mute (J17) and TOT Test (J18) switches, the backlight MOSFET (D5, panel backlight on J33), a panel brightness pot on A0 and a 12 V monitor on A8 (47 k and 10 k divider). The 1x16 breakout headers J9 to J14 were removed to make room. The layout is routed (no unconnected items in KiCad's design rule check).

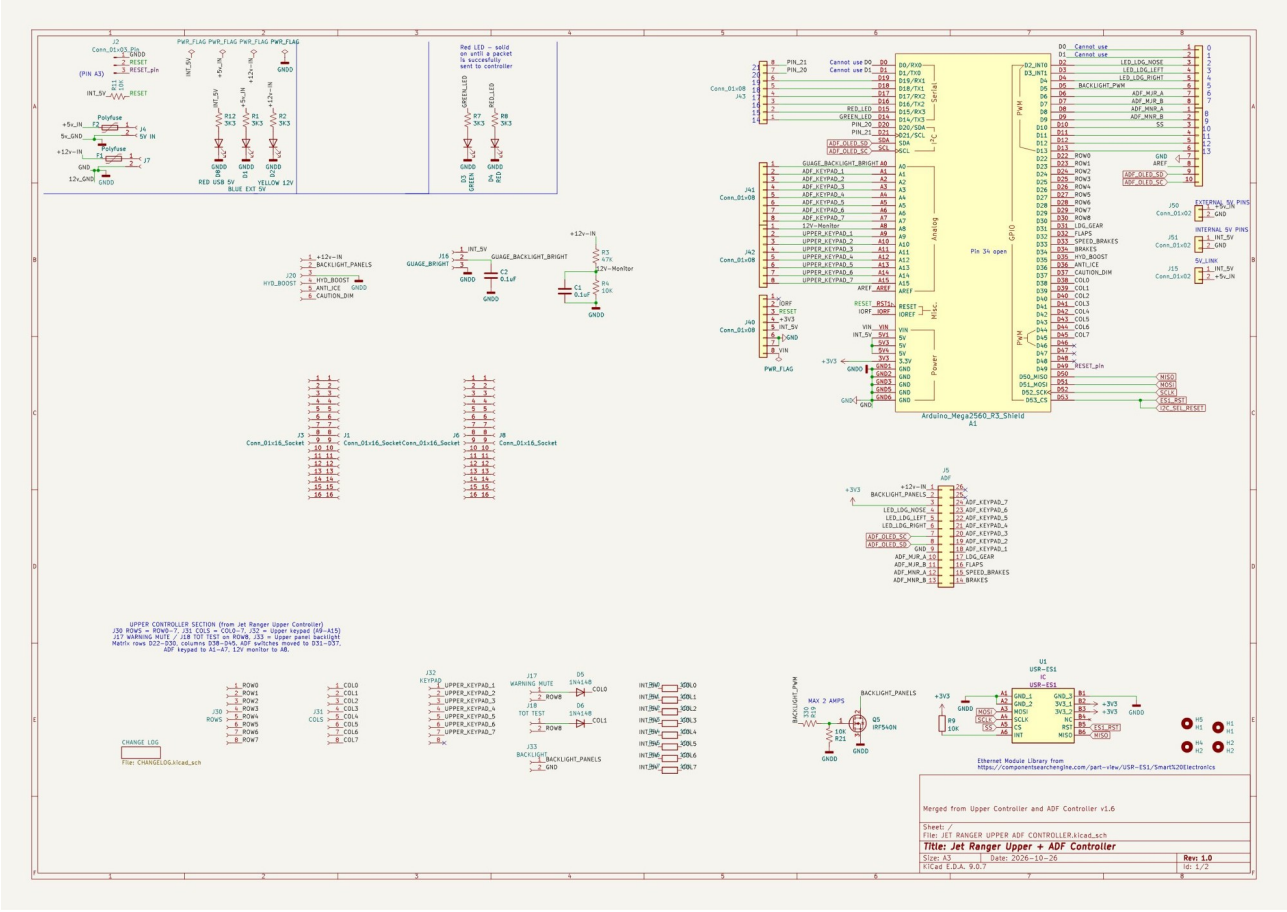



*Figure 61 — Jet Ranger Upper ADF Controller schematic*

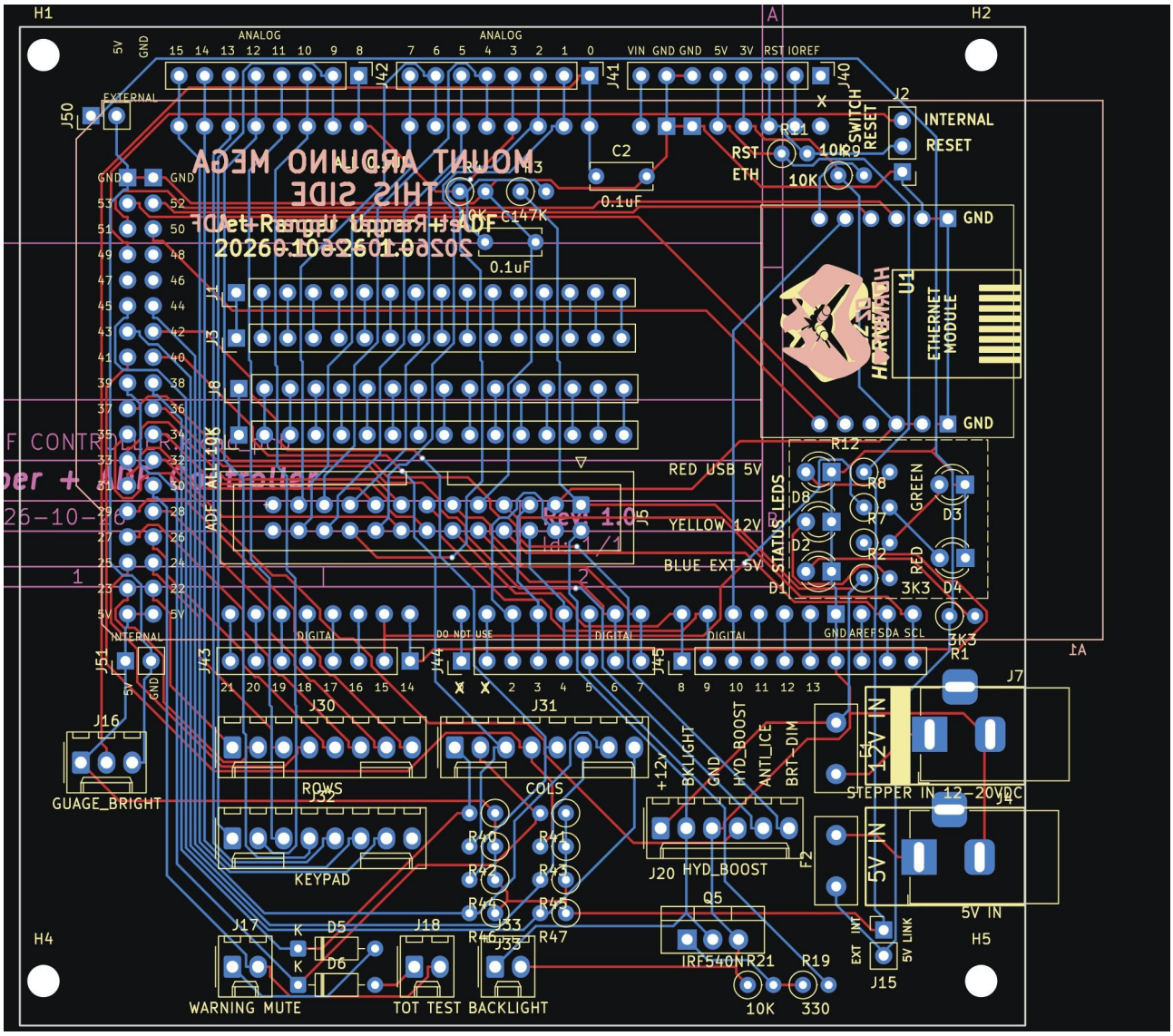



*Figure 62 — Jet Ranger Upper ADF Controller board layout*

## 18.8 Jet Ranger Audio Panel

Revision 1.1 (2026-10-17). Switch panel for the Radio Auto Pilot controller's `AUDIO` connector. Eight switches in a diode matrix on columns 3 and 4 (COM1, BOTH, NAV2, DME, COM2, NAV1, MKR, ADF), eleven LEDs on the MAX7219 digits 2 and 3, a connector for the Audio panel OLED and the backlight connector.

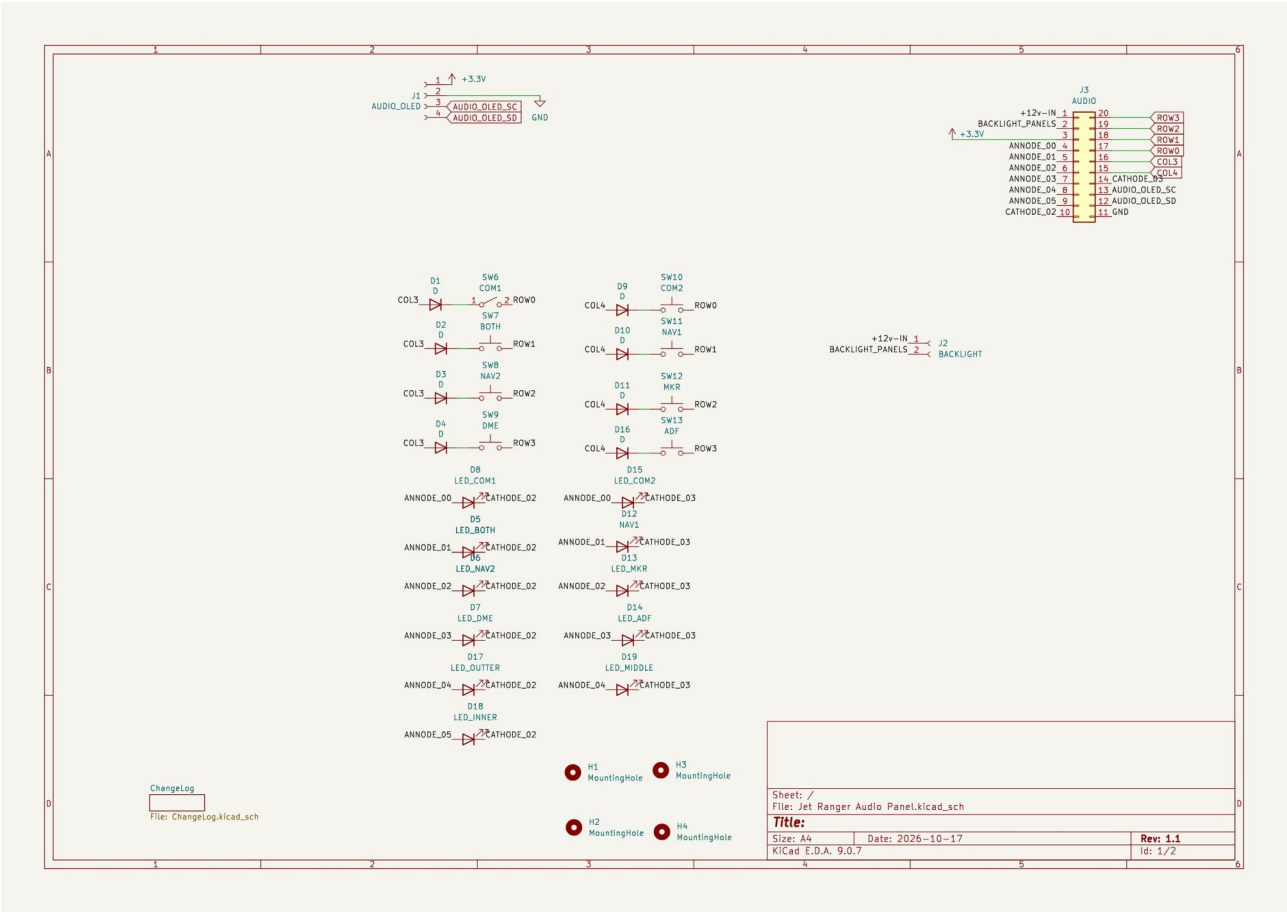



*Figure 63 — Jet Ranger Audio Panel schematic*

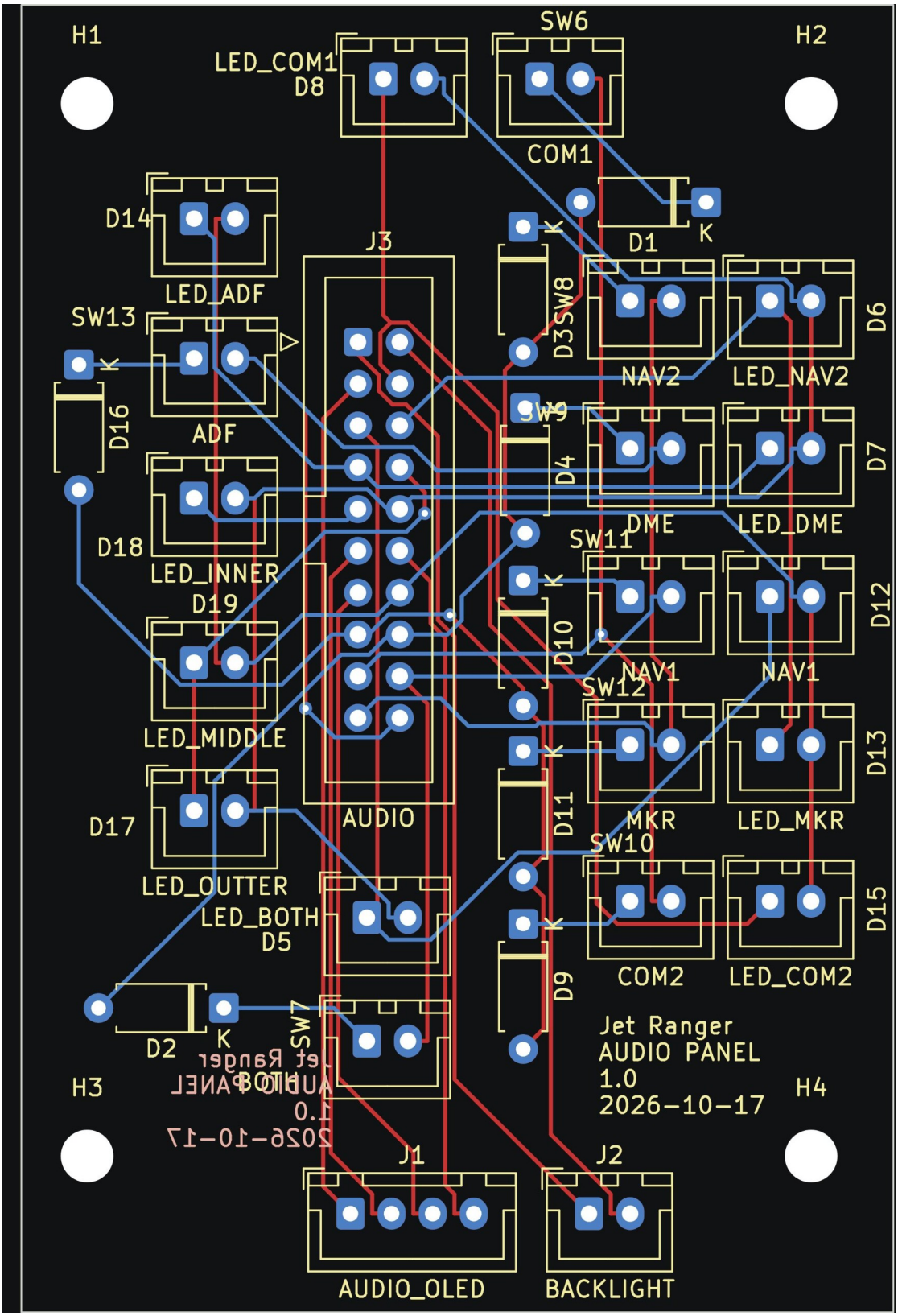



*Figure 64 — Jet Ranger Audio Panel board layout*

## 18.9 Jet Ranger Auto Pilot Panel

Revision 1.1 (2026-10-17). Switch panel for the Radio Auto Pilot controller's `AUTOPILOT` connector. Eight switches in a diode matrix on columns 1 and 2 (AP ENGAGE, HDG, CRS, ALT, APPR, FD, BC, IAS), five rotary encoders (CRS, ALT, VS, IAS, HDG), eight LEDs on the MAX7219 digits 0 and 1, the AP OLED connector, the autopilot magnet coil connection and the backlight connector.

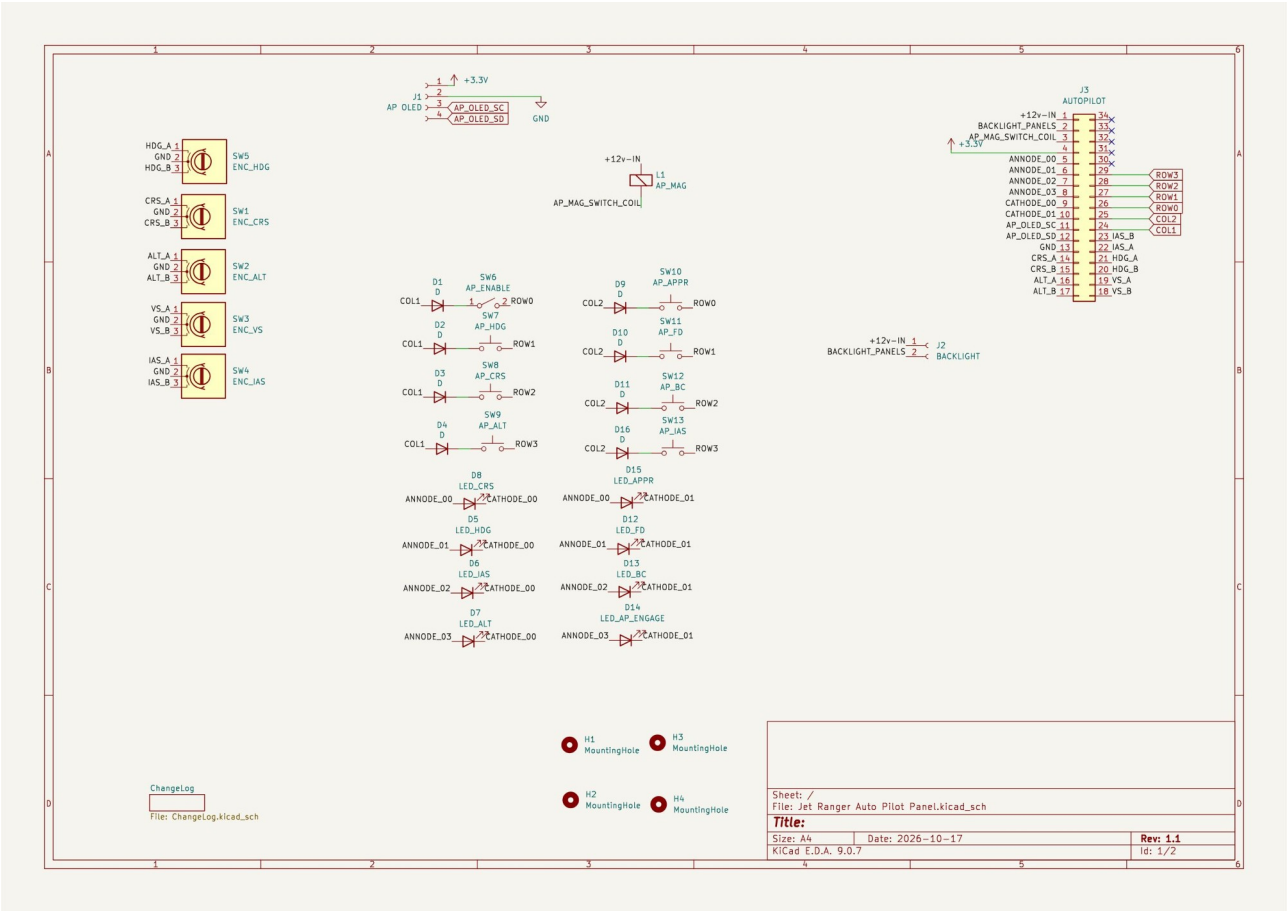



*Figure 65 — Jet Ranger Auto Pilot Panel schematic*

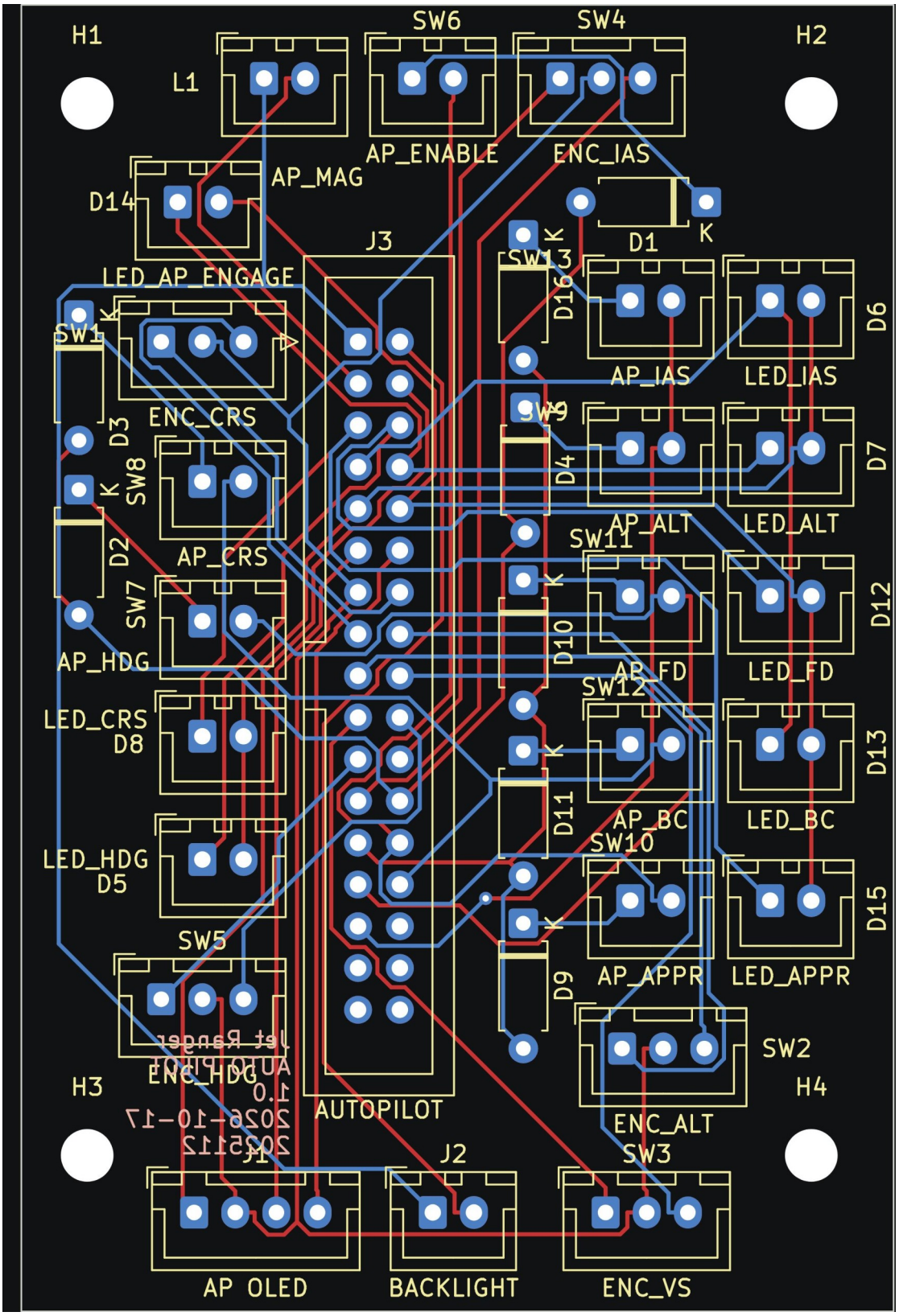



*Figure 66 — Jet Ranger Auto Pilot Panel board layout*

## 18.10 Jet Ranger ADF Panel

Revision 1.1 (2026-10-17). Panel for the ADF controller's ADF connector. The ADF major and minor encoders, the landing gear, flaps, speed brake and brakes switches, three landing gear lamps with 330 ohm resistors, the keypad connector, the ADF OLED connector and the backlight connector.

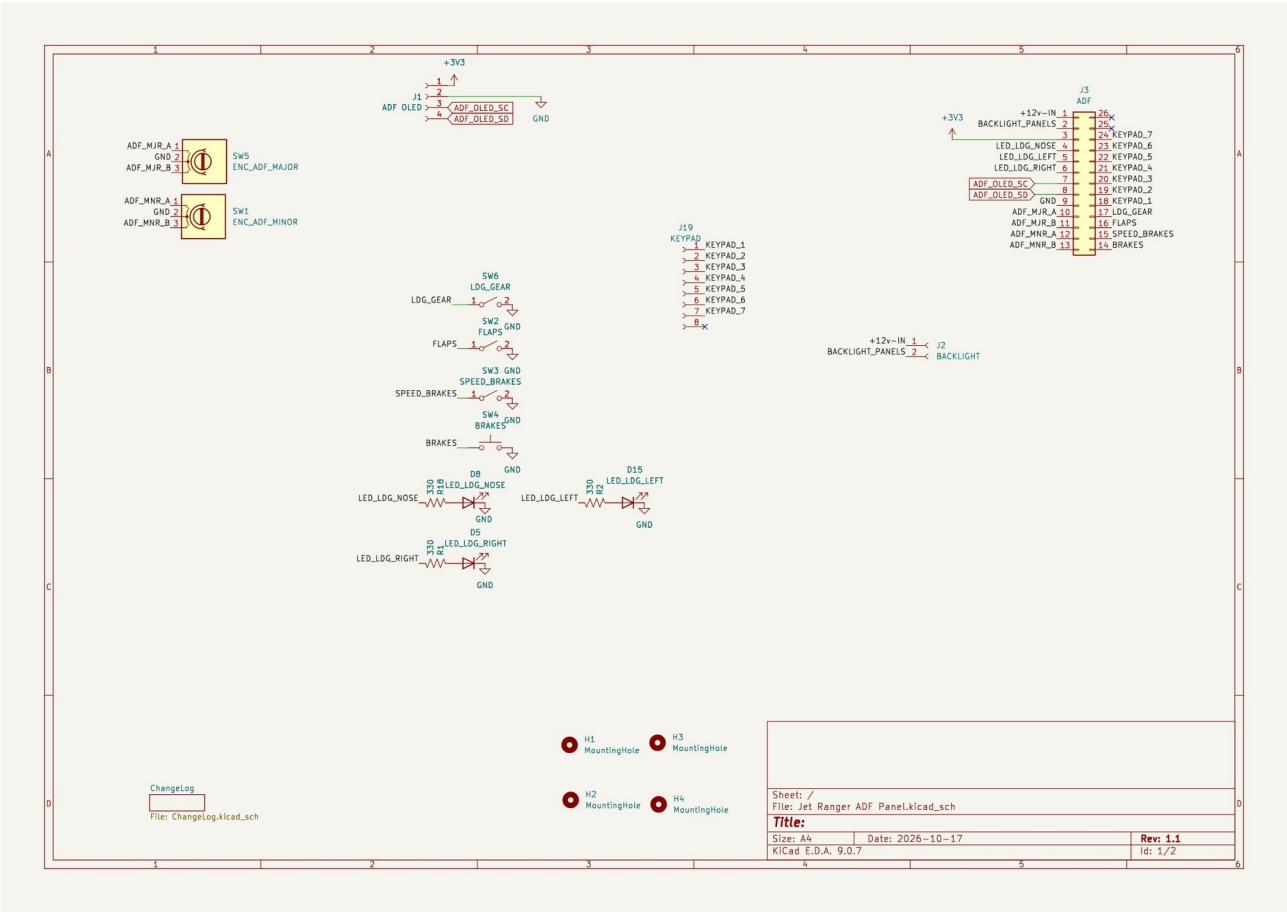



*Figure 67 — Jet Ranger ADF Panel schematic*

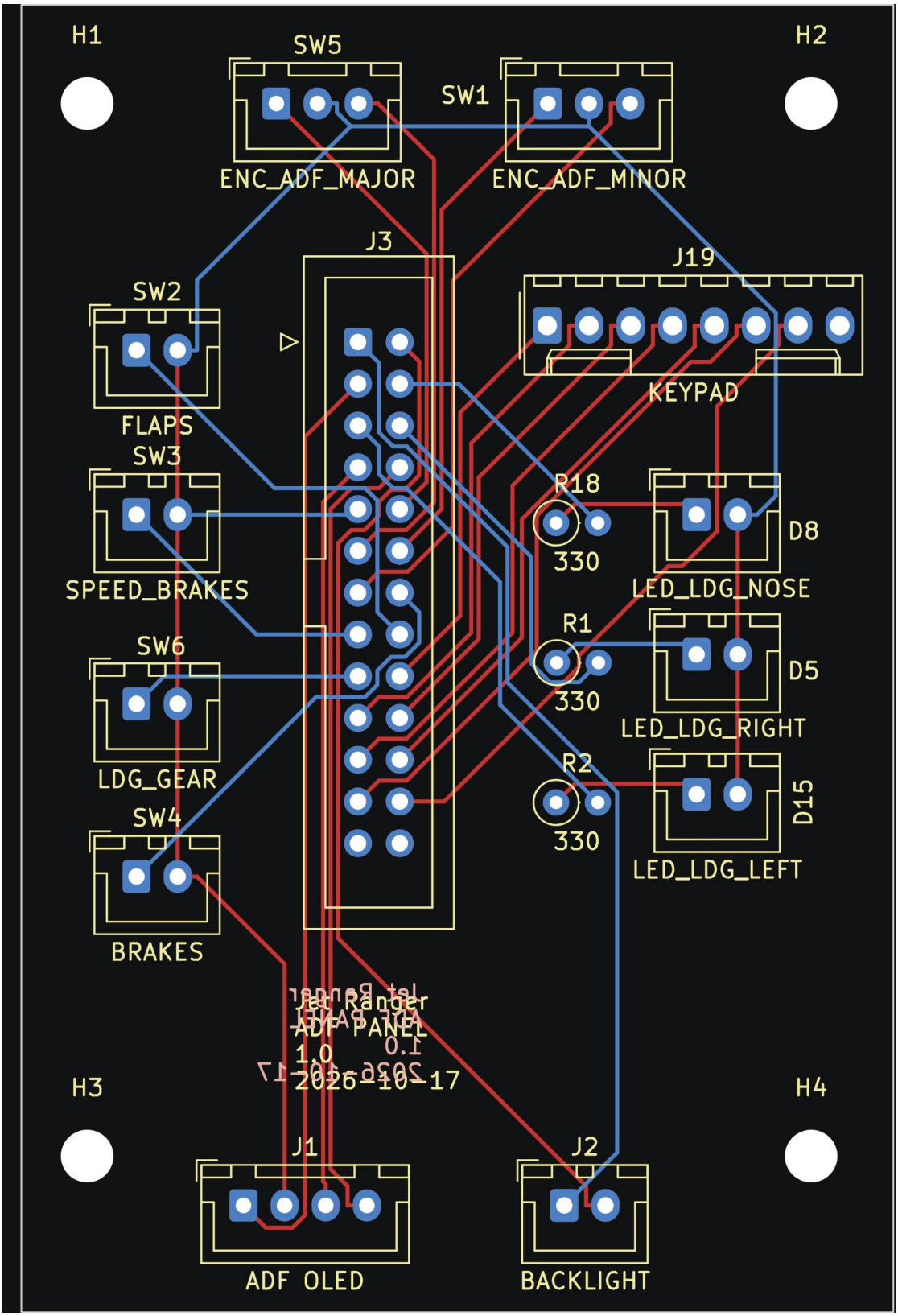



*Figure 68 — Jet Ranger ADF Panel board layout*

## 18.11 Annunciator

Schematic sheet only: the project has no board layout. The same sheet is used inside the Servo, 13 Stepper and Dual Stepper schematics. Warning lamp annunciator circuit.

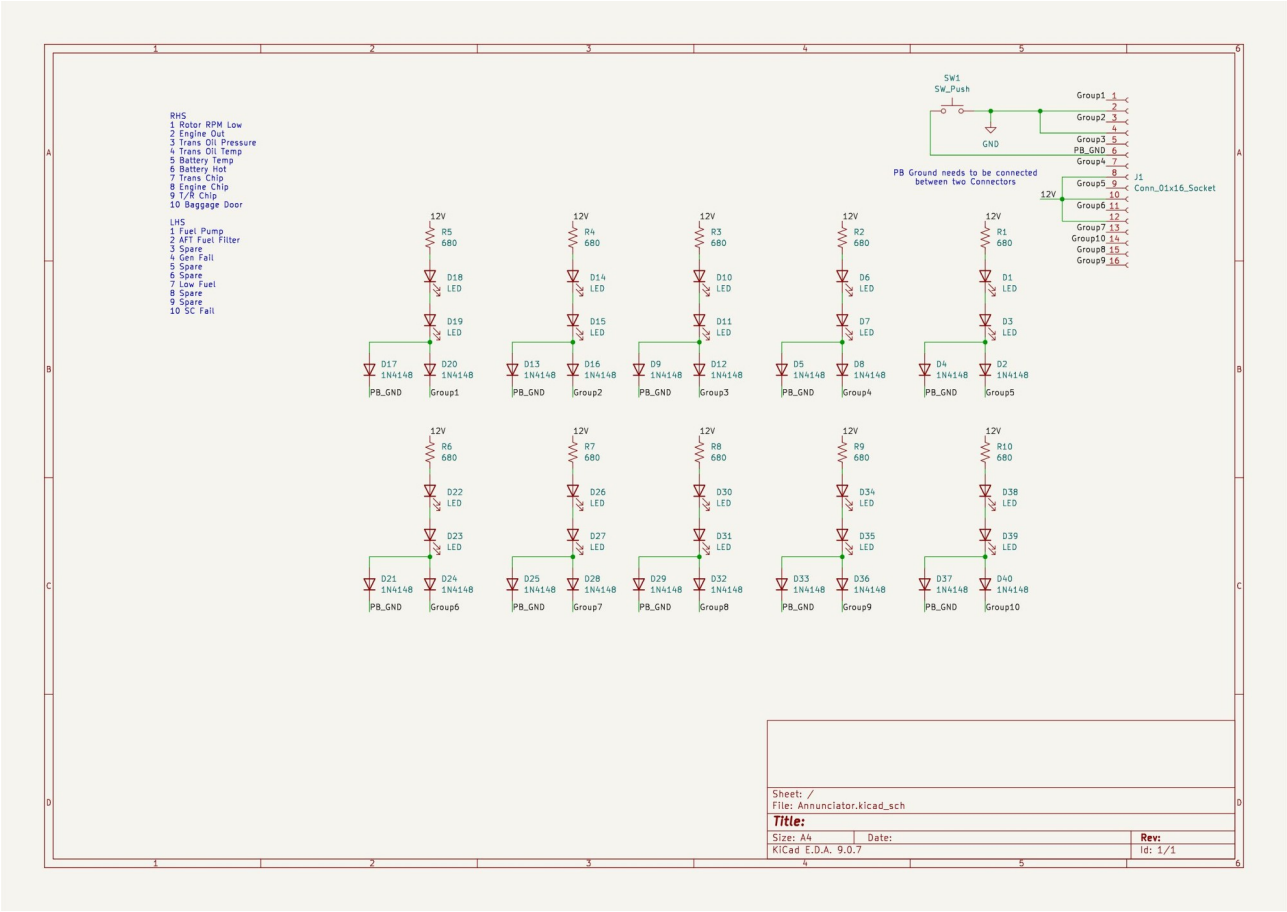



*Figure 69 — Annunciator schematic*

## 18.12 Jet Ranger Altimeter

Title block revision 2 (2022-08-04). Not covered by the sketches. Altimeter gauge board.

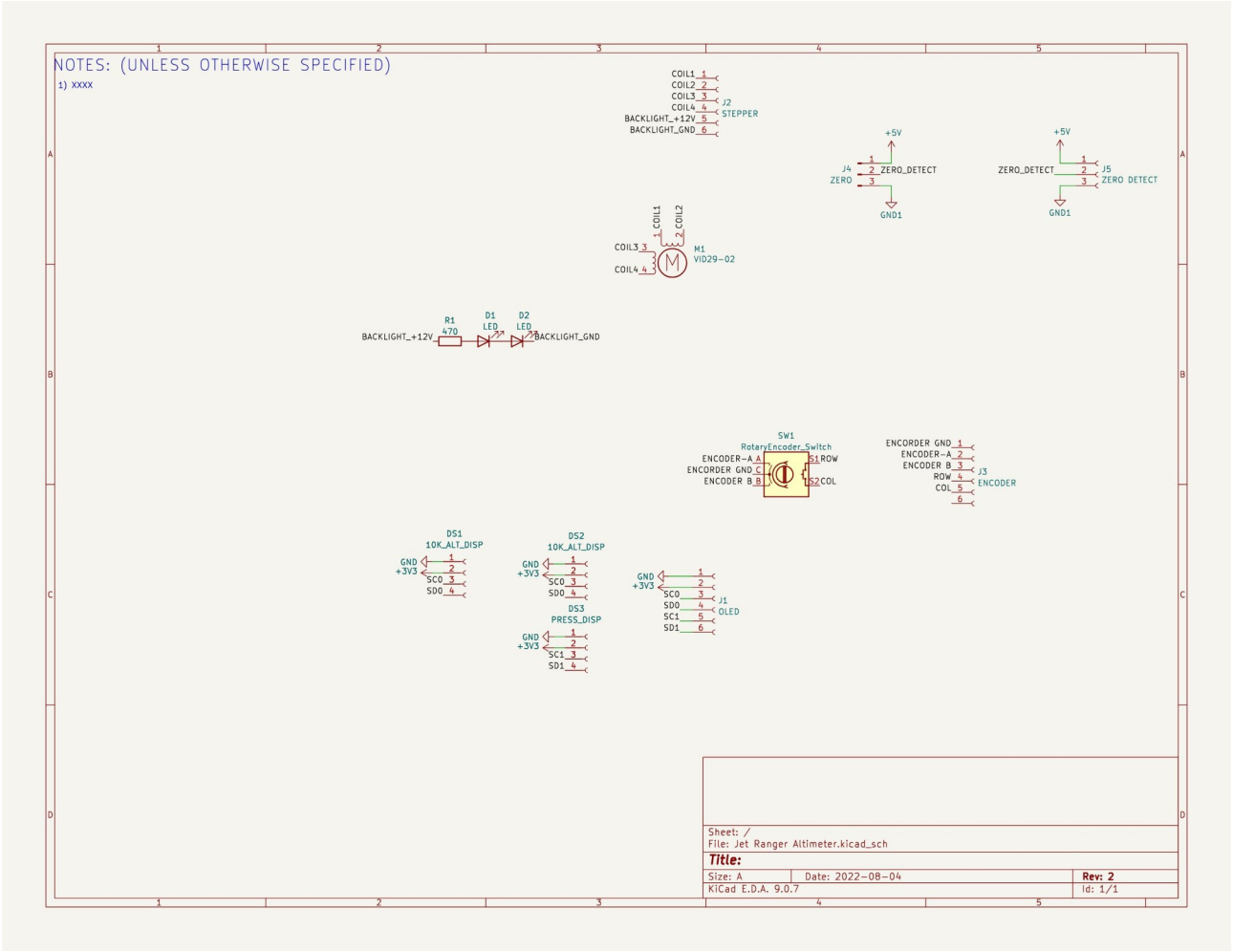



*Figure 70 — Jet Ranger Altimeter schematic*

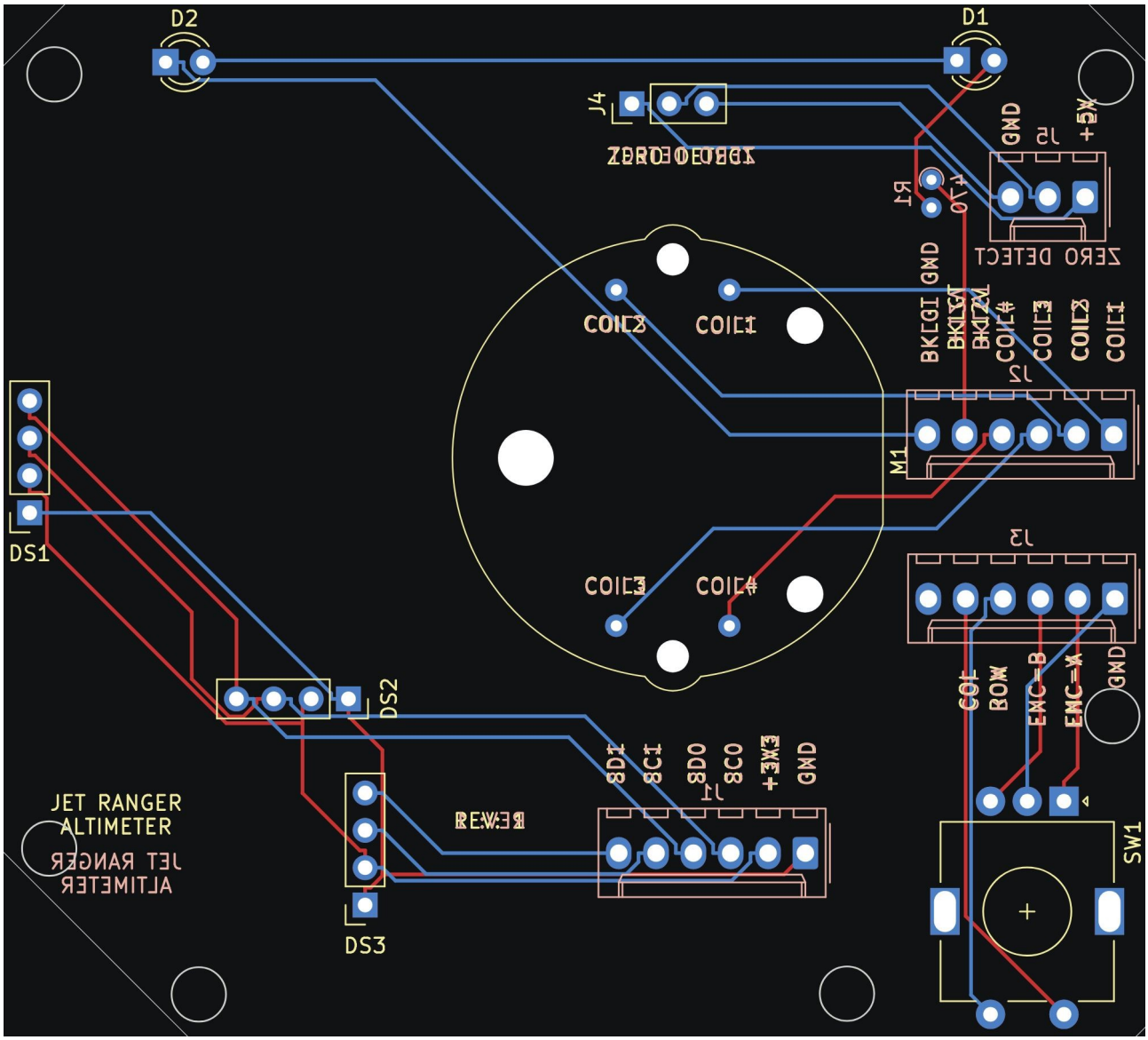



*Figure 71 — Jet Ranger Altimeter board layout*

## 18.13 Large Guage (Simmeters Large Guage Replacement)

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large Simmeters gauge.

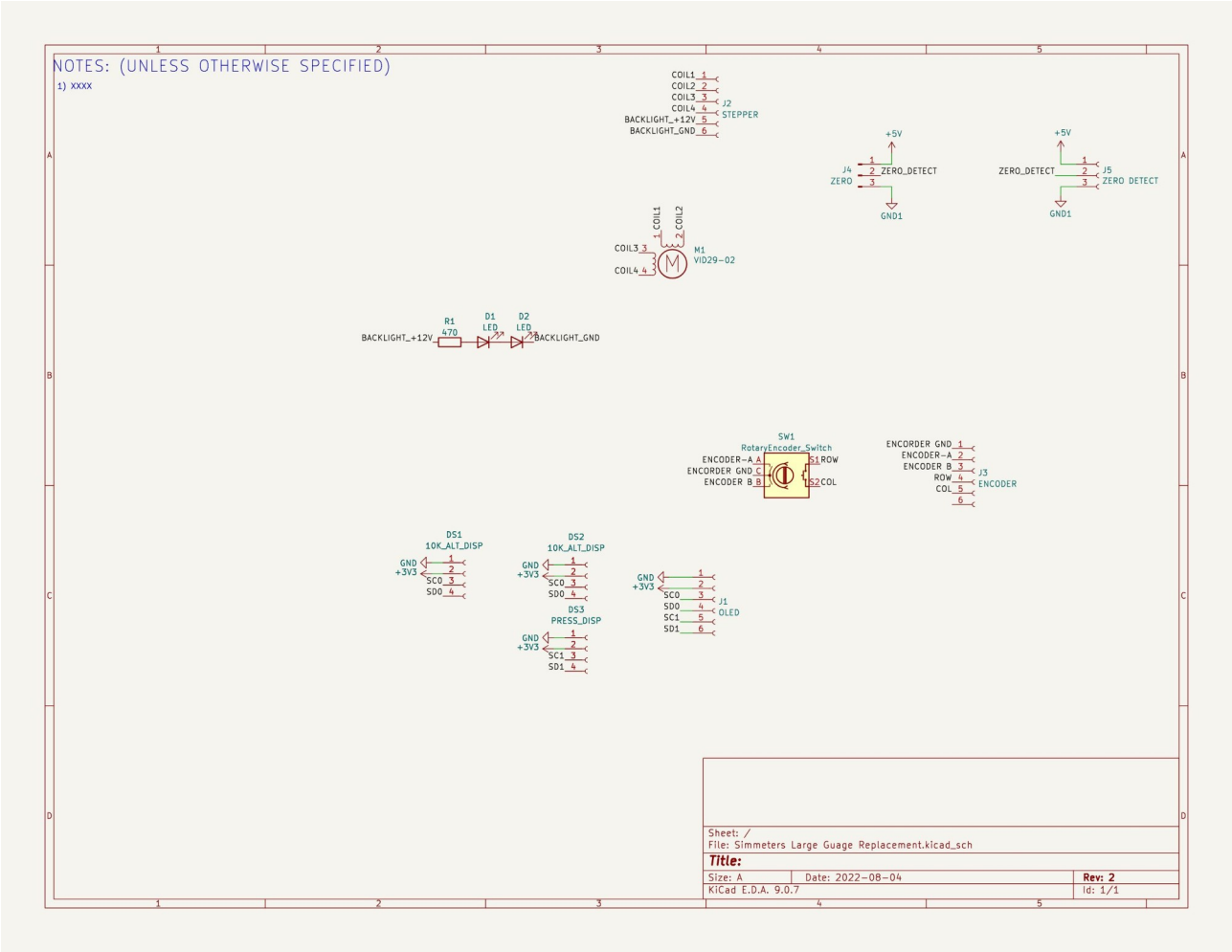



*Figure 72 — Large Guage (Simmeters Large Guage Replacement) schematic*

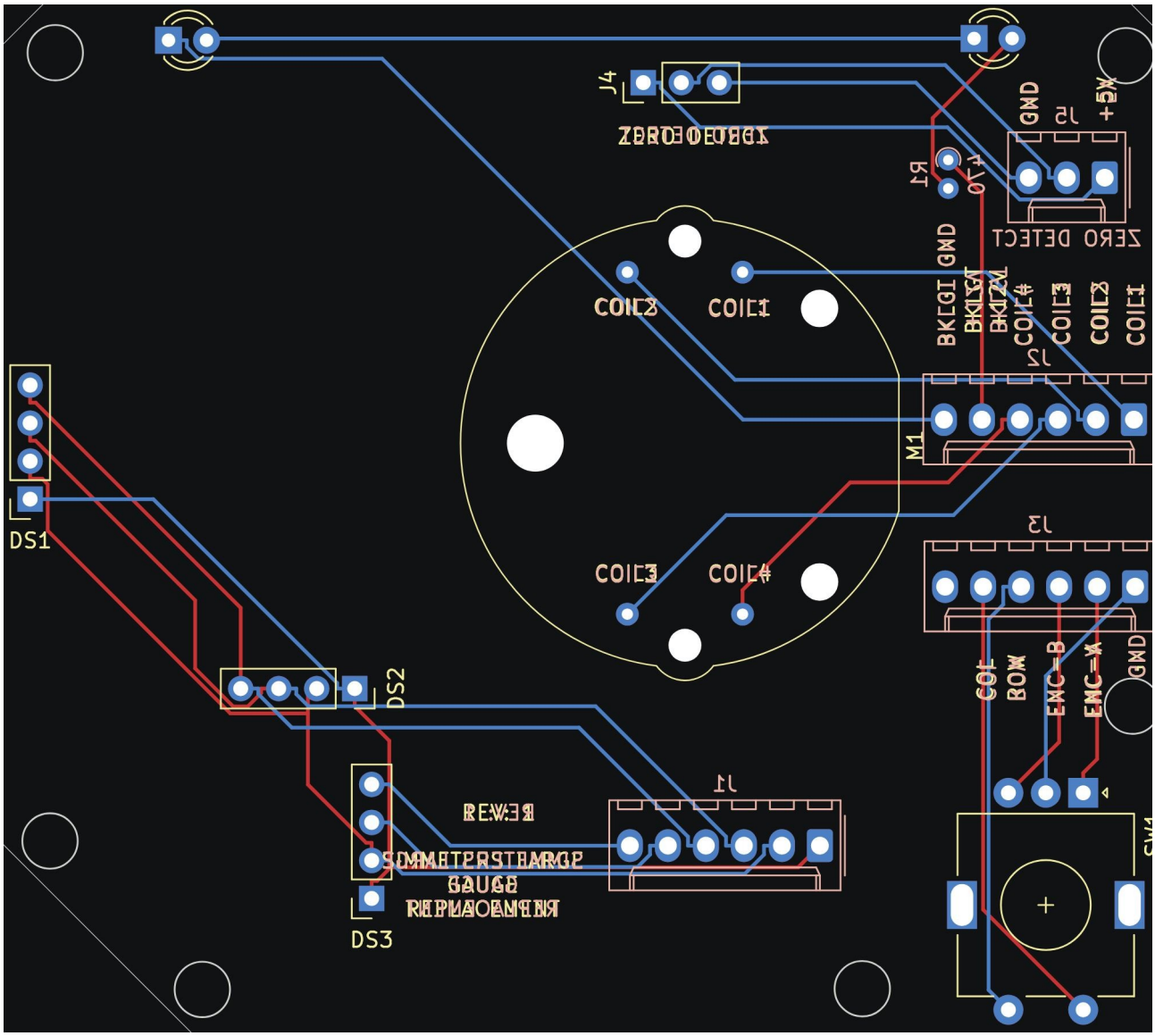



*Figure 73 — Large Guage (Simmeters Large Guage Replacement) board layout*

## 18.14 Simkits Large Dual Gauge

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large dual Simkits gauge.

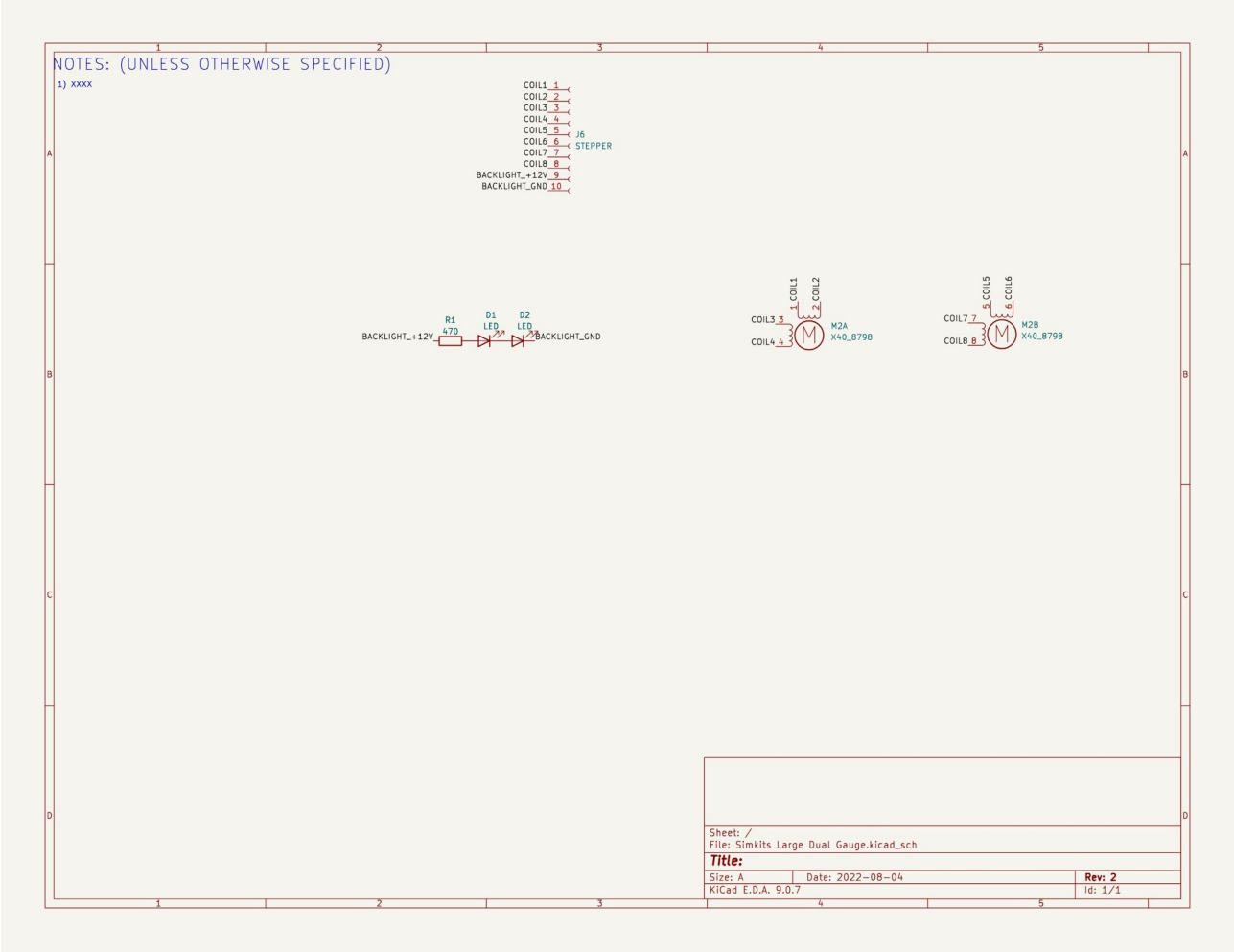



*Figure 74 — Simkits Large Dual Gauge schematic*

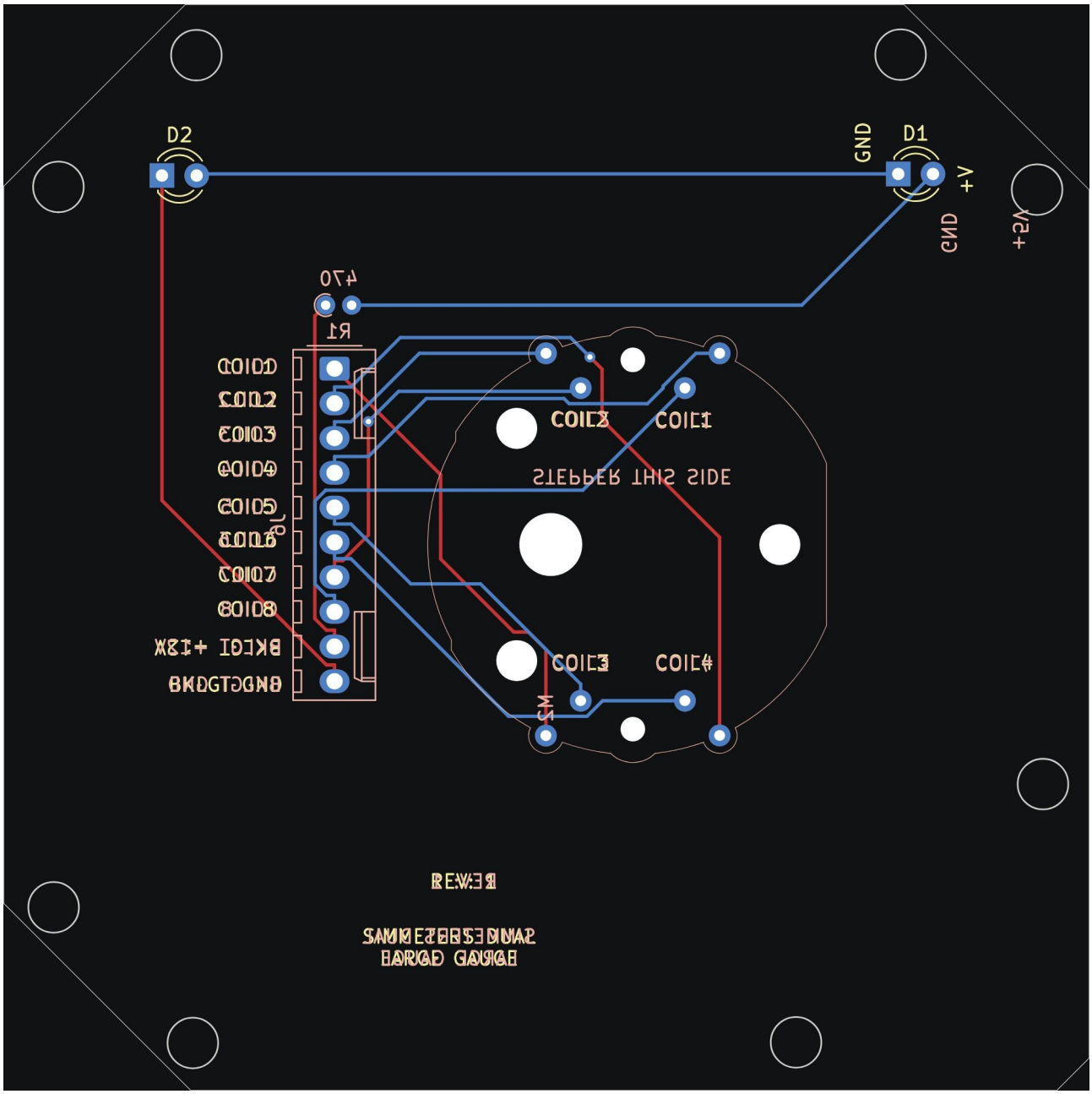



*Figure 75 — Simkits Large Dual Gauge board layout*

## 18.15 Simkits Large Guage

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large Simkits gauge.

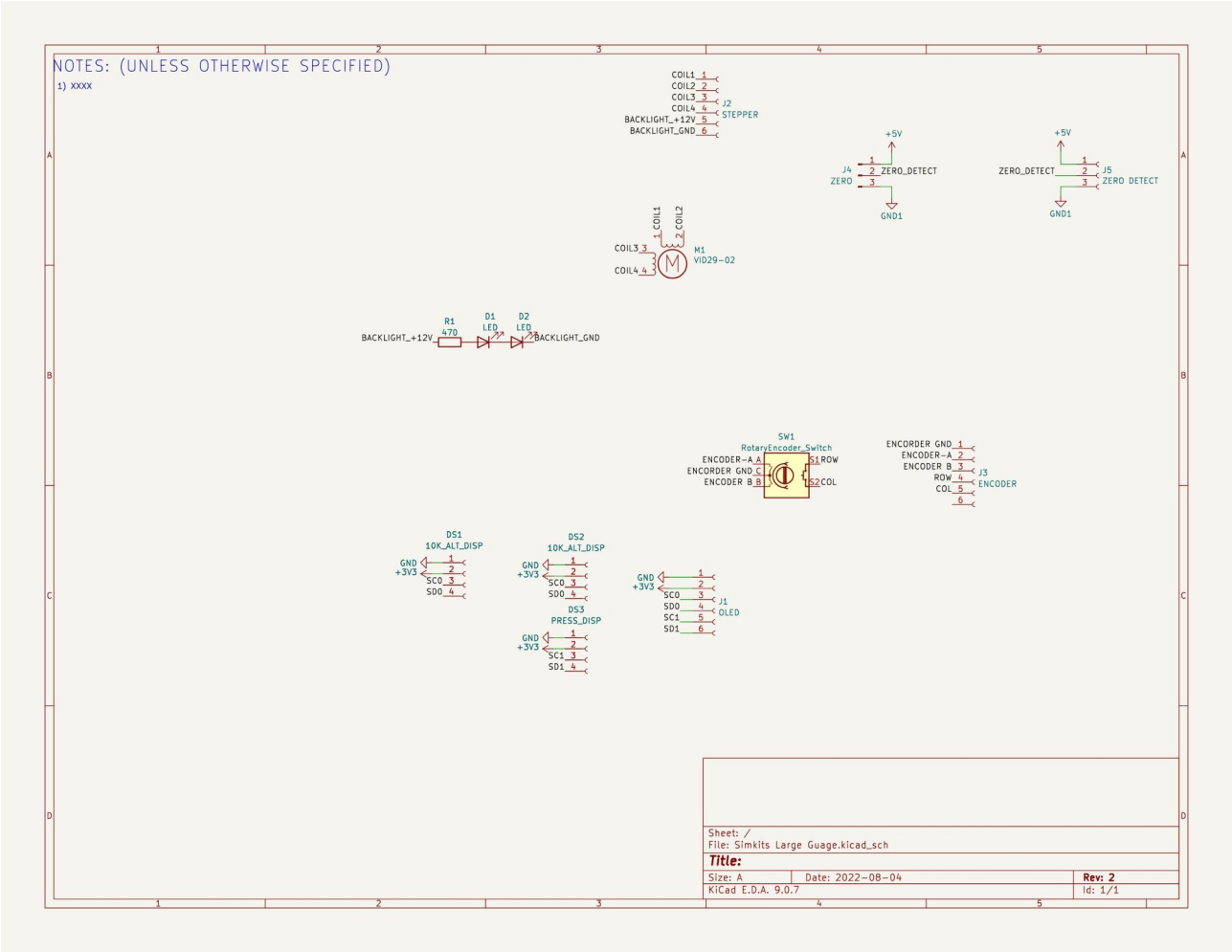



*Figure 76 — Simkits Large Guage schematic*

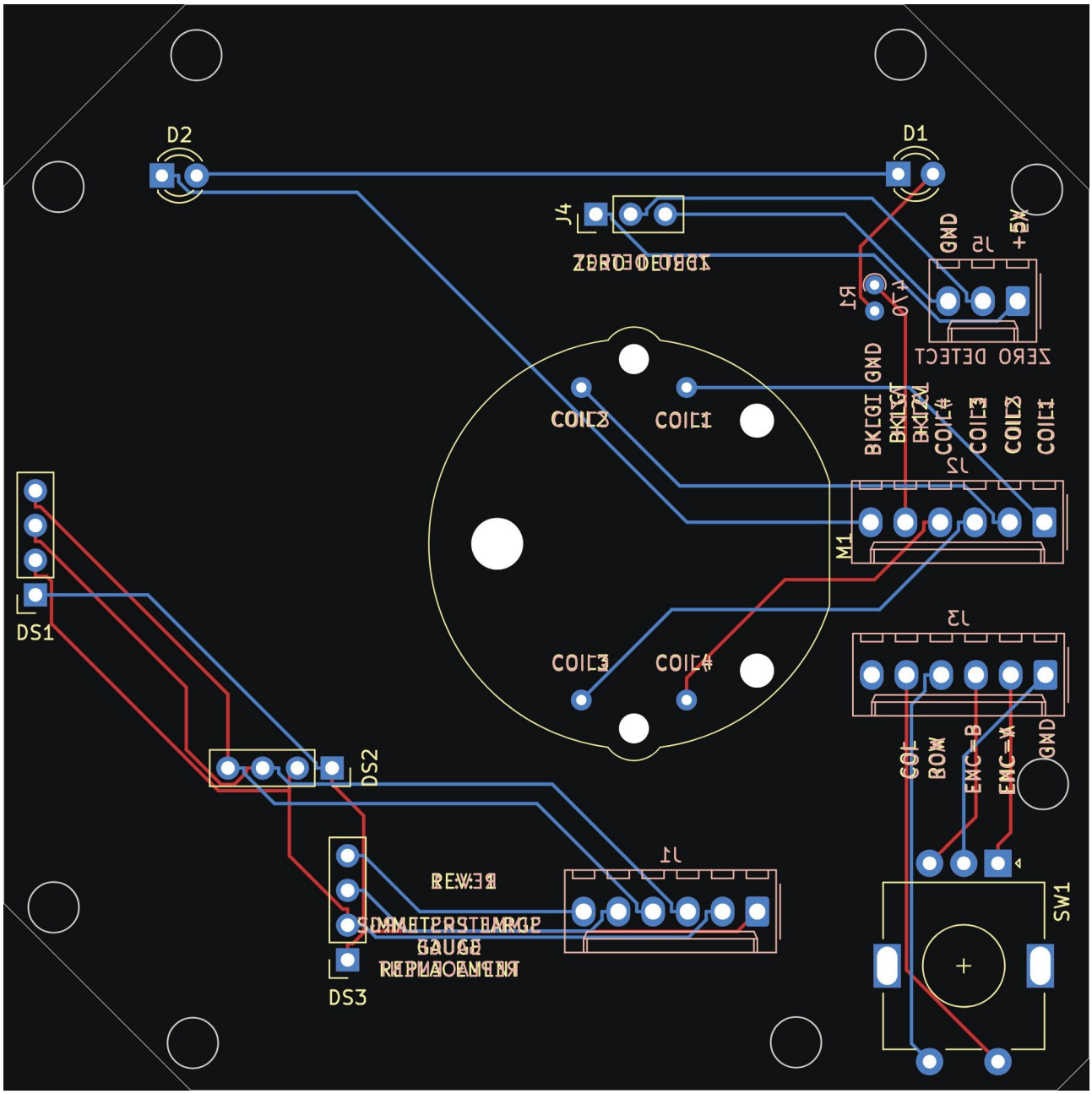



*Figure 77 — Simkits Large Guage board layout*

## 18.16 Simkits Small Dual Gauge

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a small dual Simkits gauge.

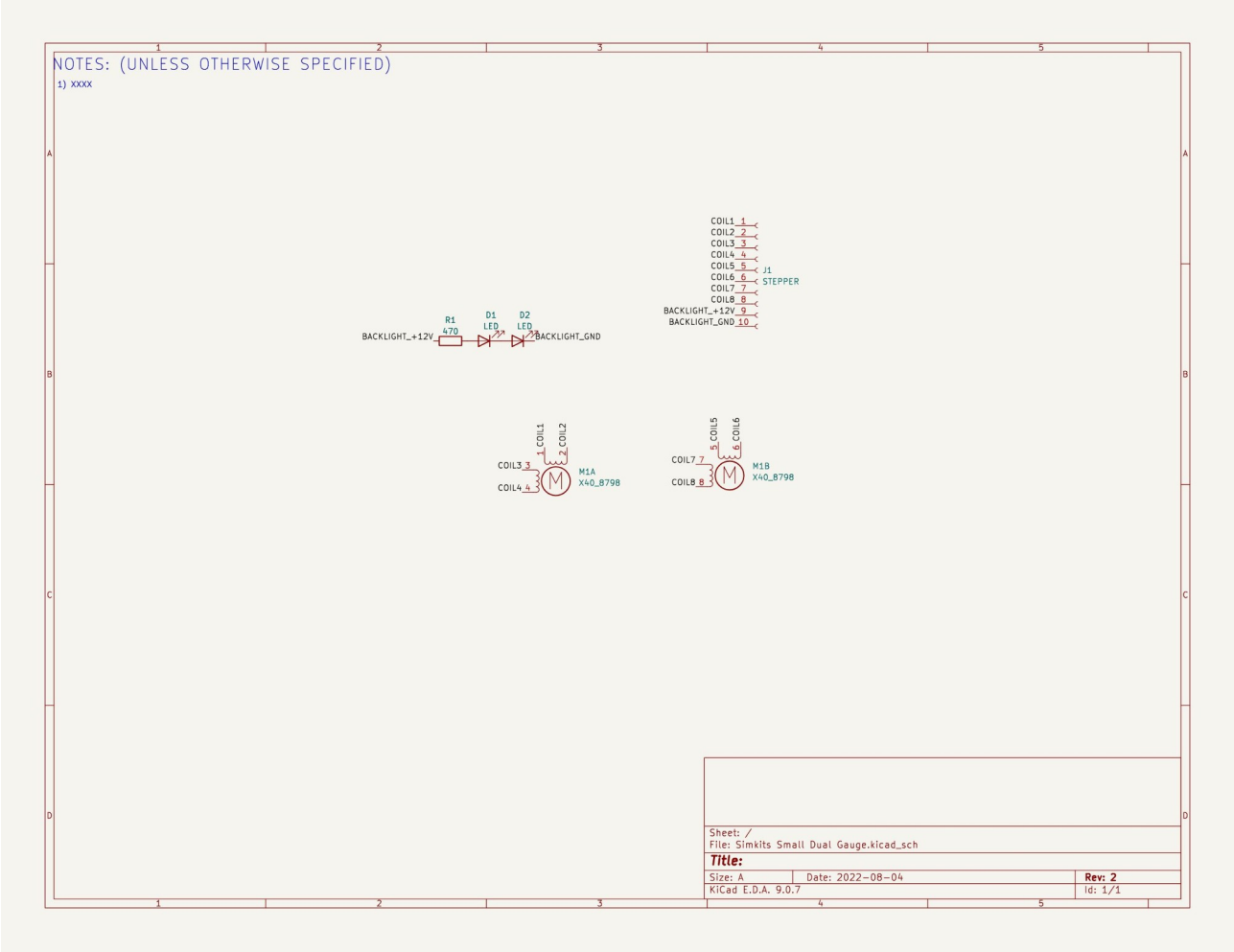



*Figure 78 — Simkits Small Dual Gauge schematic*

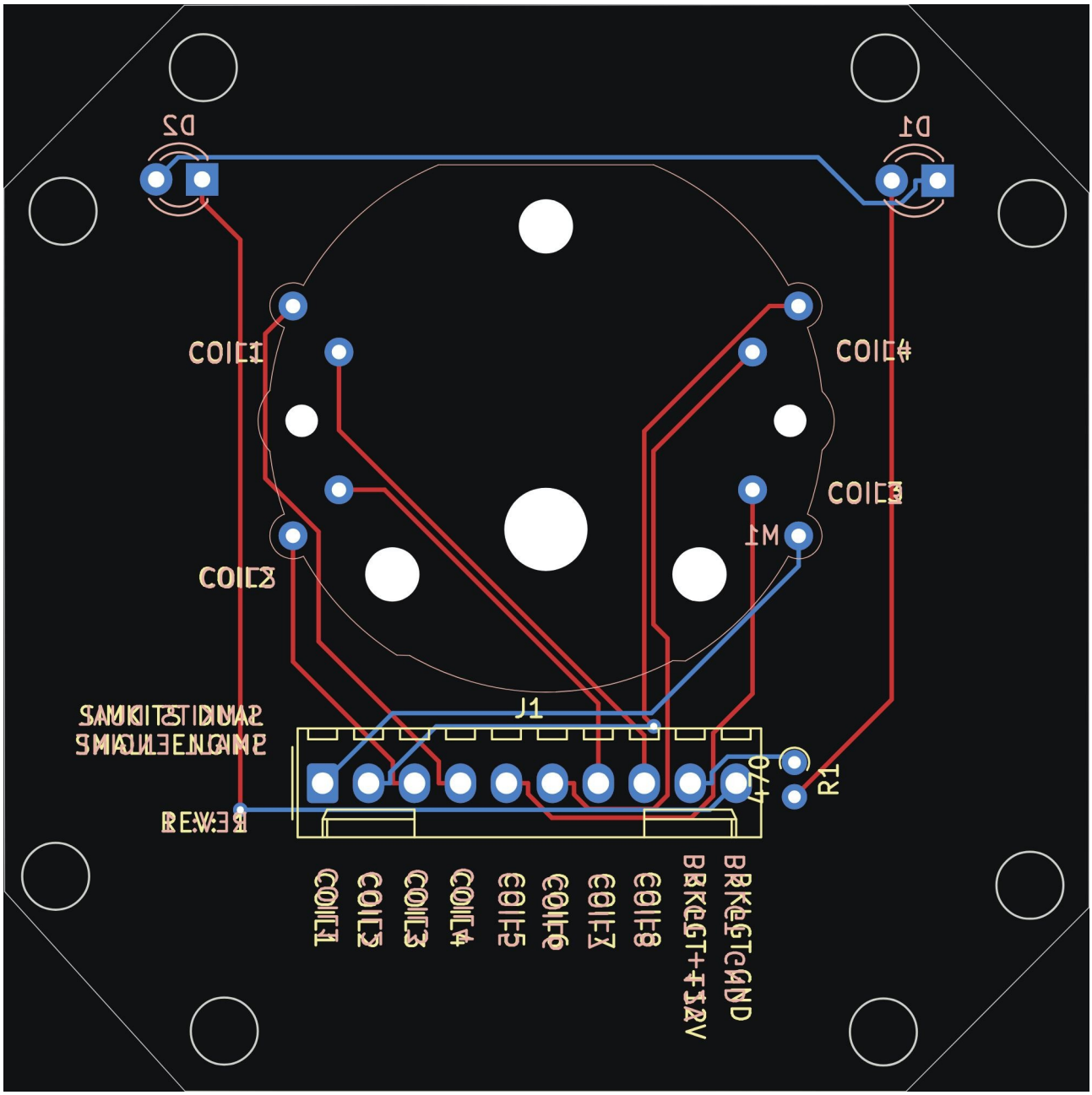



*Figure 79 — Simkits Small Dual Gauge board layout*

## 18.17 Simkits Small Guage

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a small Simkits gauge.

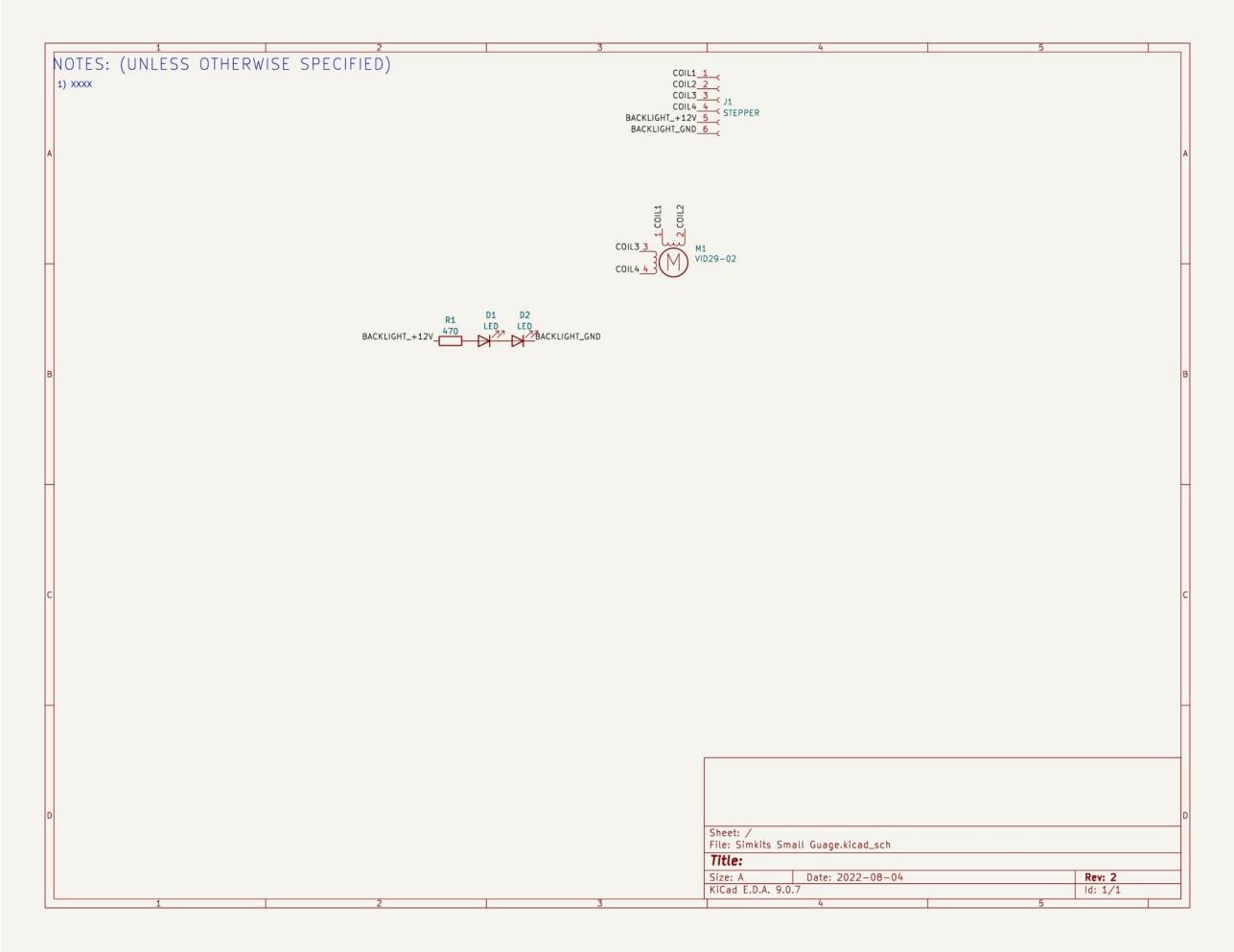



*Figure 80 — Simkits Small Guage schematic*

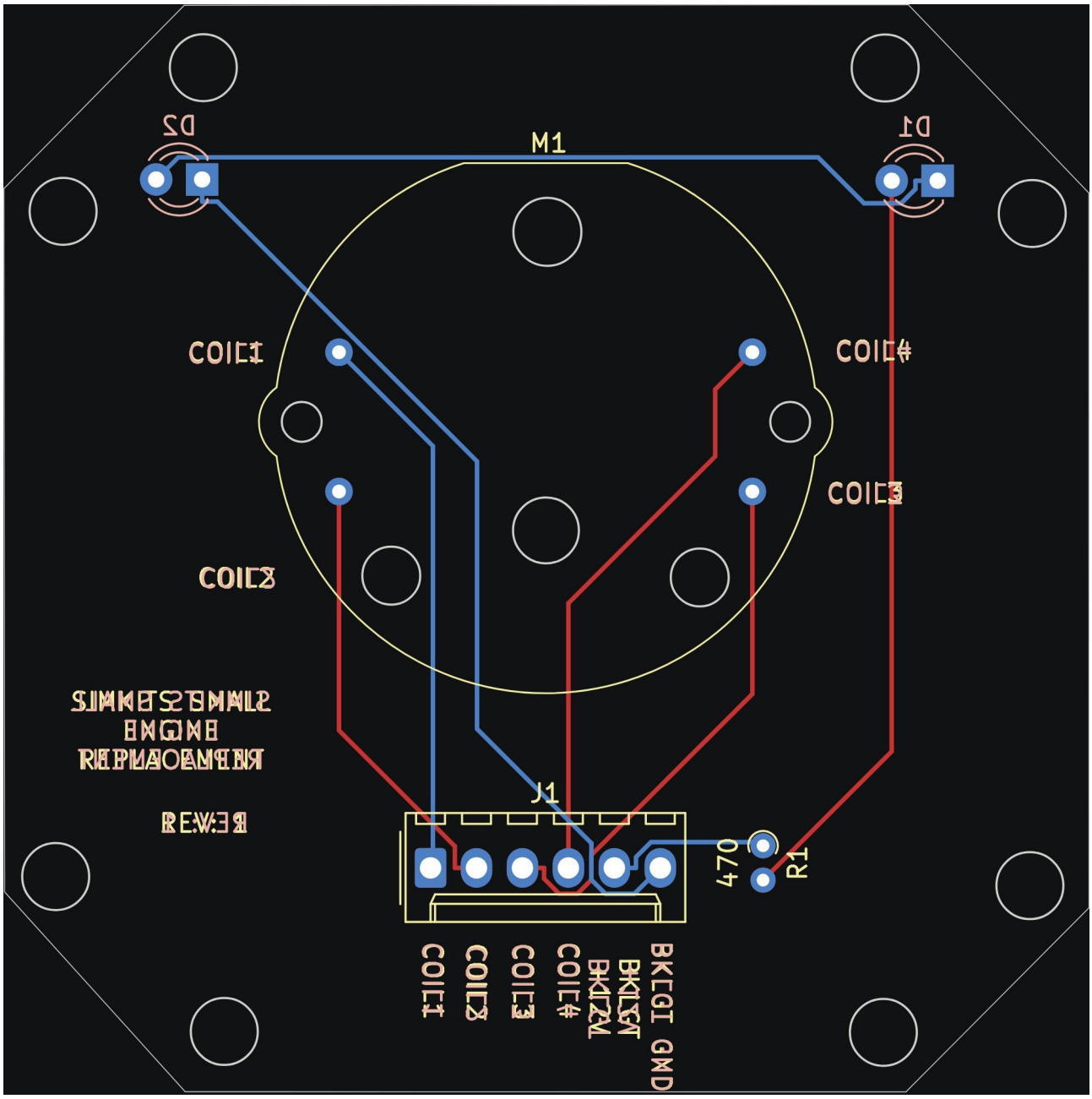



*Figure 81 — Simkits Small Guage board layout*